Demo of `abmneo()` function

Sasha D. Hafner

13 March, 2026 06:42

Overview

This demo is meant to explain the features of the package prior to its release.

What's different?

The abmneo package was meant to address some issues with ABM:

1. complexity (some deliberate, some not),
2. hard-coded behavior and special cases,
3. number of dependencies,
4. lack of verification,
5. size of codebase,
6. use of Rcpp, and
7. model speed.

On 1, the flexibility of ABM and the number of processes was all deliberate. These are not inherently bad things, but they did complicate the model and especially the code, making it difficult to understand, which *is* bad. Complexity also makes code verification, debugging, and maintenance difficult. At some point in the future, as dependencies change (see point 3 as well), it can become too difficult.

Hard-coded behavior and special cases, such as the distinctions between methanogens and sulfate reducers, with their element names indicating what they were and which code should be used for them in `rates()` and probably elsewhere makes for complex, inefficient, and fragile code. By fragile (a word I picked up from Claude) I mean likely to cause errors depending on inputs. Inhibition and respiration are two other areas where there was hard-coded behavior and special cases.

Dependencies (3) is actually not much of an issue. We had stats, dplyr (not a big fan), and of course Rcpp. If we can avoid them, why not? Rcpp is discussed below–that is significant, I think.

Verification is something that has really worried me. ABM (and abmneo) is complex enough that mistakes in mass balance or elsewhere are quite likely. So verification is important.

The size of the codebase (another term from Claude) alone also contributes to difficulty in understanding the package, and it seems like the possibility of holding most of even just the most important parts of the code in a developer's mind is impossible. I gave up trying to follow the logic on more than one occasion.

Using Rcpp was of course done for speed, but it complicates the package with respect to debugging and development in general, and increases the requirements for installation. If it could be avoided without a speed penalty, wonderful.

Of course speed has always been an issue. My hope was to not make it worse.

So, how is abmneo different? I think I made significant progress on most of those points:

1. model and codebase are both much simpler,

2. eliminated most hard-coding; even microbial groups can have any metabolic pathways and any names,
3. deSolve is the only dependency,
4. COD balance is built in (could still be improved/extended),
5. size of the codebase is half as large and the `rates()` function in particular is shorter and clearer,
6. Rcpp is no longer used, but
7. the `abmneo()` function *seems* quite fast.

Some of these changes are clear improvements (thinking about how microbial groups are handled now). Others mean fewer features and processes. These are summarized in the demos below.

Prep

```
devtools::load_all()
```

```
## i Loading abmneo
```

Some default parameters

First substrate parameters, a new argument compared to ABM(). This defines substrates. We could have any number with any names but will just use the old `VSd` here. Note that hydrolysis uses CTM again.

```
sub_ref <- list(
  subs = c('VSd'),
  T_opt_hyd = c(VSd = 60),
  T_min_hyd = c(VSd = 0),
  T_max_hyd = c(VSd = 90),
  hydrol_opt = c(VSd = 0.1),
  sub_fresh = c(VSd = 30),
  sub_init = c(VSd = 30)
)
```

Microbial parameters are similar to other ABM versions. But now we have microbial stoichiometry instead of the "special cases" that complicate the code. The ks par is now in a matrix, which could have more than one row when there are multiple substrates in a reaction (e.g., with sulfate reduction).

```
mstoich_ref <- matrix(
  c(-1, -1, -1, 1, 1, 1),
  nrow = 2,
  byrow = TRUE,
  dimnames = list(
    c('VFA', 'CH4'),
    c('m0', 'm1', 'm2')
  )
)

mstoich_ref
```

```
##     m0 m1 m2
## VFA -1 -1 -1
## CH4  1  1  1
```

```
ksmat_ref <- matrix(
  c(0.5, 0.5, 0.5),
  nrow = 1,
  byrow = TRUE,
```

```
  dimnames = list(
    c('VFA'),
    c('m0', 'm1', 'm2')
  )
)

ksmat_ref
```

```
##      m0  m1  m2
## VFA 0.5 0.5 0.5
```

With a COD basis for substrates and CH4 internally, stoichiometric coefficients are usually simple, as above. These matrices are put into `grp_pars` along with recognizable elements. Note that CTM is used for all termperature responses.

```
grp_ref <- list(
  grps = c('m0', 'm1', 'm2'),
  yield = c(default = 0.05),
  xa_fresh = c(default = 0.05),
  xa_init = c(default = 0.05),
  dd_rate = c(default = 0.02),
  qhat_opt = c(m0 = 0.5, m1 = 1, m2 = 1),
  T_opt = c(m0 = 10, m1 = 18, m2 = 28),
  T_min = c(m0 = 0, m1 = 5, m2 = 10),
  T_max = c(m0 = 25, m1 = 25, m2 = 38),
  ksmat = ksmat_ref,
  mstoich = mstoich_ref
)
```

The `dd_rate` parameter is for "death and decay".

These last two parameter lists here are similar to other versions. VFA is hard-coded as an intermediate. The name `VFA` is used internally and in output. No name is set here in the elements. This `inf_pars` holds what is left of the original `man_pars`. The name is for *inflow*, meant to be more generic than manure.

```
inf_ref <- list(VFA_fresh = 2, VFA_init = 2, pH = 7, dens = 1000)
```

The chem_pars list now specifies which components are emitted as gases and which are otherwise solutes. Gases have no association with the slurry like solutes or substrates do–their production is taken as emission and cumulative values are tracked.

```
chem_ref <- list(COD_conv = c(CH4 = 5.32), gases = c('CH4'))
```

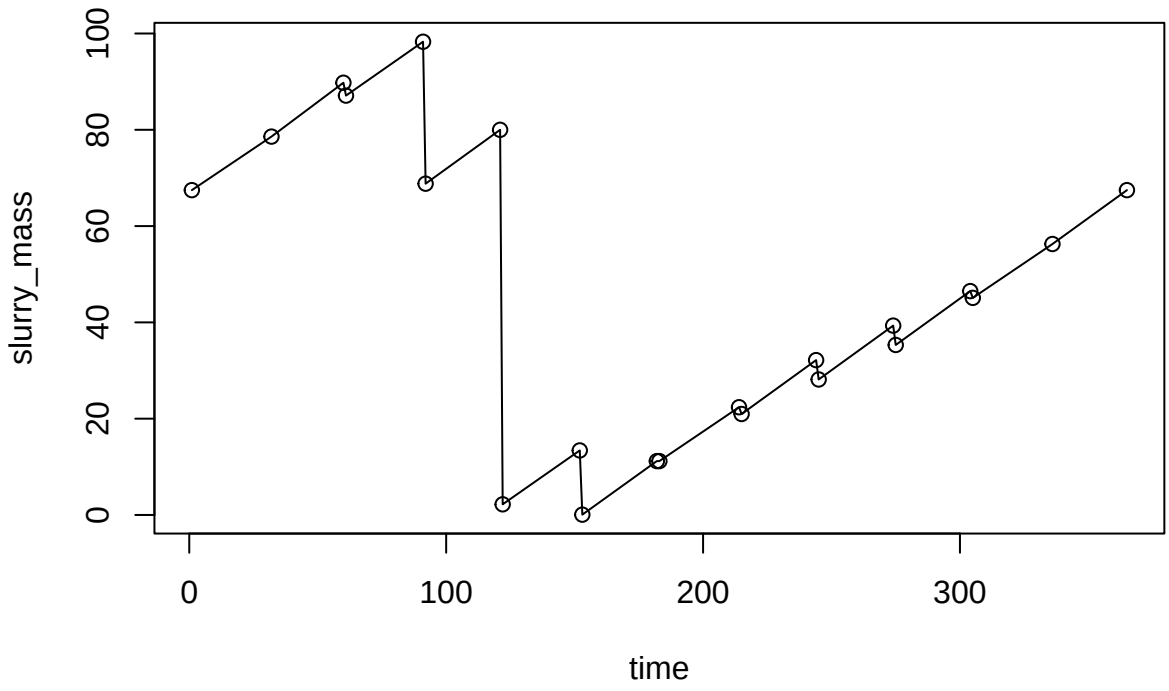
Conversion factor from: `calc_COD('CH4') / mol_mass('C')` from the `mic-stoich` repo. It is g COD / g CH4-C.

Slurry mass series versus regular inputs

When it comes to how slurry level and the fill/empty schedule is entered, we now distinguish between *regular* inputs (as in repeating/consistent, as in earlier versions of `abmneo()`) and *series* inputs, where the schedule is inferred from time series data on slurry mass (what was called time-variable in earlier versions of `abmneo()`). These are specified in the `storage` argument. The name indicates that this argument describes the storage structure. The slurry mass data frame for storage only has time and slurry_mass. It is no longer a part of `var_pars`–the two are now independent (at least externally).

We'll work on a single pig scale here with some normal outdoor tank levels from a spreadsheet calculator from Peter Kai.

```
mass_series <- read.csv('level_pig_od_ref.csv')
plot(slurry_mass ~ time, data = mass_series, type = 'o')
```



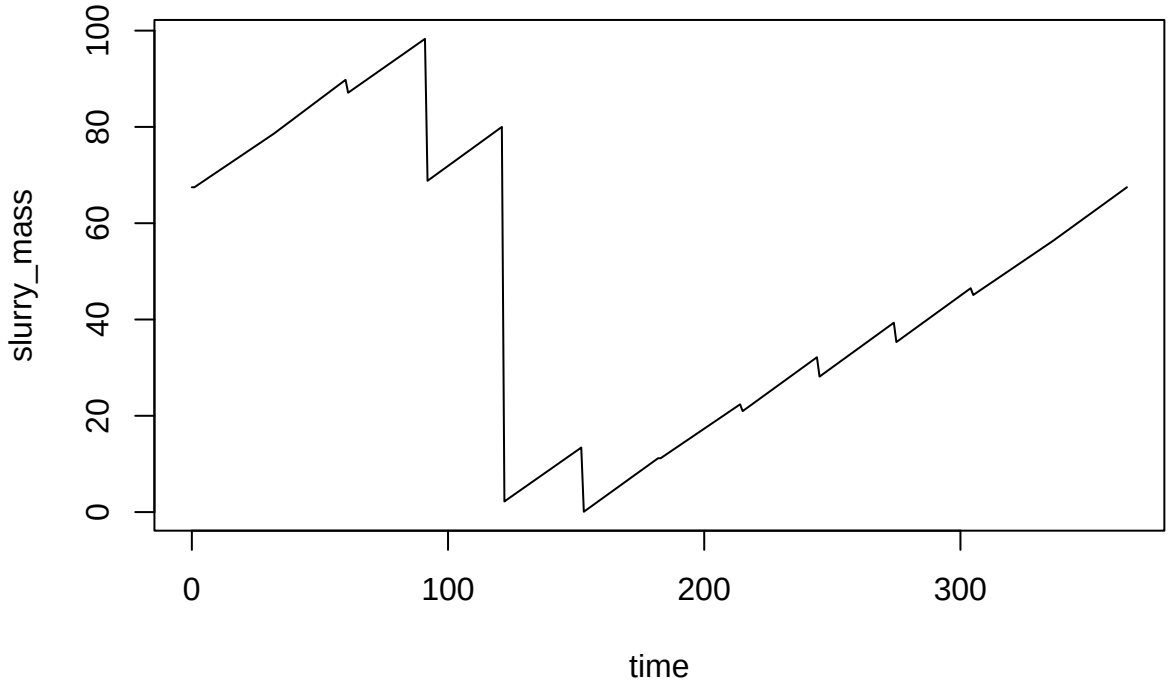

```
stor_ser_ref <- list(
  type = 'series',
  dat = mass_series,
  storage_depth = 2,
  area = 0.7,
  temp_C = 18,
  resid_enrich = 0.2
)
```

Unlike `ABM()`, no `var_pars` is needed. And the fixed inputs like `slurry_prod_rate` that were ignored in these cases with `ABM()` are no longer expected. (But the `storage` argument expects these when `type = 'regular'`.)

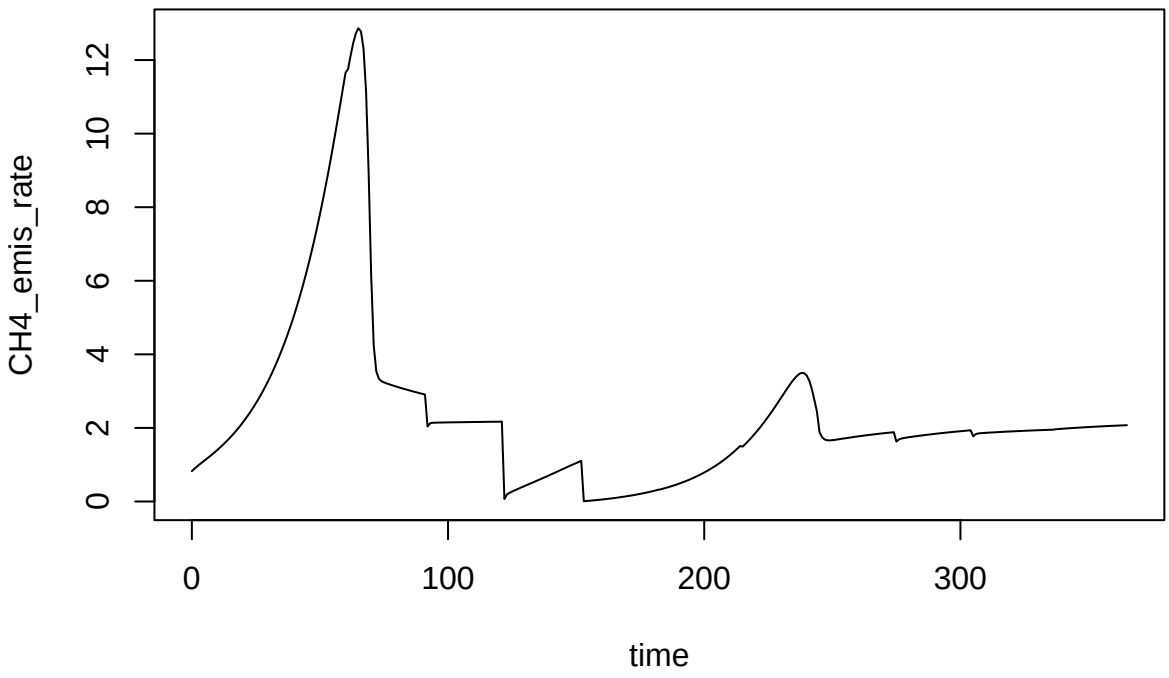
```
outpodr <- abmneo(
  storage = stor_ser_ref,
  365,
  inf_pars = inf_ref,
  grp_pars = grp_ref,
  sub_pars = sub_ref,
  chem_pars = chem_ref
)
```

```
## Warning in check_COD(dat = dat, grps = pars$grps, subs = pars$subs, COD_conv =
## pars$COD_conv, : COD balance is off by 2.7%
```

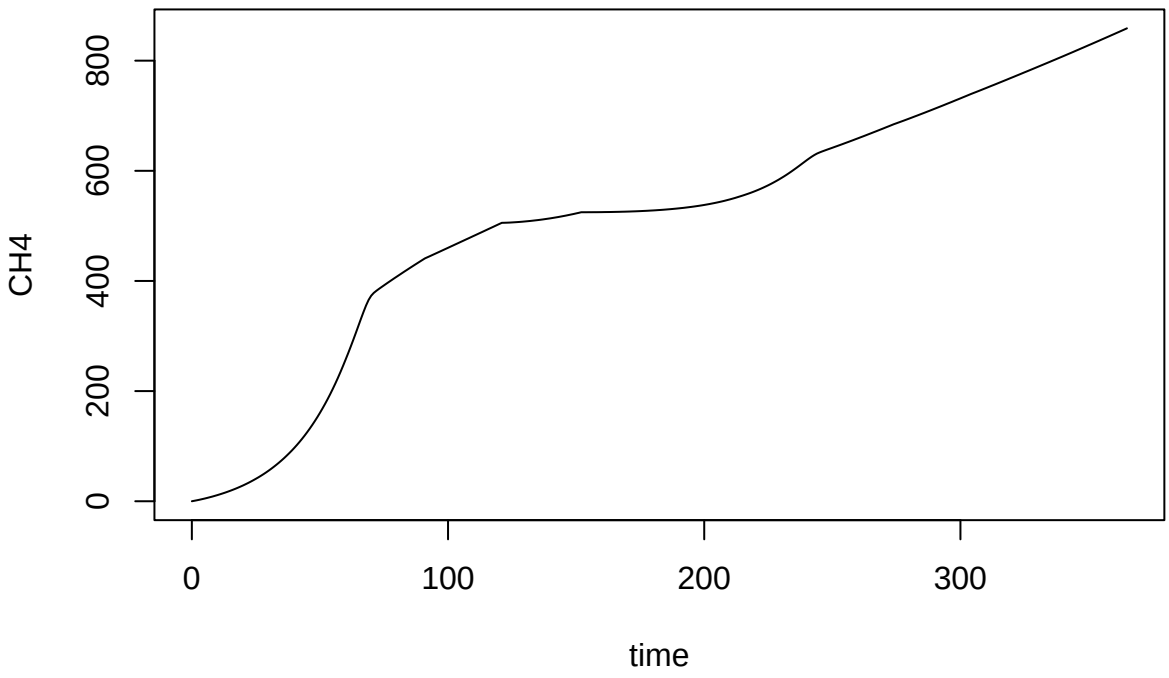
```
plot(slurry_mass ~ time, data = outpodr, type = 'l')
```



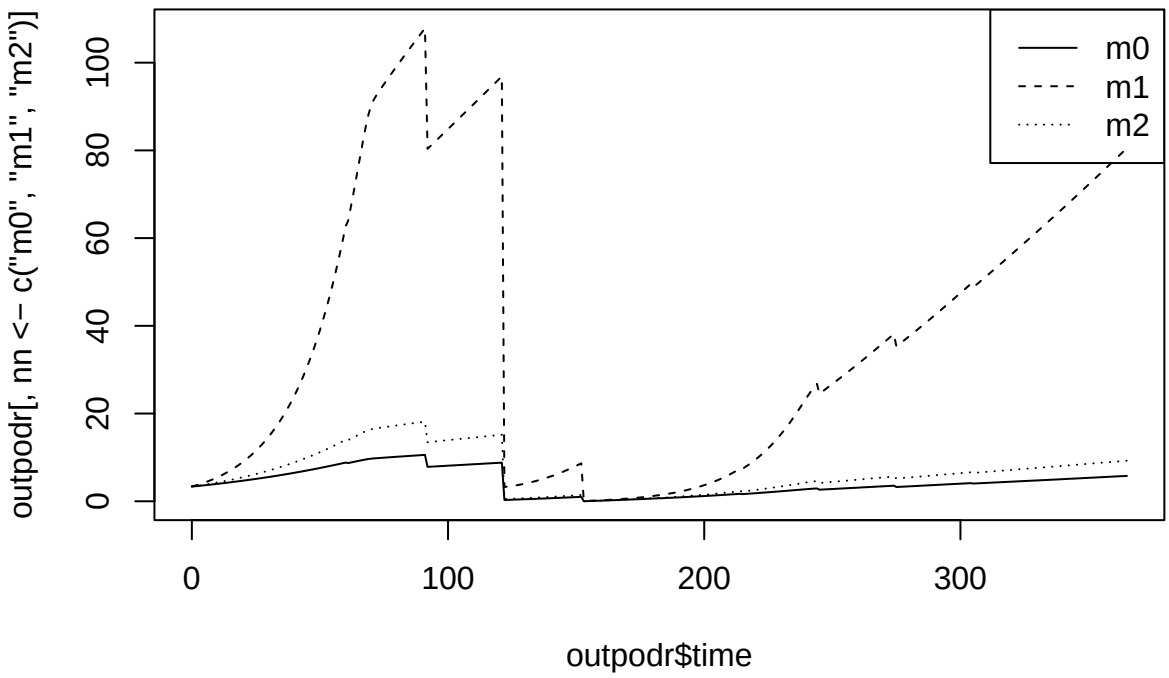

```
plot(CH4_emis_rate ~ time, data = outpodr, type = 'l')
```



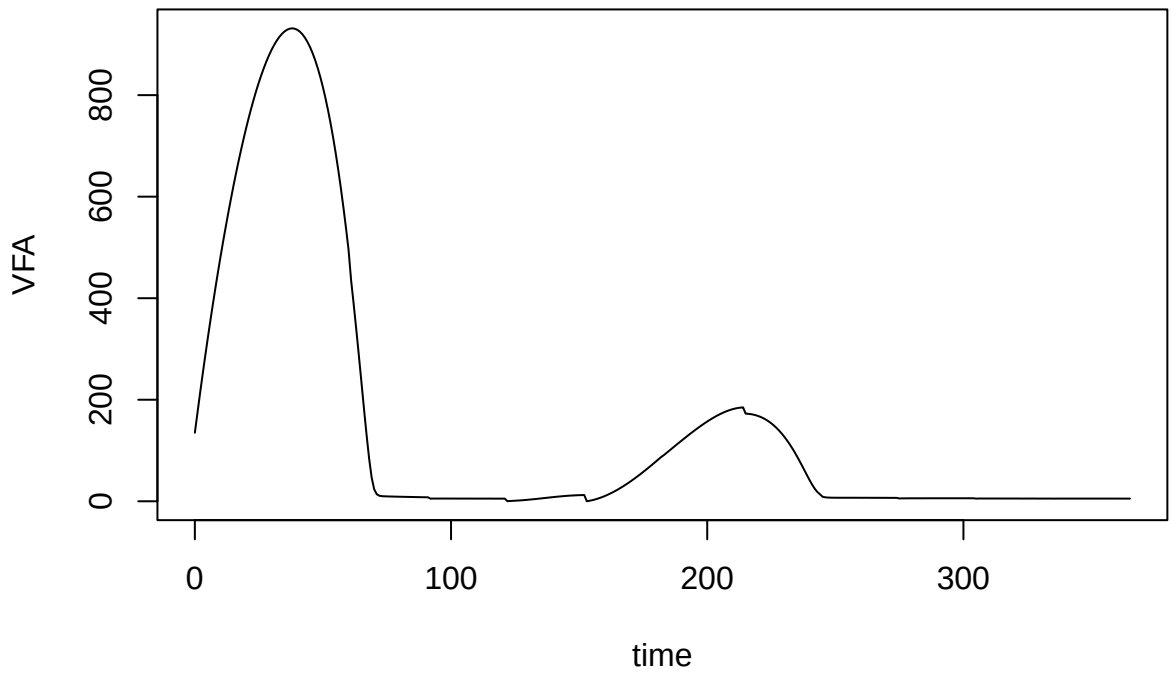

```
plot(CH4 ~ time, data = outpodr, type = 'l')
```



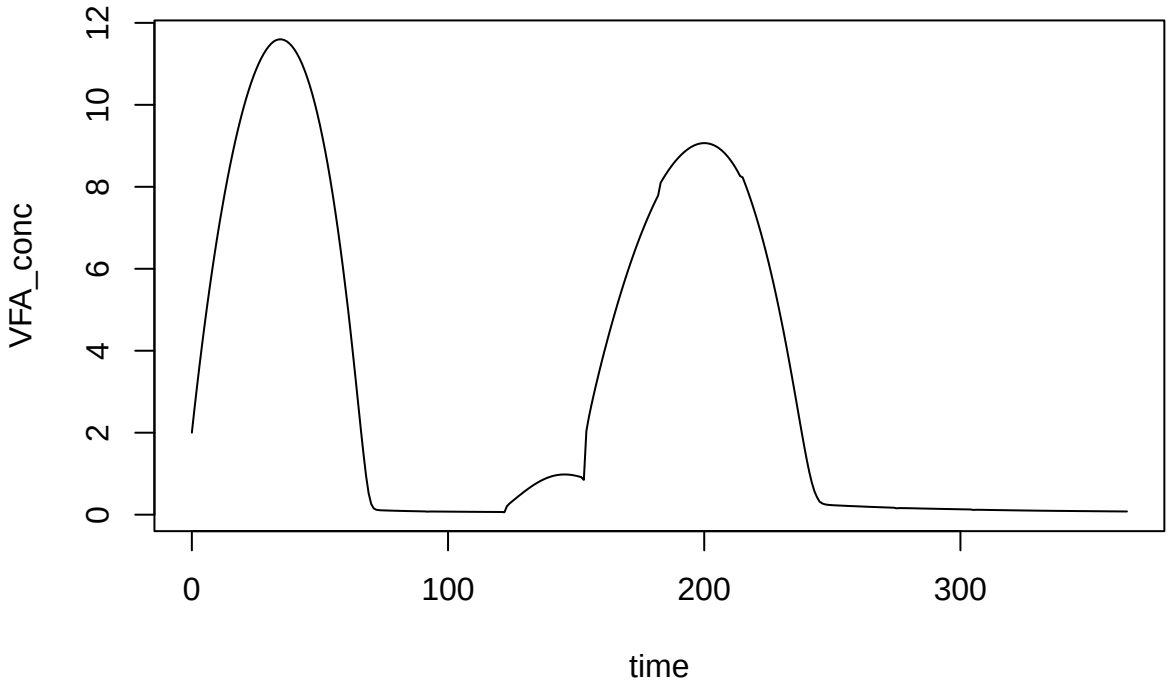

```
matplot(outpodr$time, outpodr[, nn <- c('m0','m1','m2')], col = 'black', lty = 1:length(nn), type = 'l')
legend('topright', legend = nn, lty = 1:length(nn))
```



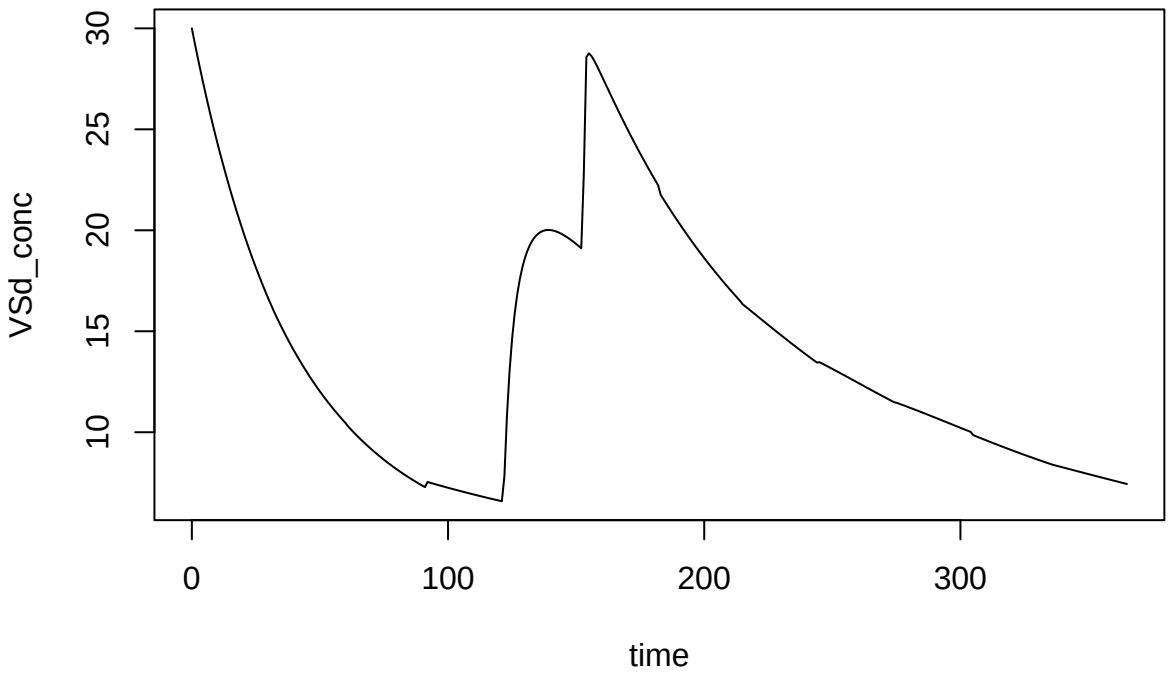

```
plot(VFA ~ time, data = outpodr, type = 'l')
```



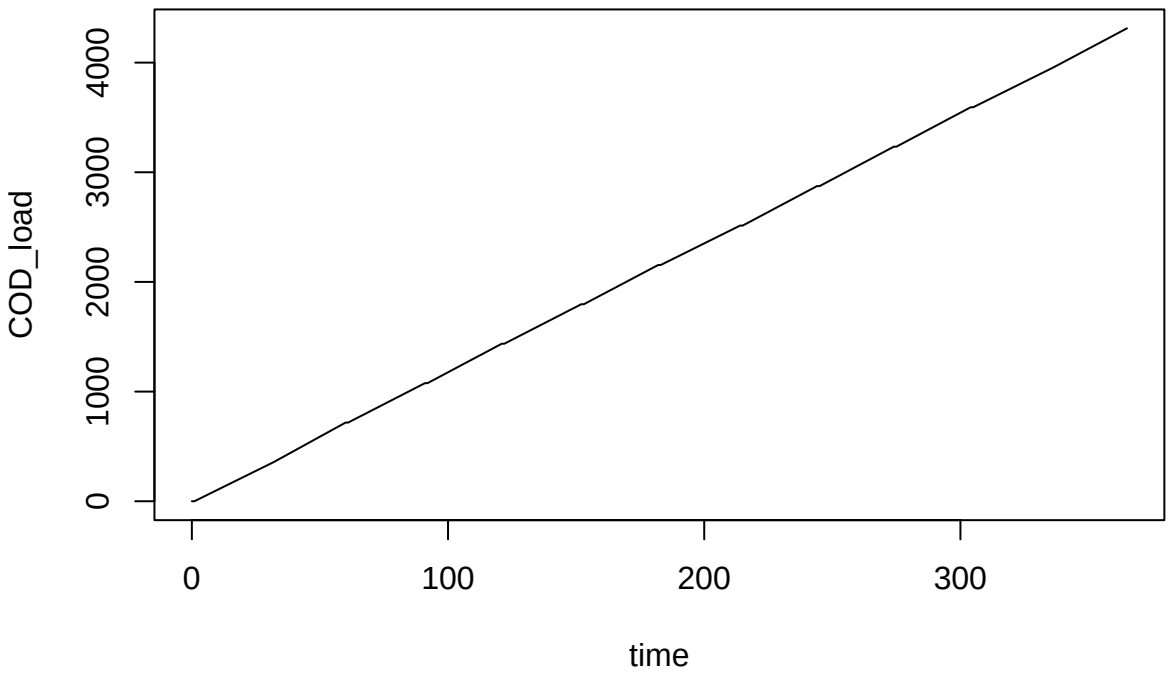

```
plot(VFA_conc ~ time, data = outpodr, type = 'l')
```




```
plot(VSd_conc ~ time, data = outpodr, type = 'l')
```




```
plot(COD_load ~ time, data = outpodr, type = 'l')
```



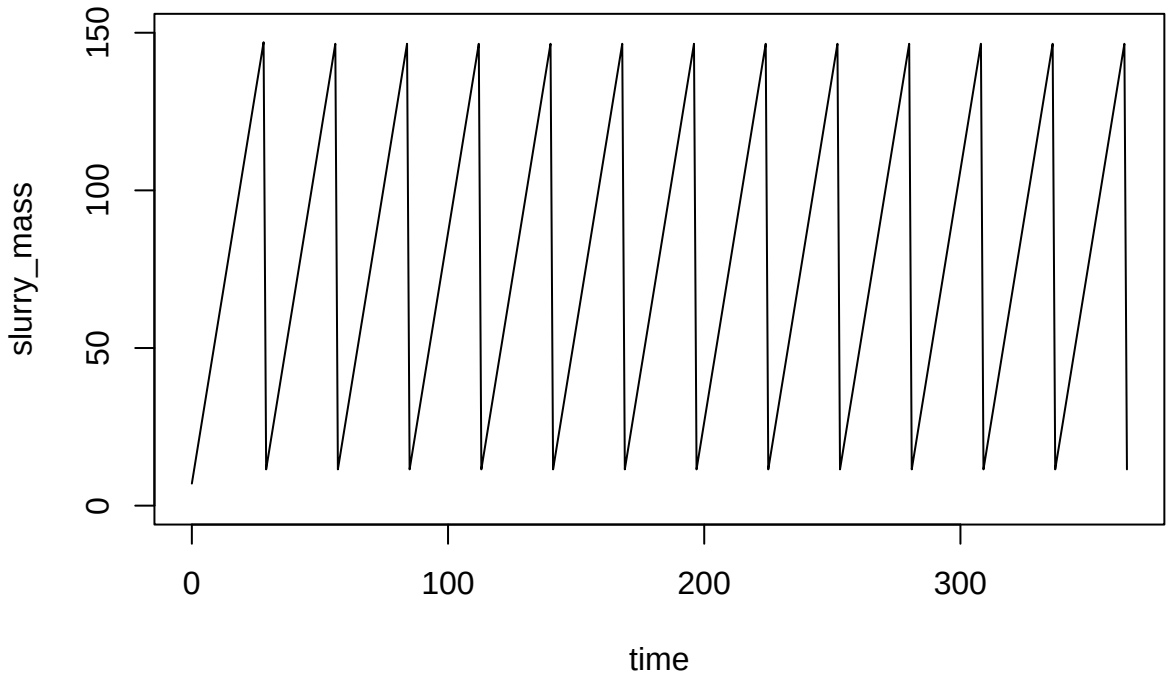
With regular inputs, `slurry_prod_rate` etc. are now explicitly given. This is also for one pig, with a 28 d batch period.

```
stor_reg_ref <- list(
  type = 'regular',
  slurry_prod_rate = 5,
  storage_depth = 2,
  area = 0.65,
  temp_C = 20,
  resid_enrich = 0.2,
  slurry_mass = 7,
  resid_depth = 0.01,
  empty_int = 28,
  wash_water = 0,
  wash_int = 90,
  rest_d = 1
)
```

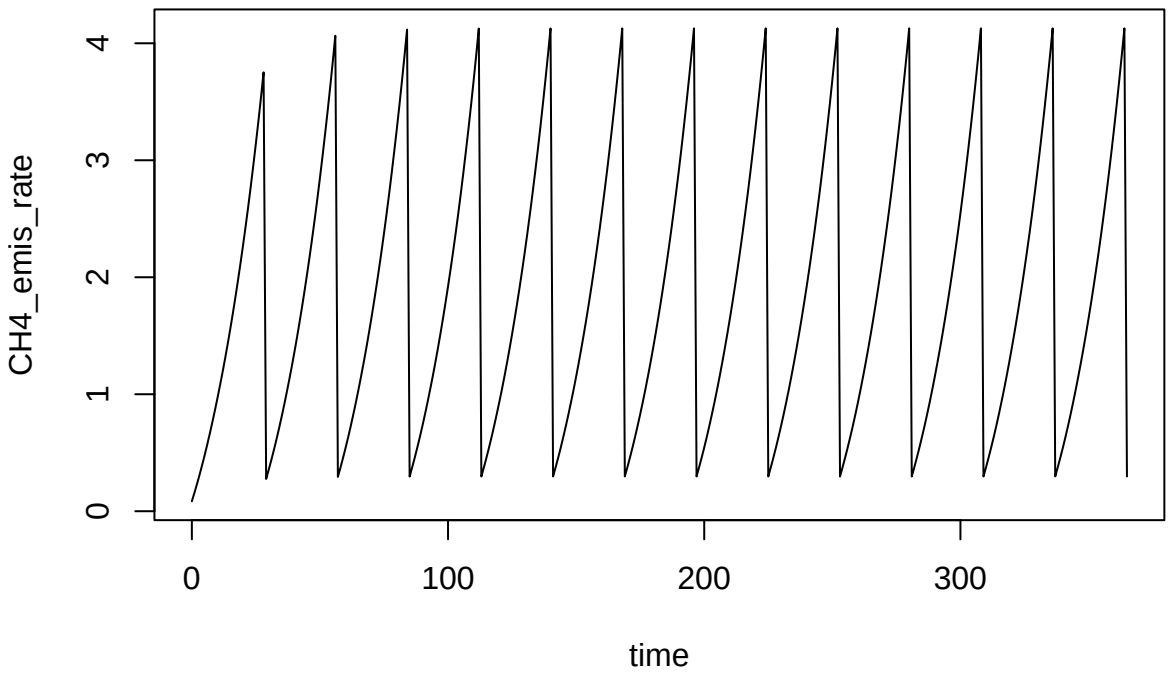
```
outpinr <- abmneo(
  storage = stor_reg_ref,
  365,
  inf_pars = inf_ref,
  grp_pars = grp_ref,
  sub_pars = sub_ref,
  chem_pars = chem_ref
)
```

(Will get an error when starting with 0 slurry_mass)

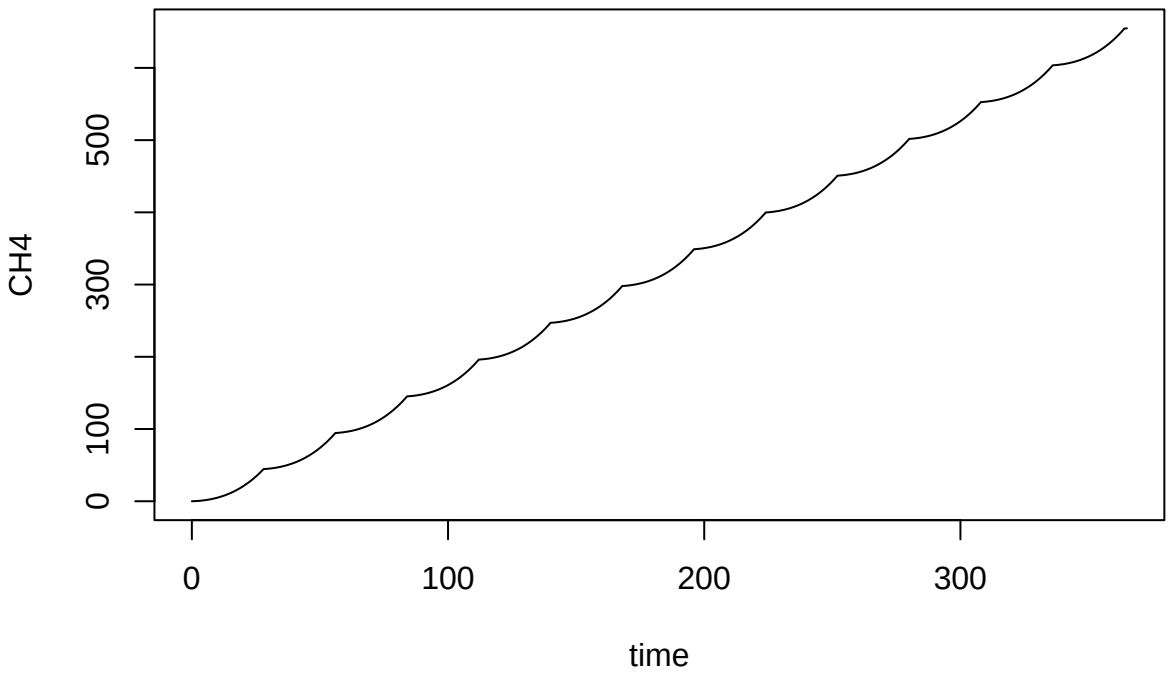
```
plot(slurry_mass ~ time, data = outpinr, type = 'l', ylim = c(0, 150))
```



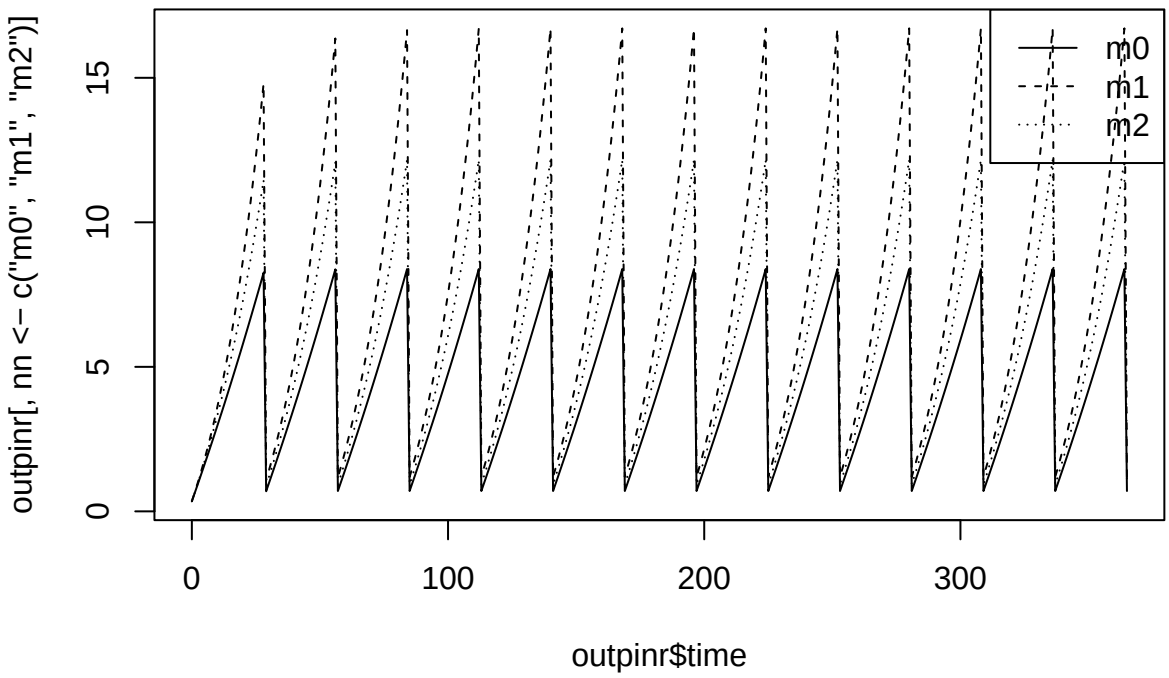

```
plot(CH4_emis_rate ~ time, data = outpinr, type = 'l')
```



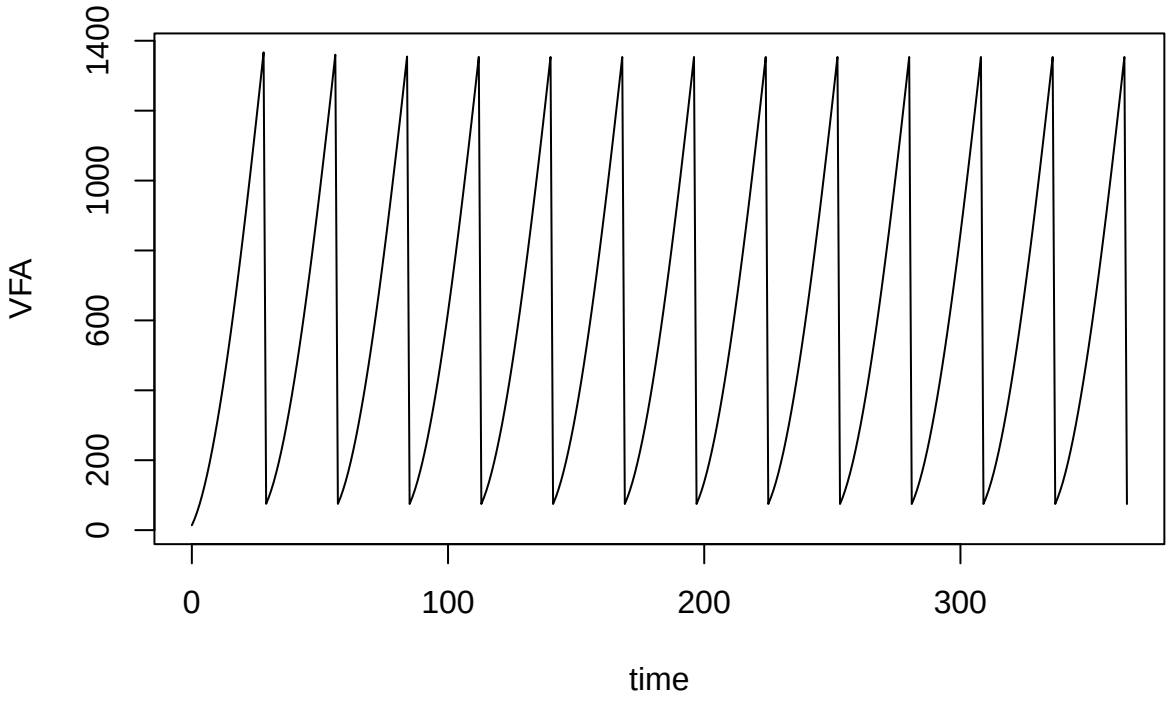

```
plot(CH4 ~ time, data = outpinr, type = 'l')
```



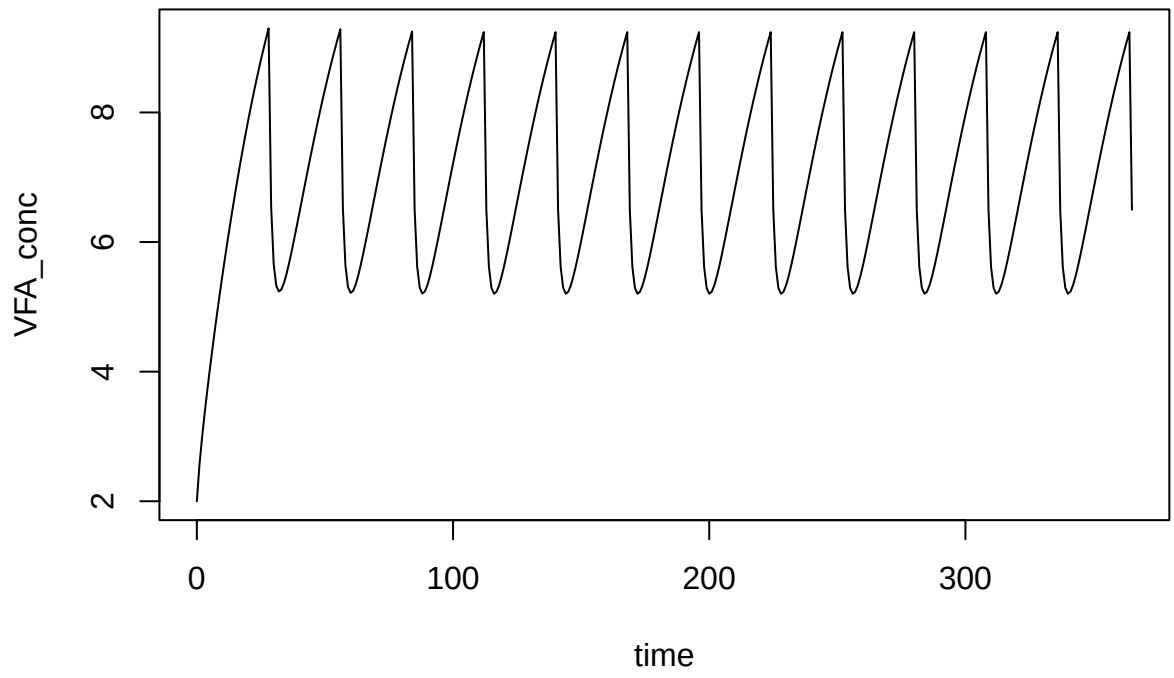

```
matplot(outpinr$time, outpinr[, nn <- c('m0','m1','m2')], col = 'black', lty = 1:length(nn), type = 'l')
legend('topright', legend = nn, lty = 1:length(nn))
```



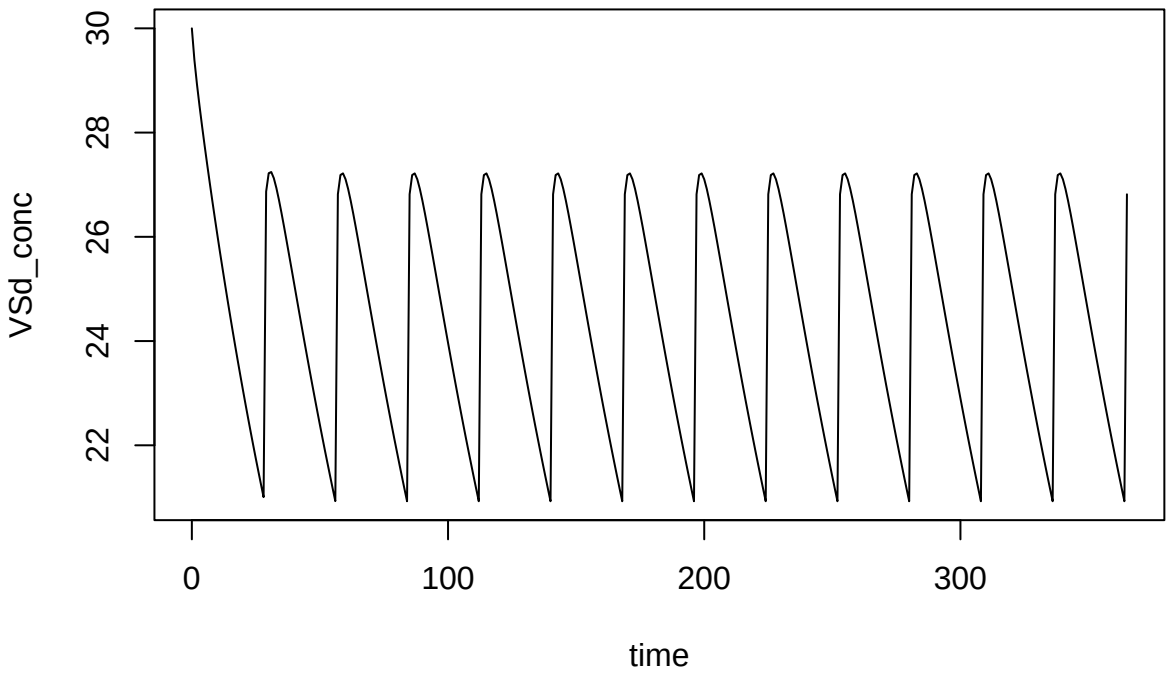

```
plot(VFA ~ time, data = outpinr, type = 'l')
```



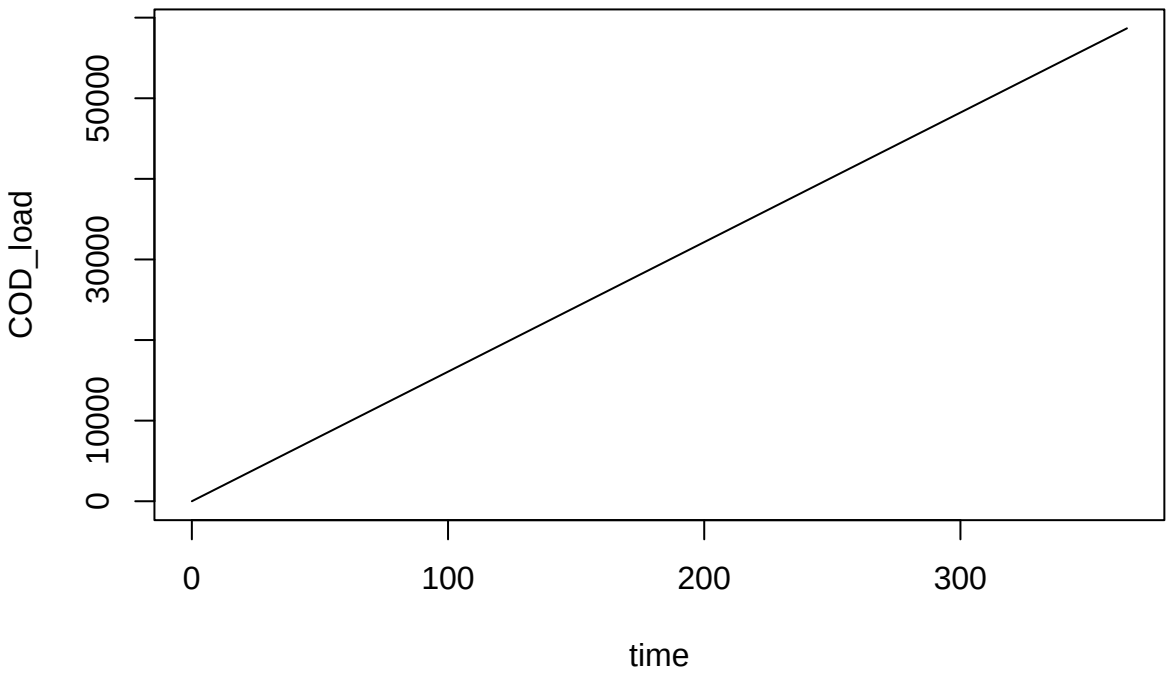

```
plot(VFA_conc ~ time, data = outpinr, type = 'l')
```




```
plot(VSd_conc ~ time, data = outpinr, type = 'l')
```



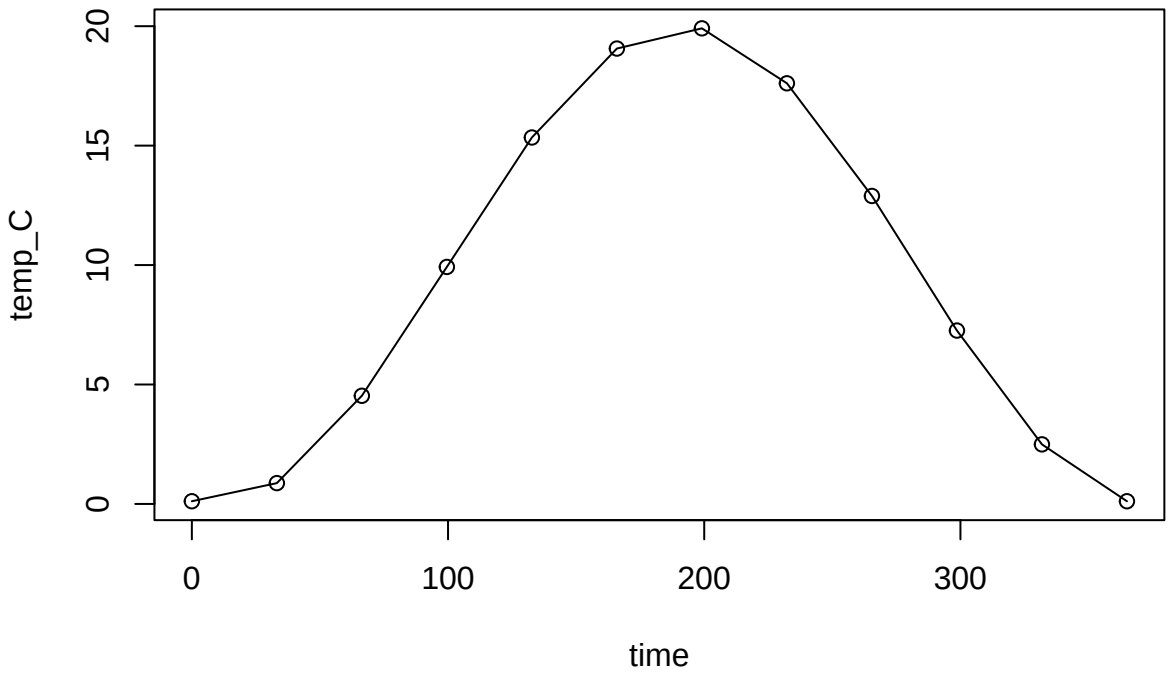

```
plot(COD_load ~ time, data = outpinr, type = 'l')
```



Time-variable inputs

Time-variable inputs (now separate from slurry mass series data) can be combined with both of the above examples. Let's use it in the outdoor storage data (series type `storage`).

```
temp_dat <- data.frame(time = seq(0, 365, length.out = 12))
temp_dat$temp_C <- 10 + 10 * sin((temp_dat$time - 100) / 365 * 2 * pi)
plot(temp_C ~ time, data = temp_dat, type = 'o')
```



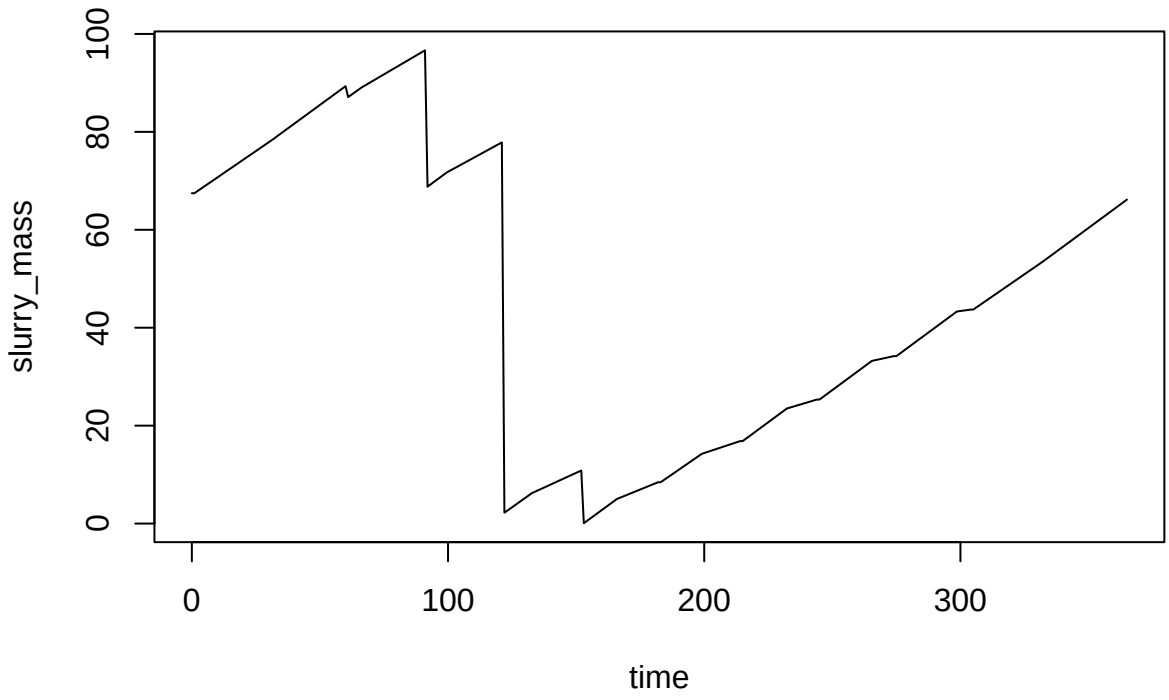

```
devtools::load_all()
```

```
## i Loading abmneo
```

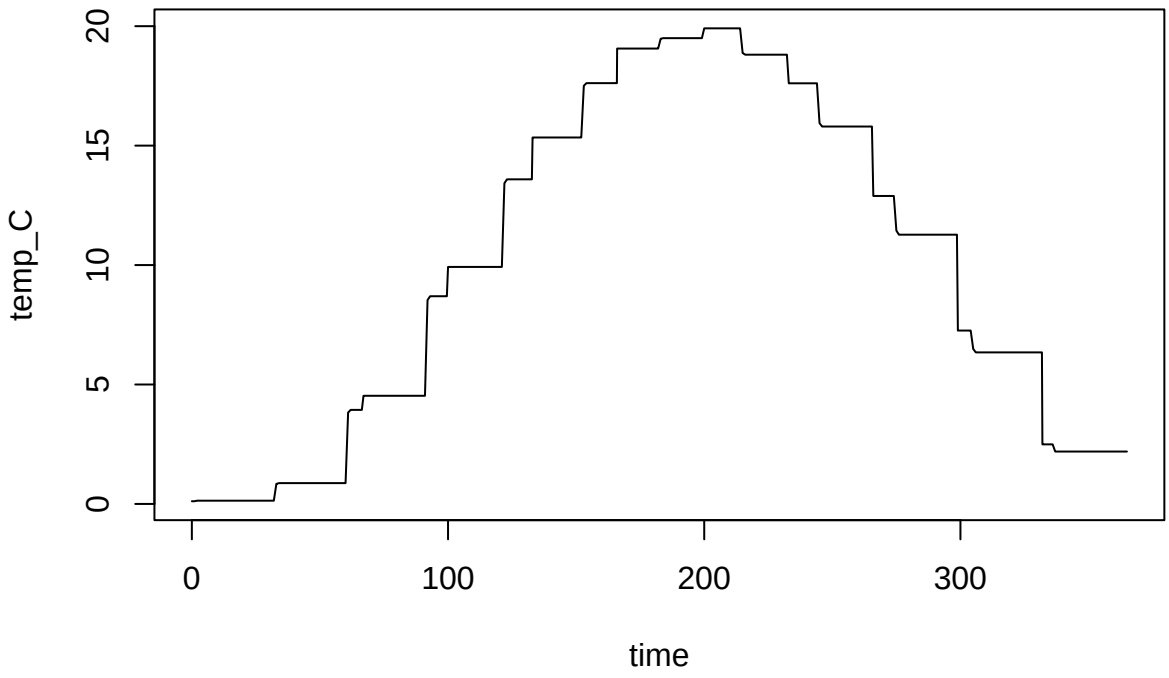
```
outpodv <- abmneo(
  storage = stor_ser_ref,
  365,
  inf_pars = inf_ref,
  grp_pars = grp_ref,
  sub_pars = sub_ref,
  chem_pars = chem_ref,
  var_pars = list(var = temp_dat)
)
```

```
## Warning in empty_store(y, resid_mass = pars$resid_mass, resid_enrich =
## pars$resid_enrich, : Emptying skipped.
## Warning in empty_store(y, resid_mass = pars$resid_mass, resid_enrich =
## pars$resid_enrich, : Emptying skipped.
## Warning in empty_store(y, resid_mass = pars$resid_mass, resid_enrich =
## pars$resid_enrich, : Emptying skipped.
## Warning in empty_store(y, resid_mass = pars$resid_mass, resid_enrich =
## pars$resid_enrich, : Emptying skipped.
```

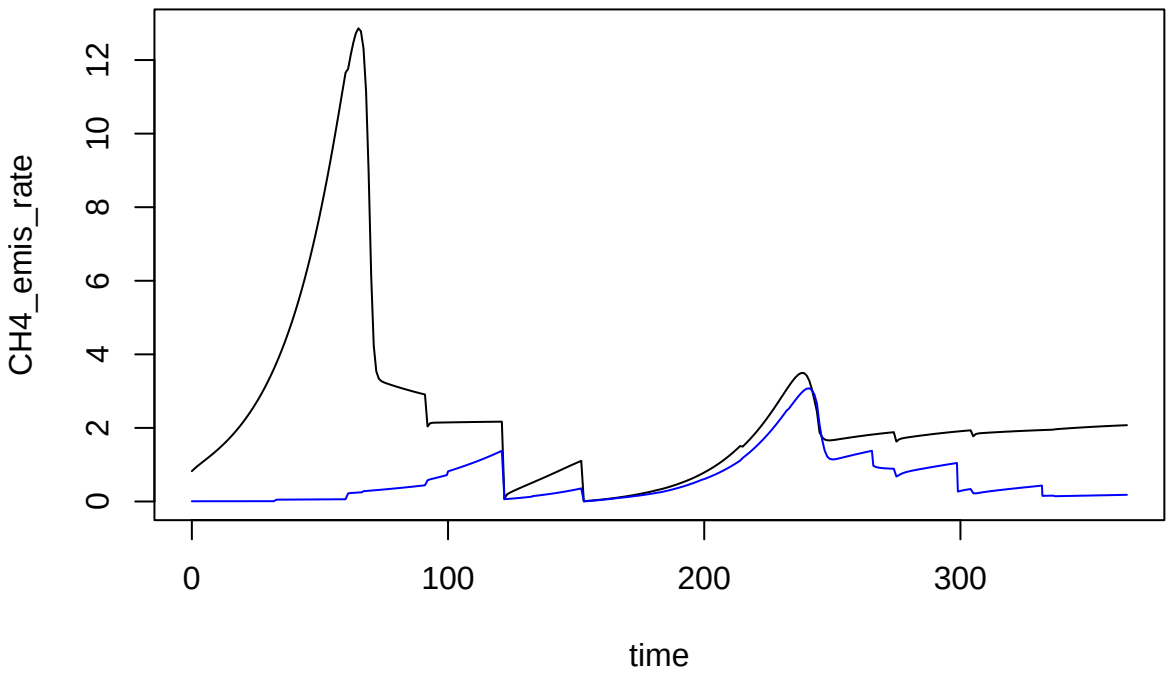
```
plot(slurry_mass ~ time, data = outpodv, type = 'l')
```



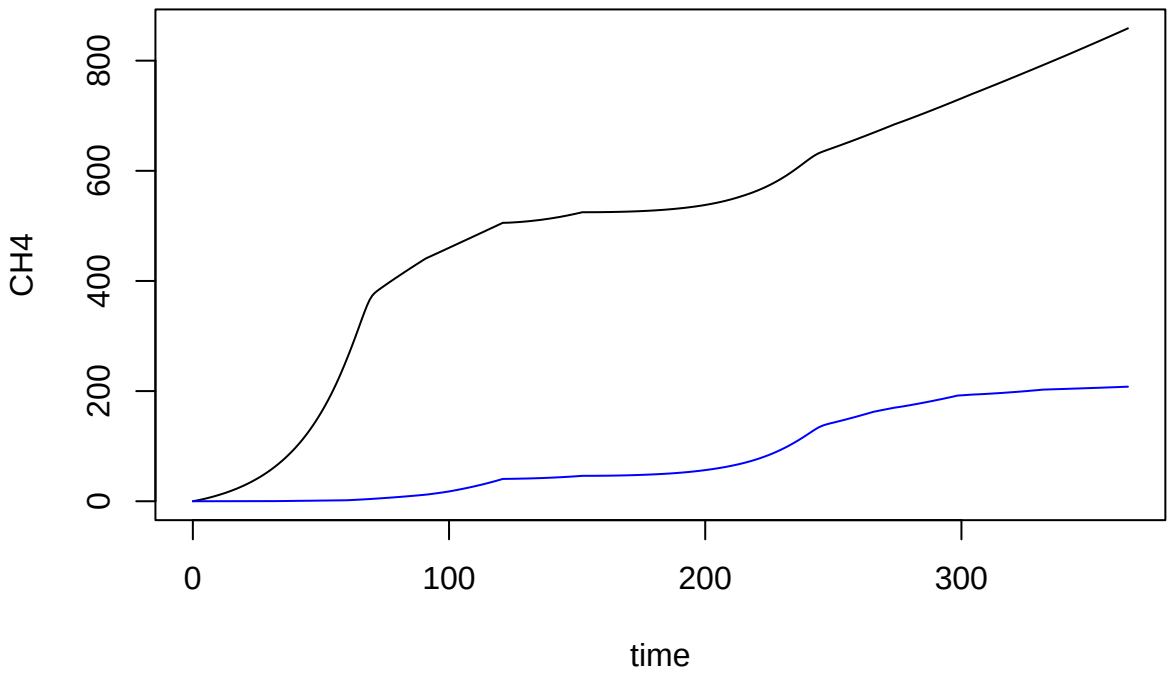

```
plot(temp_C ~ time, data = outpodv, type = 'l')
```



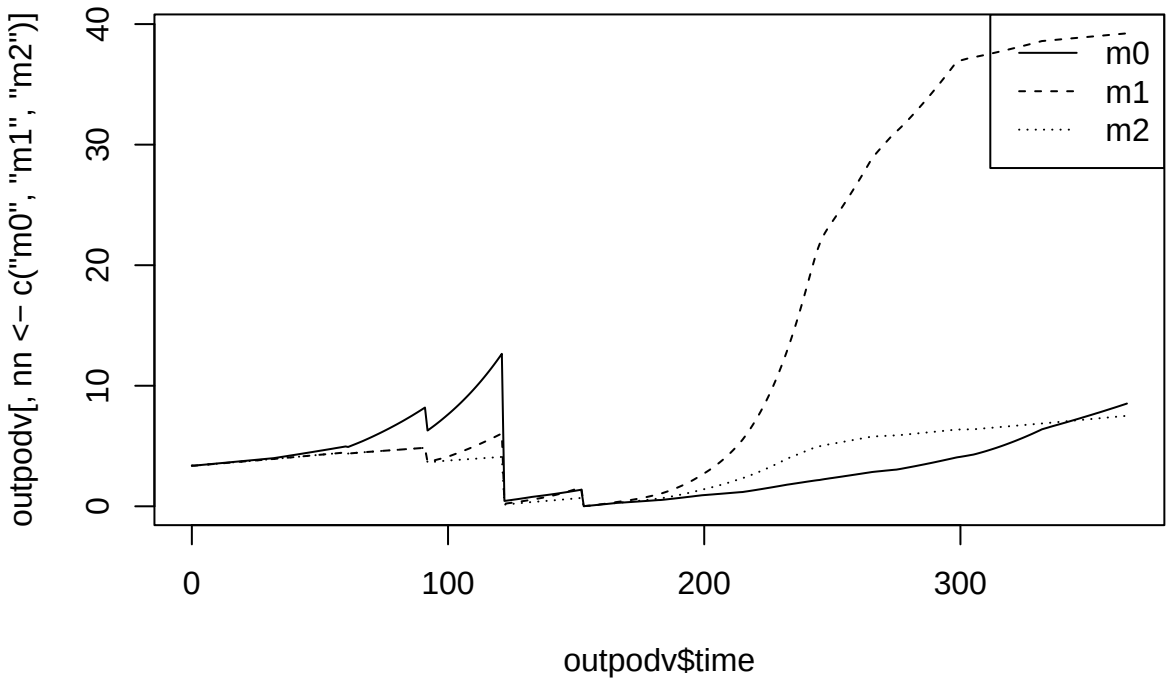

```
plot(CH4_emis_rate ~ time, data = outpodr, type = 'l')
lines(CH4_emis_rate ~ time, data = outpodv, type = 'l', col = 'blue')
```



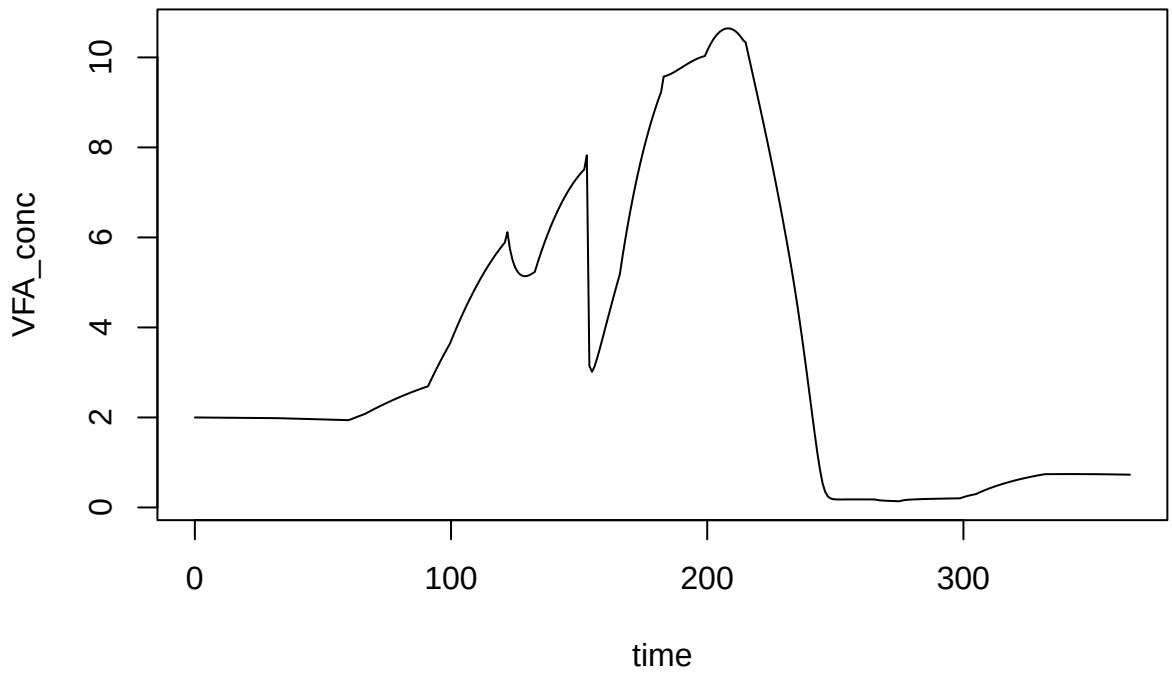

```
plot(CH4 ~ time, data = outpodr, type = 'l')
lines(CH4 ~ time, data = outpodv, type = 'l', col = 'blue')
```



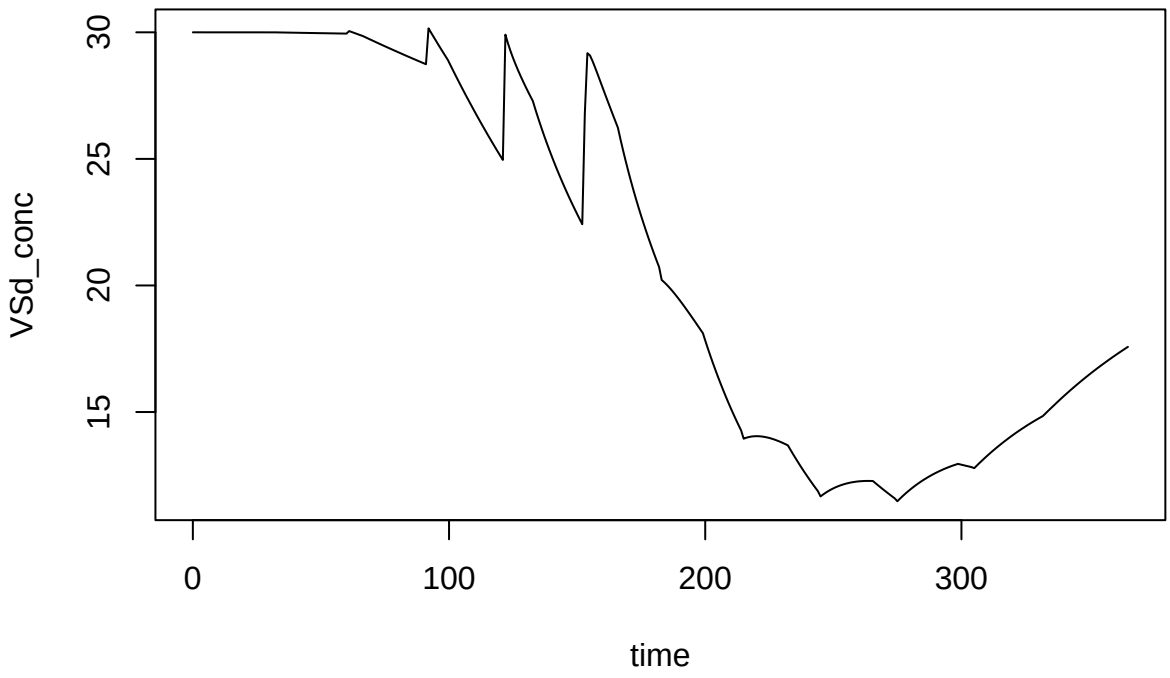

```
matplot(outpodv$time, outpodv[, nn <- c('m0','m1','m2')], col = 'black', lty = 1:length(nn), type = 'l')
legend('topright', legend = nn, lty = 1:length(nn))
```




```
plot(VFA_conc ~ time, data = outpodv, type = 'l')
```



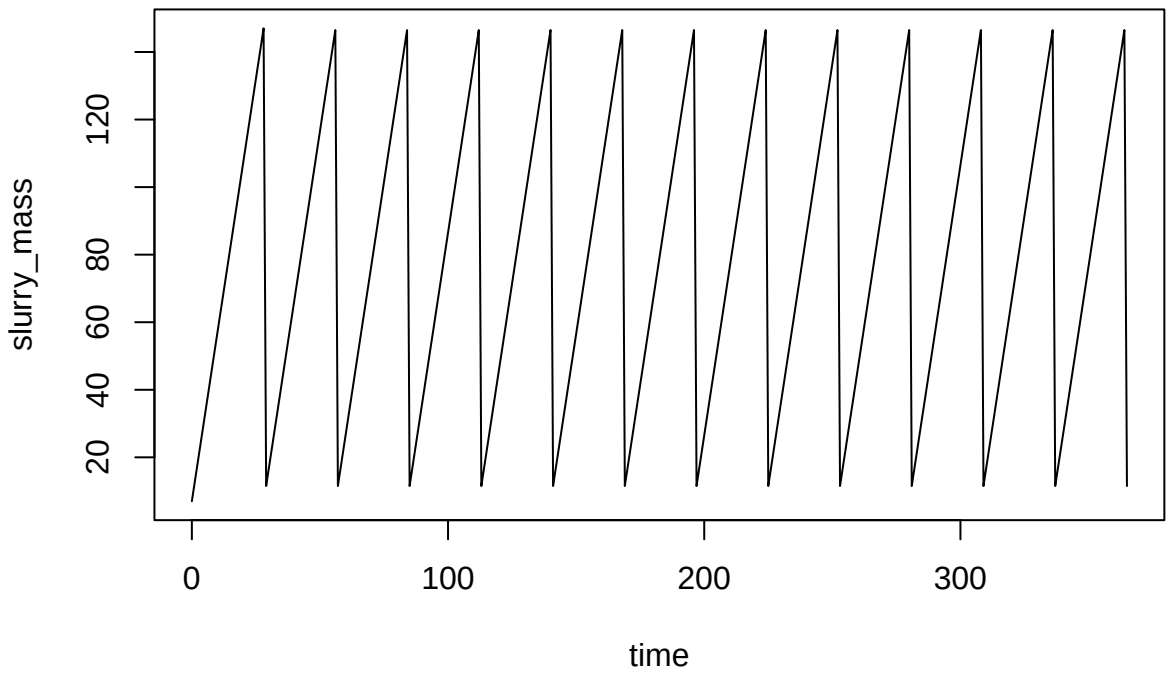

```
plot(VSd_conc ~ time, data = outpodv, type = 'l')
```



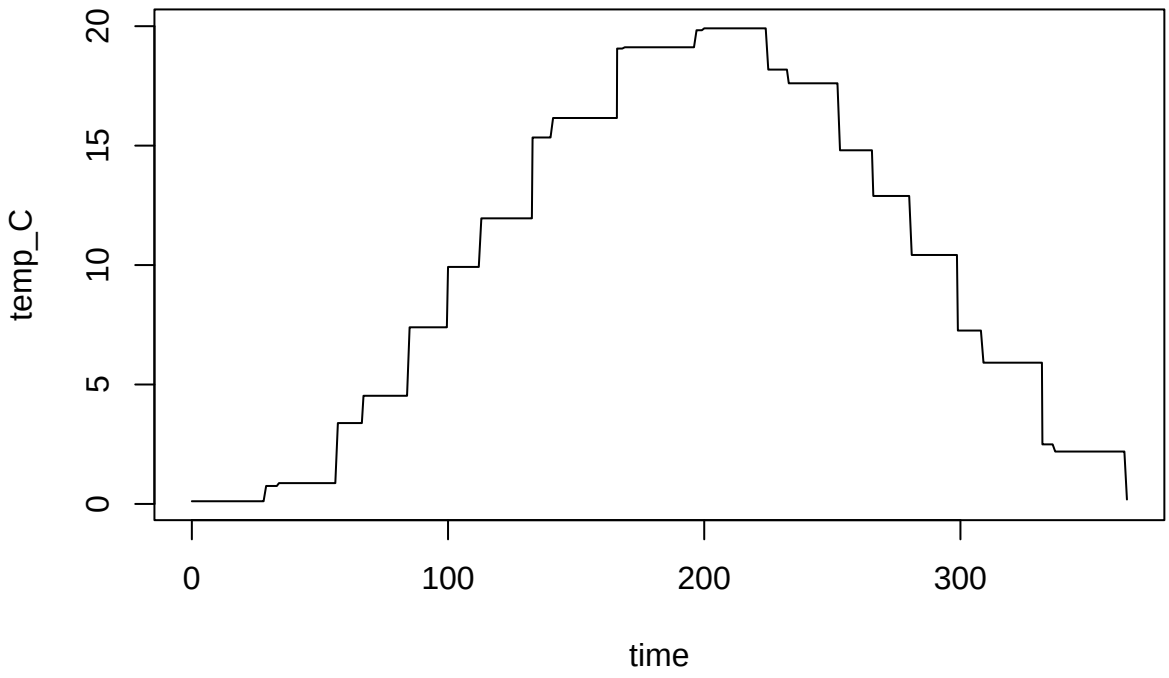
Or regular inputs.

```
outpinv <- abmneo(
  storage = stor_reg_ref,
  365,
  inf_pars = inf_ref,
  grp_pars = grp_ref,
  sub_pars = sub_ref,
  chem_pars = chem_ref,
  var_pars = list(var = temp_dat)
)
```

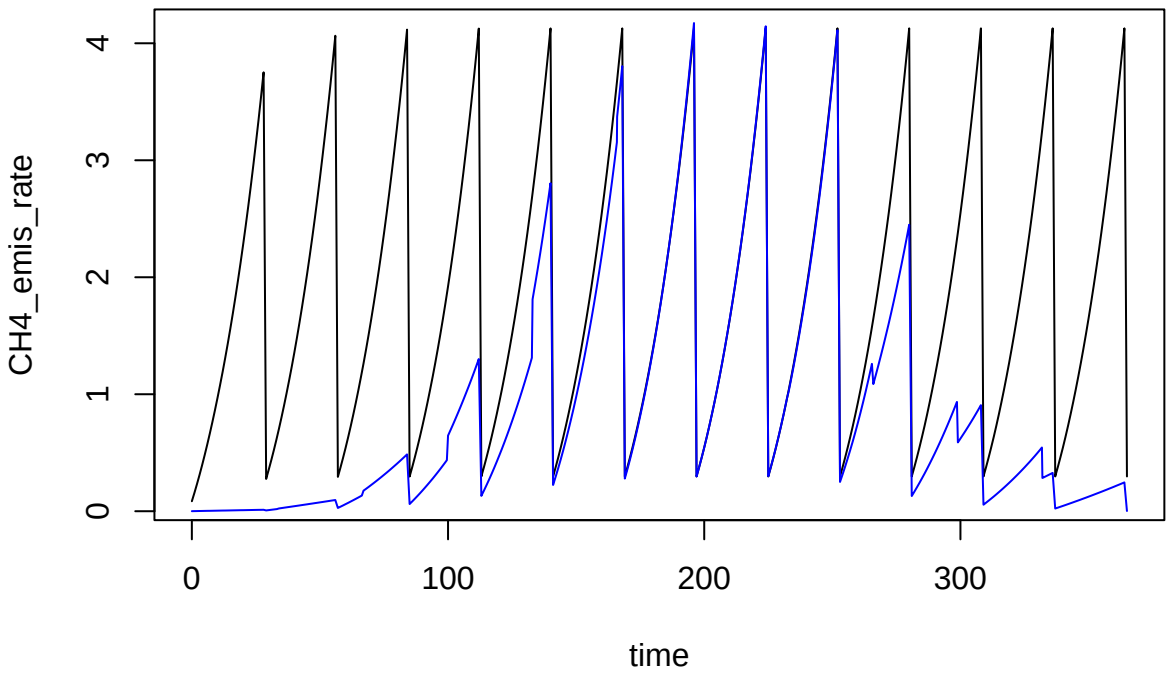
```
plot(slurry_mass ~ time, data = outpinv, type = 'l')
```



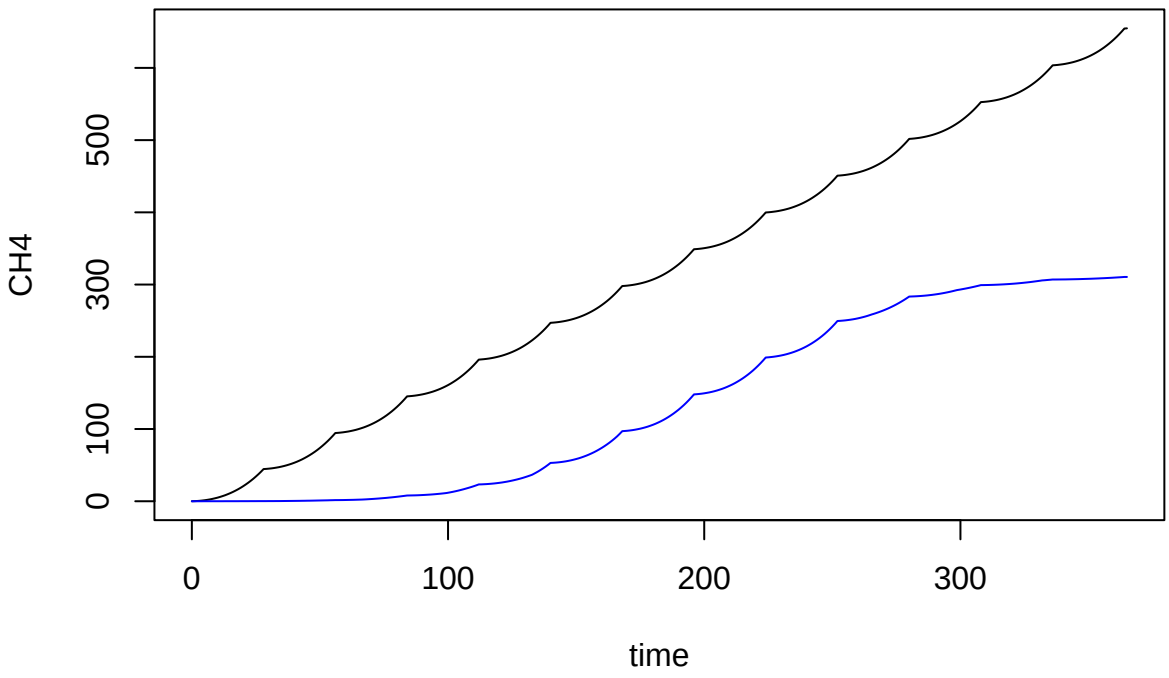

```
plot(temp_C ~ time, data = outpinv, type = 'l')
```



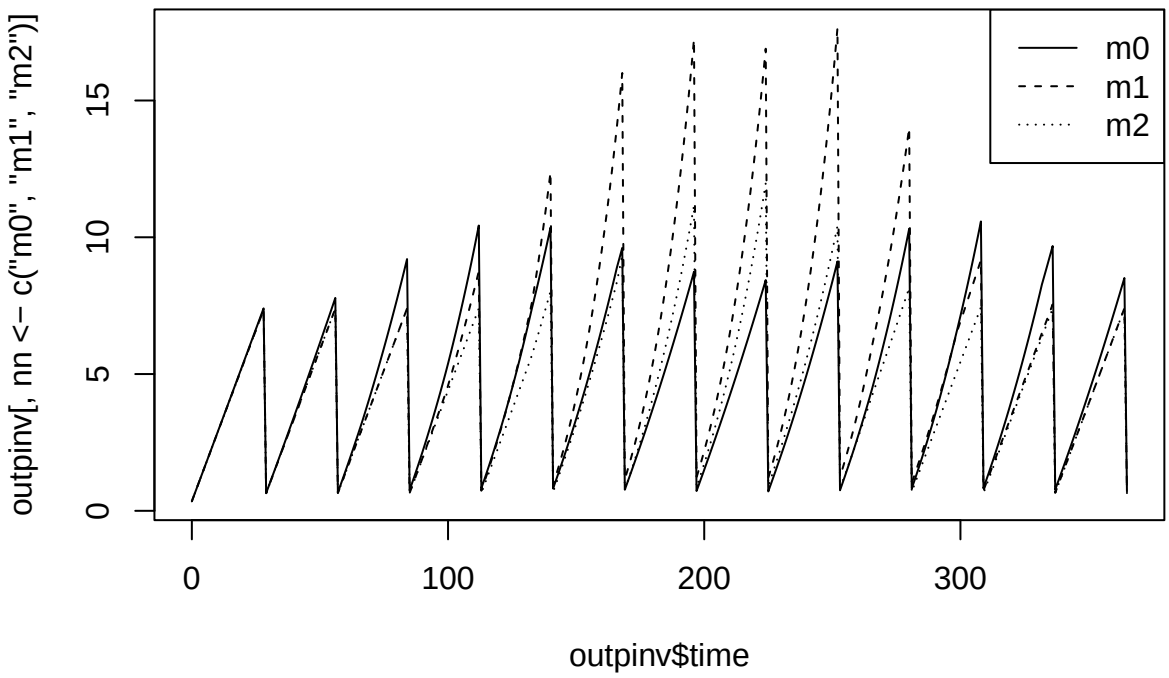

```
plot(CH4_emis_rate ~ time, data = outpinr, type = 'l')
lines(CH4_emis_rate ~ time, data = outpinv, type = 'l', col = 'blue')
```



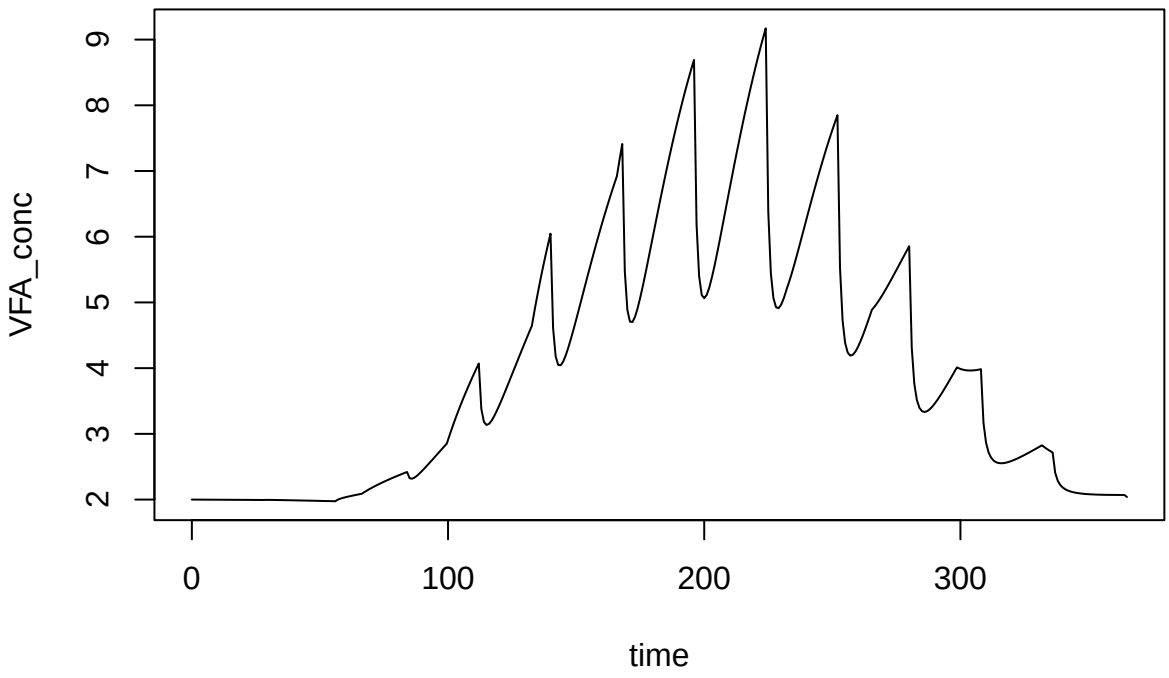

```
plot(CH4 ~ time, data = outpinr, type = 'l')
lines(CH4 ~ time, data = outpinv, type = 'l', col = 'blue')
```



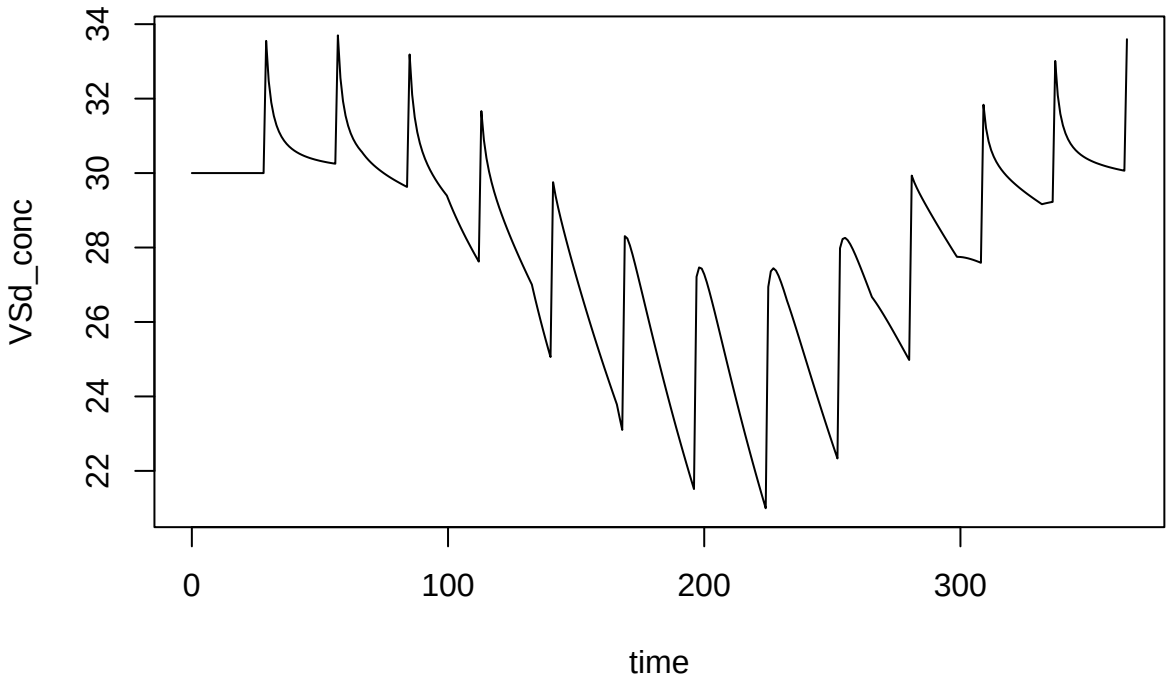

```
matplot(outpinv$time, outpinv[, nn <- c('m0','m1','m2')], col = 'black', lty = 1:length(nn), type = 'l')
legend('topright', legend = nn, lty = 1:length(nn))
```




```
plot(VFA_conc ~ time, data = outpinv, type = 'l')
```




```
plot(VSd_conc ~ time, data = outpinv, type = 'l')
```



Note the "steps" in the time-variable inputs. There is no longer active interpolation. This was a deliberate change aimed at simplification of the code and an increase in flexibility (no special cases, i.e., make interpolation functions for x and y inputs, and almost any inputs can vary over time now). There is almost certainly a speed improvement. Users can go for a finer resolution in the input, but there will be a (small?) speed penalty.

We might recommend that output resolution should match input resolution when inputs vary over time. That can be done using `times`.

Q

```
devtools::load_all()
```

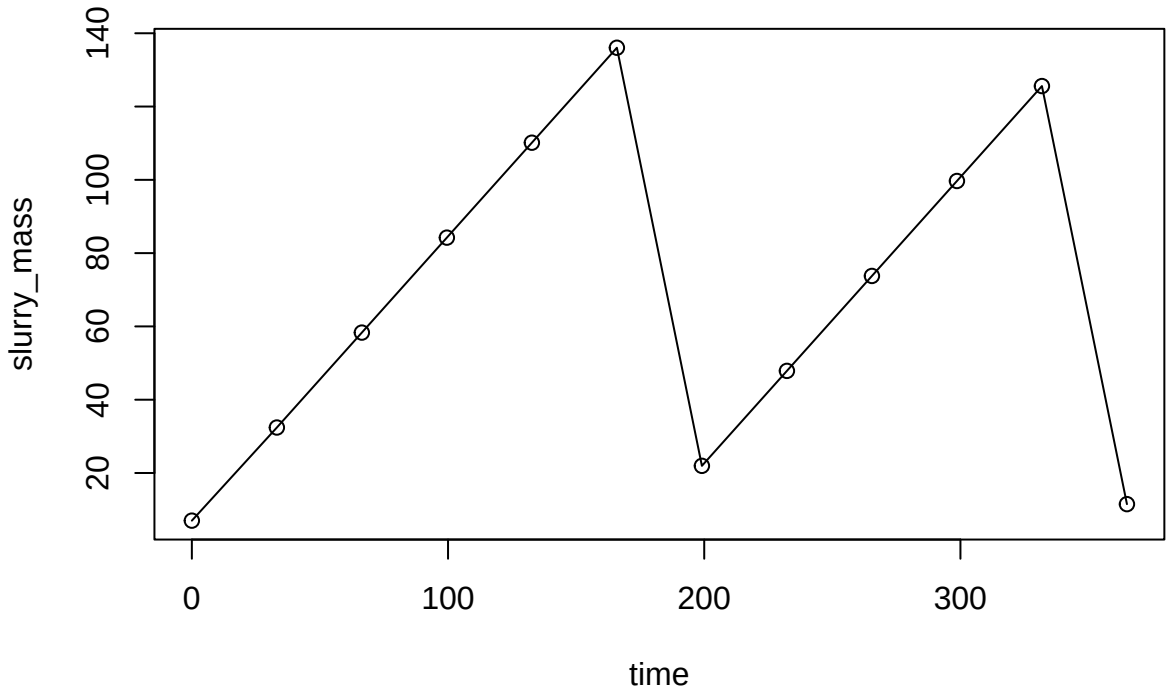
```
## i Loading abmneo
```

```
outpinvt <- abmneo(
  storage = stor_reg_ref,
  365,
  times = temp_dat$time,
  inf_pars = inf_ref,
  grp_pars = grp_ref,
  sub_pars = sub_ref,
  chem_pars = chem_ref,
  var_pars = list(var = temp_dat)
)
```

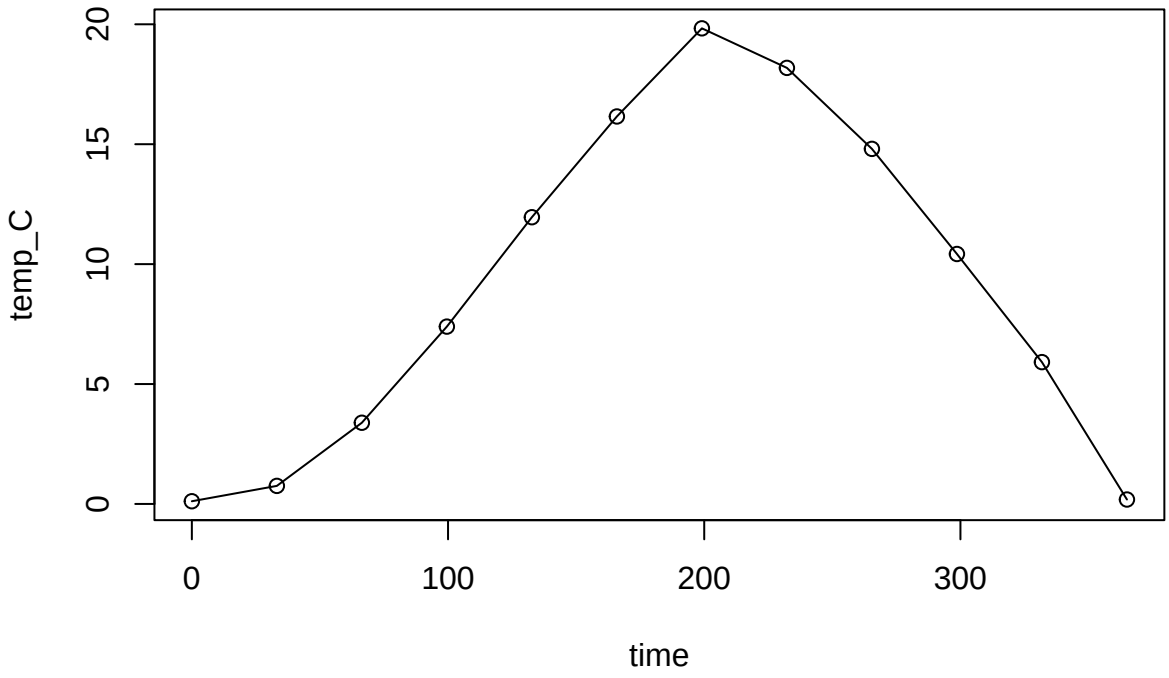
```
## Warning in check_COD(dat = dat, grps = pars$grps, subs = pars$subs, COD_conv =
## pars$COD_conv, : COD balance is off by 15%
```

With this approach, instantaneous emission rate is not very meaningful. The purple line makes more sense.

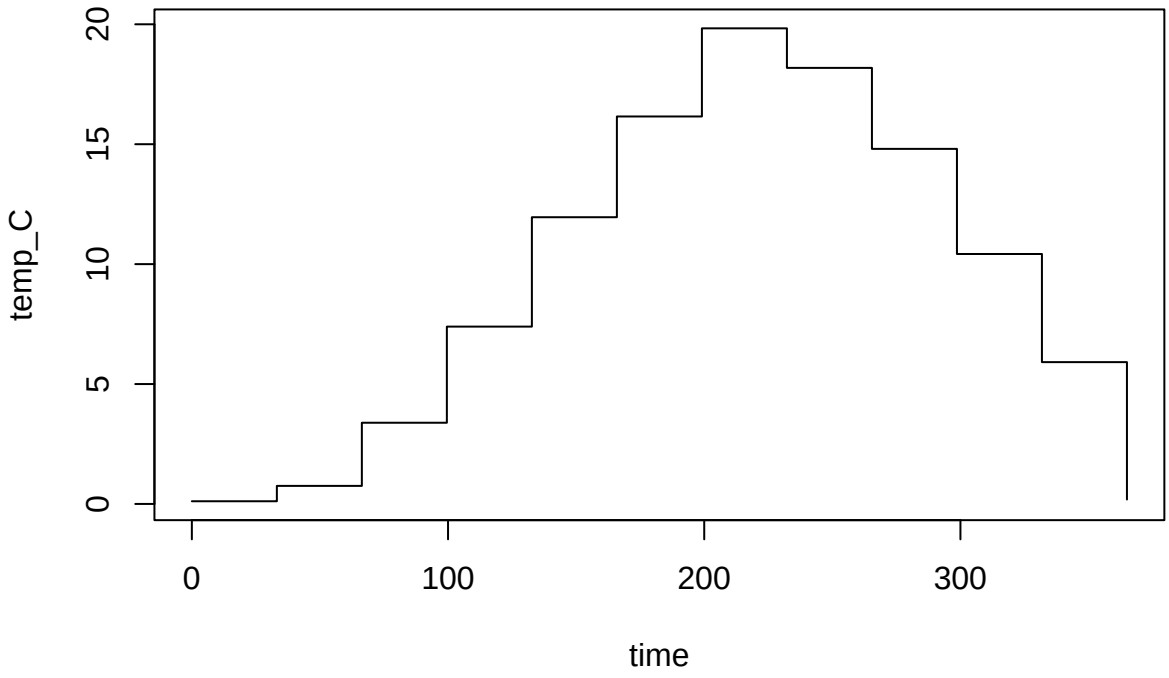
```
plot(slurry_mass ~ time, data = outpinvt, type = 'o')
```



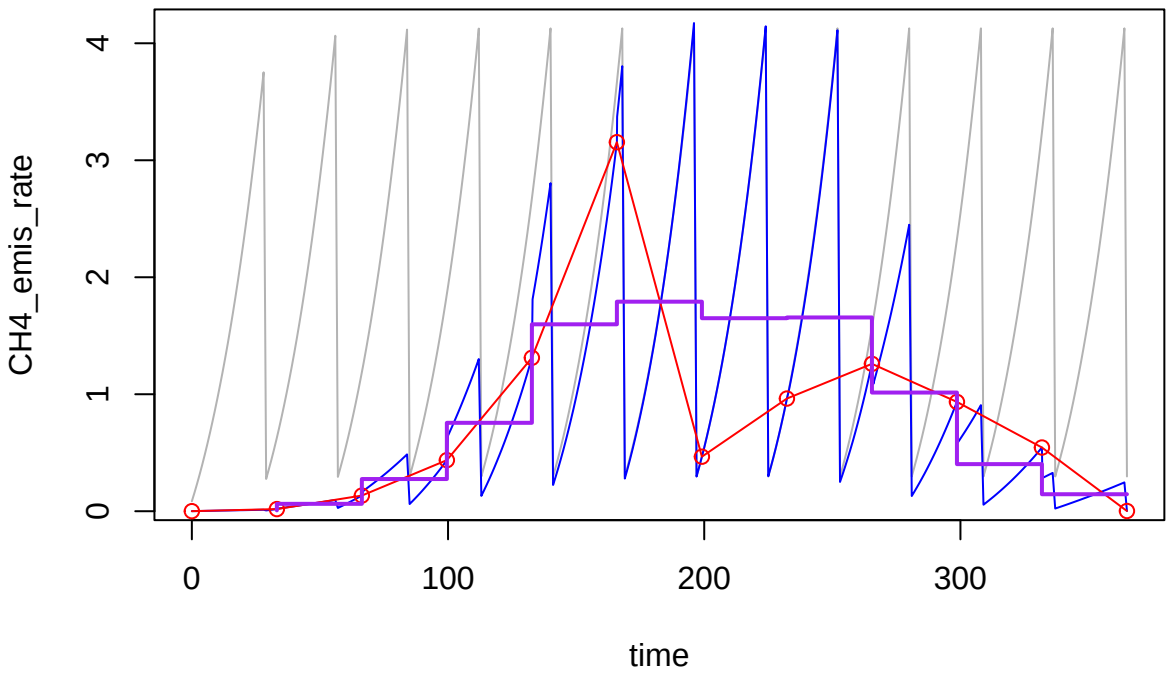

```
plot(temp_C ~ time, data = outpinvt, type = 'o')
```



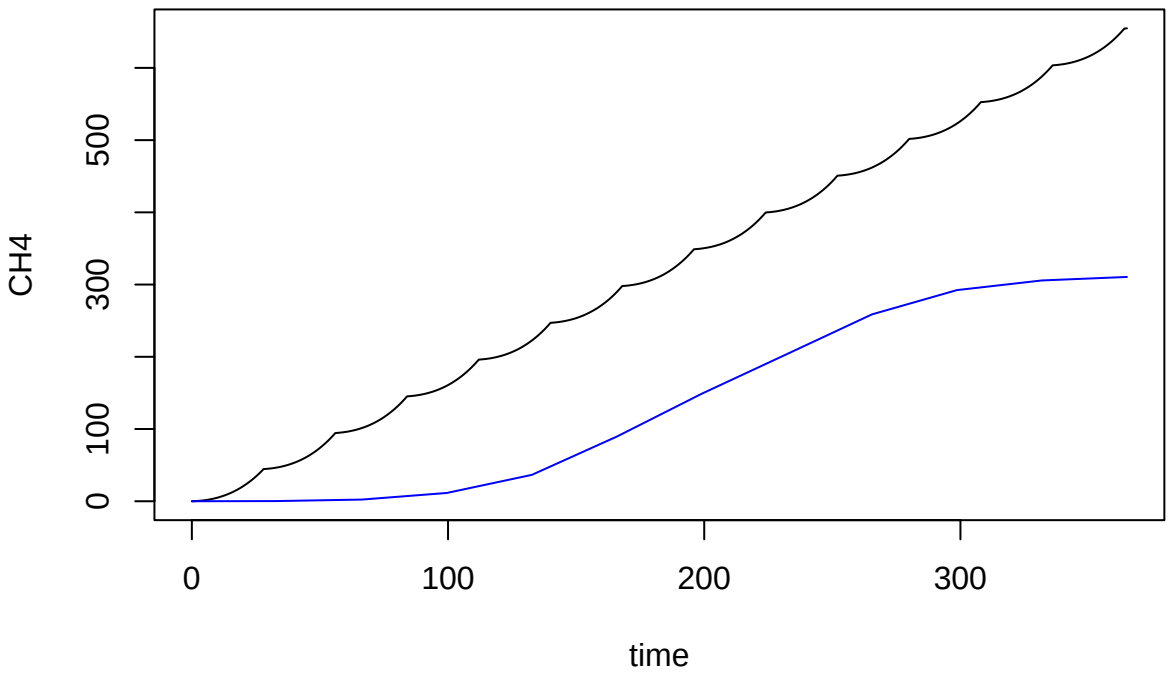

```
plot(temp_C ~ time, data = outpinvt, type = 's')
```



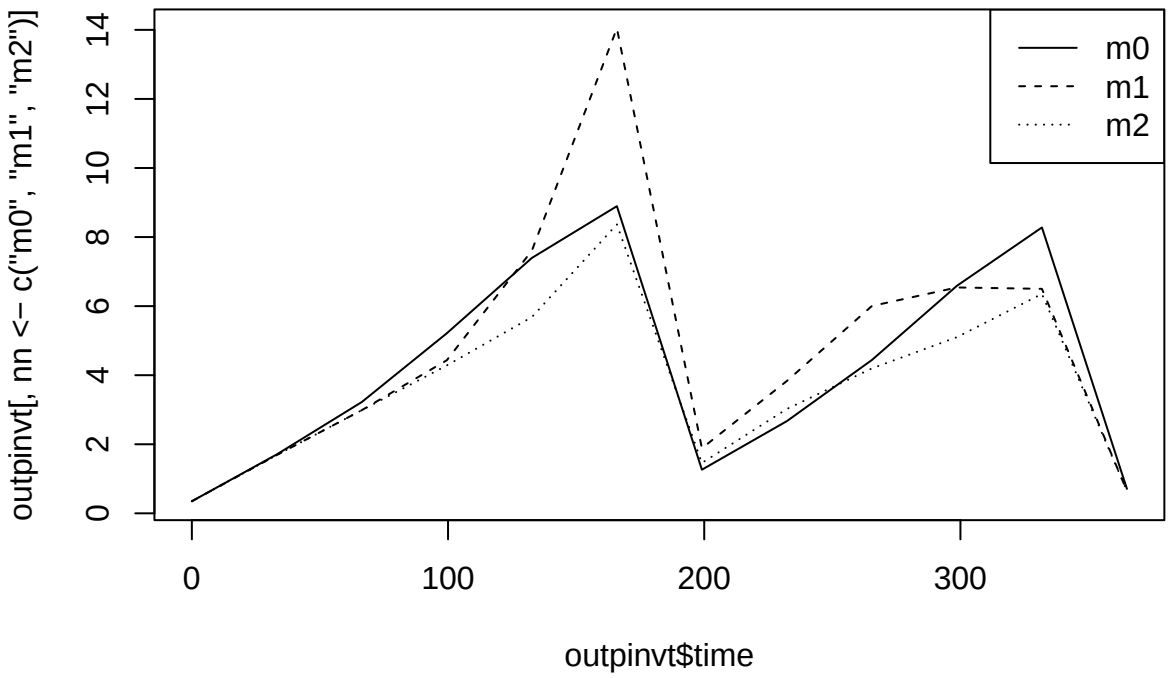

```
plot(CH4_emis_rate ~ time, data = outpinr, type = 'l', col = 'gray70')
lines(CH4_emis_rate ~ time, data = outpinv, type = 'l', col = 'blue')
lines(CH4_emis_rate ~ time, data = outpinvt, type = 'o', col = 'red')
lines(CH4_emis_rate_ave ~ time, data = outpinvt, type = 'S', col = 'purple', lwd = 2)
```



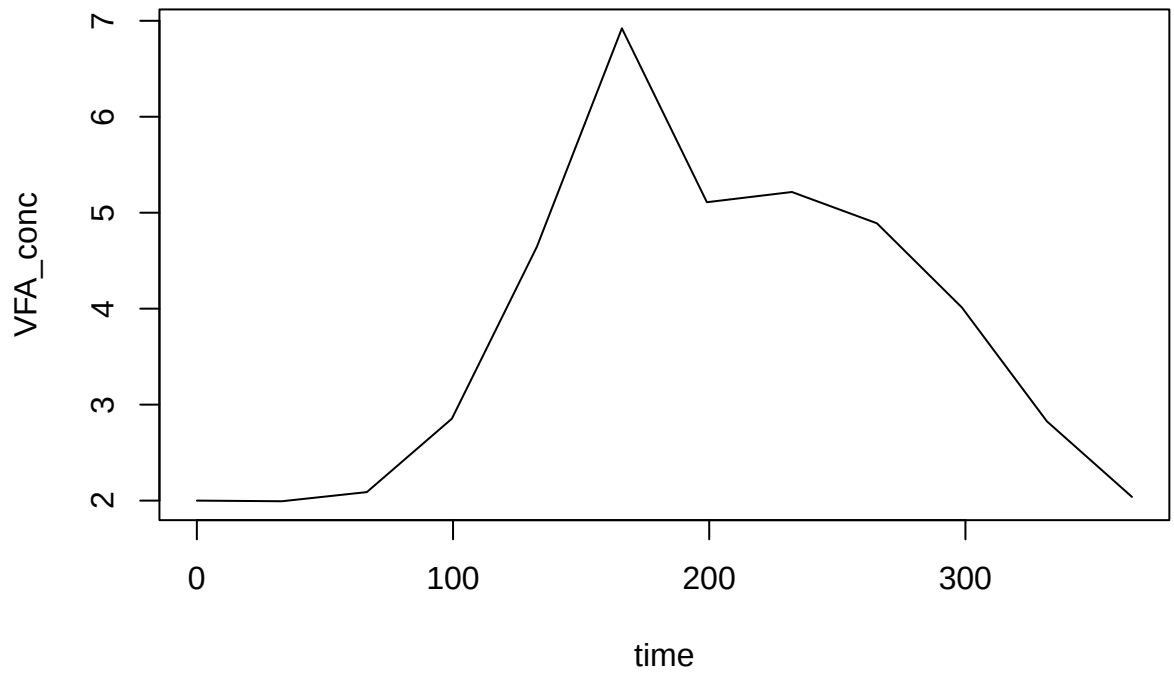

```
plot(CH4 ~ time, data = outpinr, type = 'l')
lines(CH4 ~ time, data = outpinvt, type = 'l', col = 'blue')
```



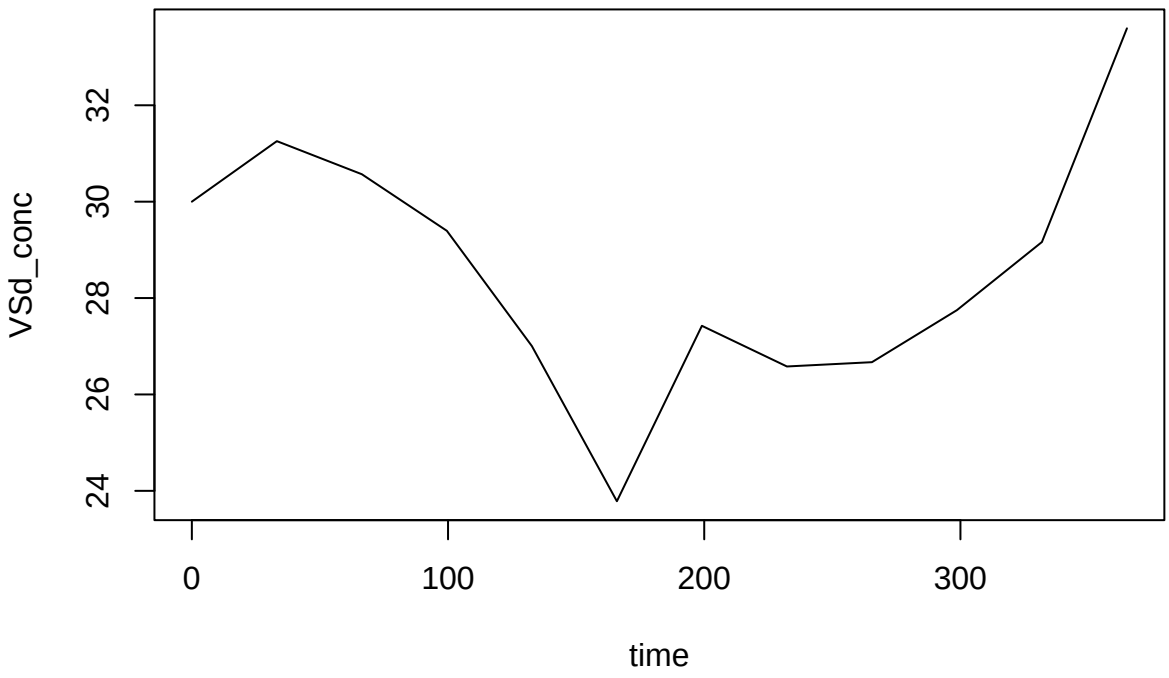

```
matplot(outpinvt$time, outpinvt[, nn <- c('m0','m1','m2')], col = 'black', lty = 1:length(nn), type = '
legend('topright', legend = nn, lty = 1:length(nn))
```




```
plot(VFA_conc ~ time, data = outpinvt, type = 'l')
```




```
plot(VSd_conc ~ time, data = outpinvt, type = 'l')
```



Flexible substrates

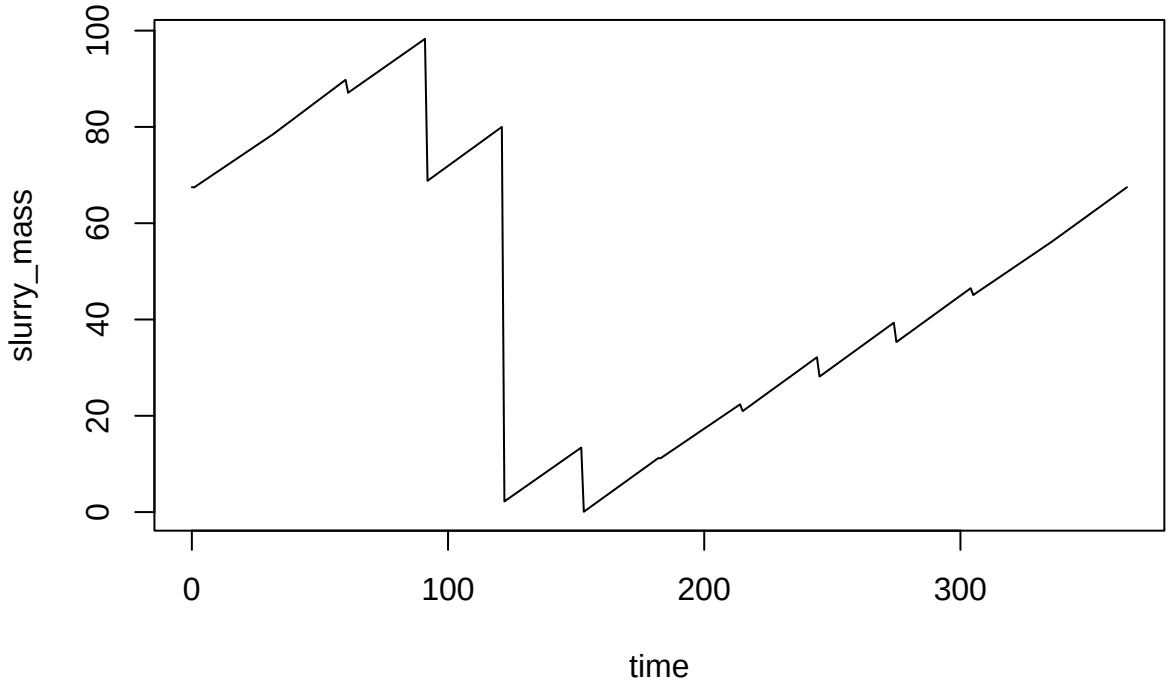
Above only a single particulate substrate was used. But any substrates, with arbitrary names and stoichiometry can be defined.

```
sub_smf <- list(
  subs = c('slow', 'mid', 'fast'),
  T_opt_hyd = c(default = 60),
  T_min_hyd = c(default = 0),
  T_max_hyd = c(default = 90),
  hydrol_opt = c(slow = 0.01, mid = 0.1, fast = 1),
  sub_fresh = c(slow = 10, mid = 10, fast = 10),
  sub_init = c(slow = 10, mid = 10, fast = 10)
)
```

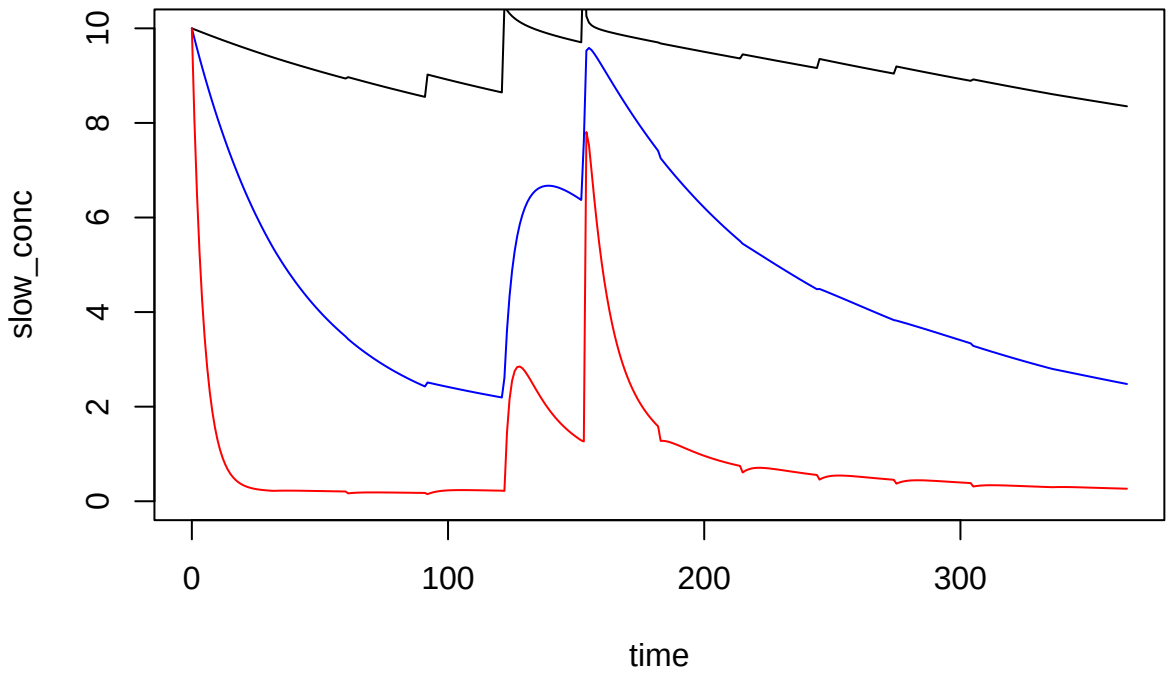
```
outpodf <- abmneo(
  storage = stor_ser_ref,
  365,
  inf_pars = inf_ref,
  grp_pars = grp_ref,
  sub_pars = sub_smf,
  chem_pars = chem_ref
)
```

```
## Warning in check_COD(dat = dat, grps = pars$grps, subs = pars$subs, COD_conv =
## pars$COD_conv, : COD balance is off by 2.4%
```

```
plot(slurry_mass ~ time, data = outpodf, type = 'l')
```




```
plot(slow_conc ~ time, data = outpodf, type = 'l', ylim = c(0, 10))
lines(mid_conc ~ time, data = outpodf, col = 'blue')
lines(fast_conc ~ time, data = outpodf, col = 'red')
```



This is the simple way to specify substrates. When entered in this way, there is no stoichiometry *per se*, but each g substrate COD produces 1 g VFA intermediate. The alternative is to define stoichiometry. That is described below.

Sulfate reduction and other metabolic pathways

The microbial metabolism code (whatever happens after VFA intermediate is formed) now used is completely flexible with respect to reactants and products. Sulfate reduction is not hard-coded but can easily and very explicitly be specified with the new approach. We need the appropriate stoichiometry. Masses/coefficients are specified as:

1. COD mass, or if COD = 0, then
2. N mass, or if N = 0, then
3. C mass, or if C = 0, then
4. S mass, or if S = 0, then
5. total mass.

So for SO4-2, we want the the mass of S divided by the COD of acetate, i.e., 32 / 64 = 0.5. (We might think about flexibility in how stoichiometry is defined, e.g., allow user to input moles, but it is probably better to fix it.)

```
mstoich_sr <- matrix(
  c(-1, -1, -1, -1,
     0,  0,  0, -0.5,
     1,  1,  1,  0),
  nrow = 3,
  byrow = TRUE,
```

```
  dimnames = list(
    c('VFA', 'SO4', 'CH4'),
    c('m0', 'm1', 'm2', 'sr1')
  )
)

mstoich_sr
```

```
##     m0 m1 m2  sr1
## VFA -1 -1 -1 -1.0
## SO4  0  0  0 -0.5
## CH4  1  1  1  0.0
```

```
ksmat_sr <- matrix(
  c(0.5, 0.5, 0.5, 0.5,
    NA,   NA,  NA, 0.5),
  nrow = 2,
  byrow = TRUE,
  dimnames = list(
    c('VFA', 'SO4'),
    c('m0', 'm1', 'm2',  'sr1')
  )
)

ksmat_sr
```

```
##      m0  m1  m2 sr1
## VFA 0.5 0.5 0.5 0.5
## SO4  NA  NA  NA 0.5
```

Put these in a grp_pars list.

```
grp_sr <- list(
  grps = c('m0', 'm1', 'm2', 'sr1'),
  yield = c(default = 0.05),
  xa_fresh = c(default = 0.05),
  xa_init = c(default = 0.05),
  dd_rate = c(default = 0.02),
  qhat_opt = c(m0 = 1, m1 = 1, m2 = 2, sr1 = 2),
  T_opt = c(m0 = 18, m1 = 18, m2 = 28, sr1 = 25),
  T_min = c(m0 = 0, m1 = 6.41, m2 = 6.41, sr1 = 0),
  T_max = c(m0 = 25, m1 = 25, m2 = 38, sr1 = 40),
  ksmat = ksmat_sr,
  mstoich = mstoich_sr
)
```

Sulfate needs to be added as a component.

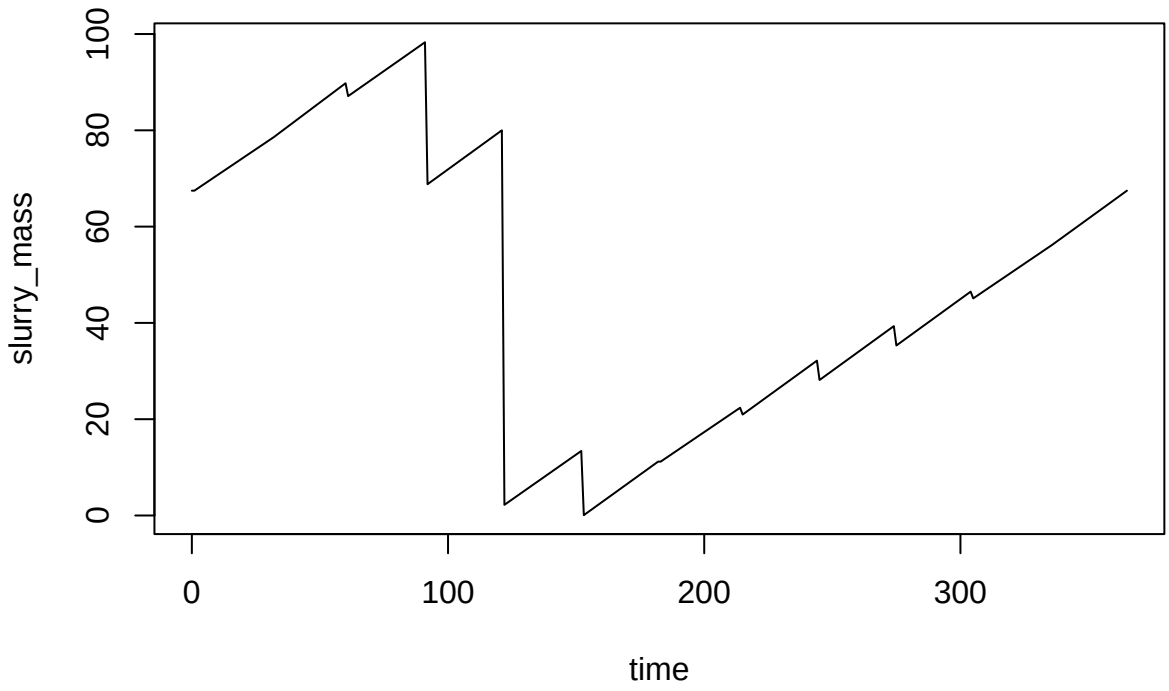
```
inf_sr <- list(VFA_fresh = 2,
          VFA_init = 2,
          comps = c('SO4'),
                  comp_fresh = c(SO4 = 10),
                  comp_init = c(SO4 = 10),
                  pH = 7,
          dens = 1000)
```

Because it is a solute, we don't need to change the `chem_pars` contents (we would if it were a gas).

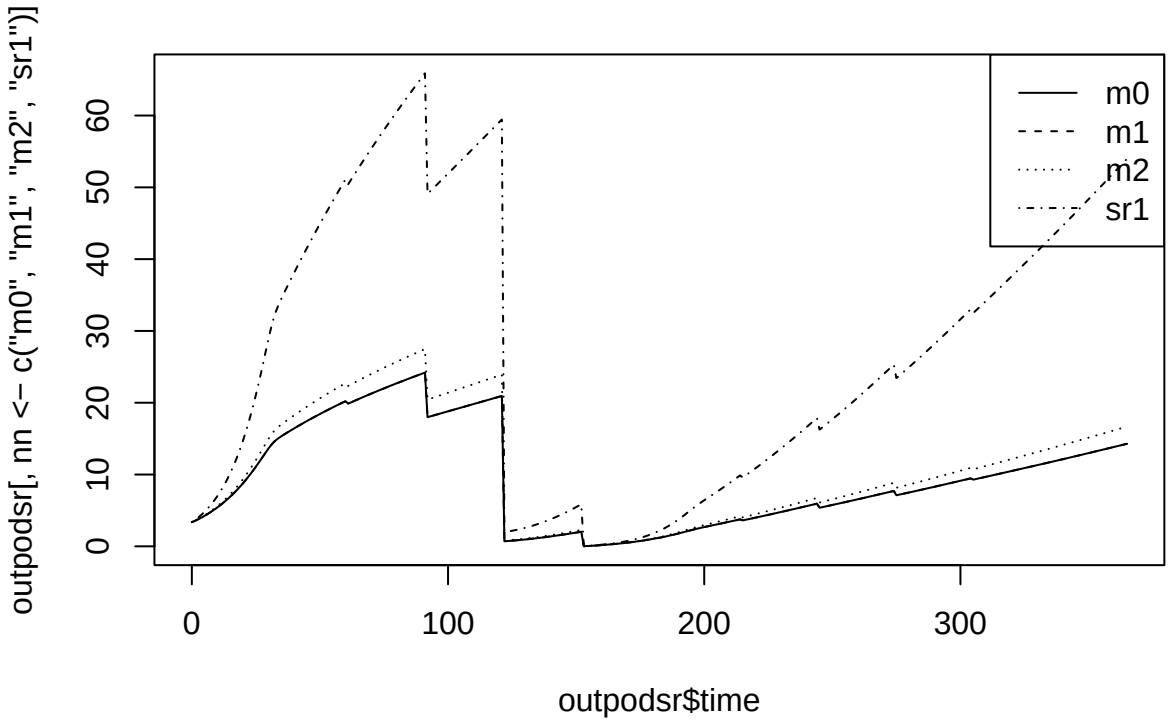
```
outpodsr <- abmneo(
  storage = stor_ser_ref,
  365,
  inf_pars = inf_sr,
  grp_pars = grp_sr,
  sub_pars = sub_ref,
  chem_pars = chem_ref
)
```

```
## Warning in check_COD(dat = dat, grps = pars$grps, subs = pars$subs, COD_conv =
## pars$COD_conv, : COD balance is off by 51%
```

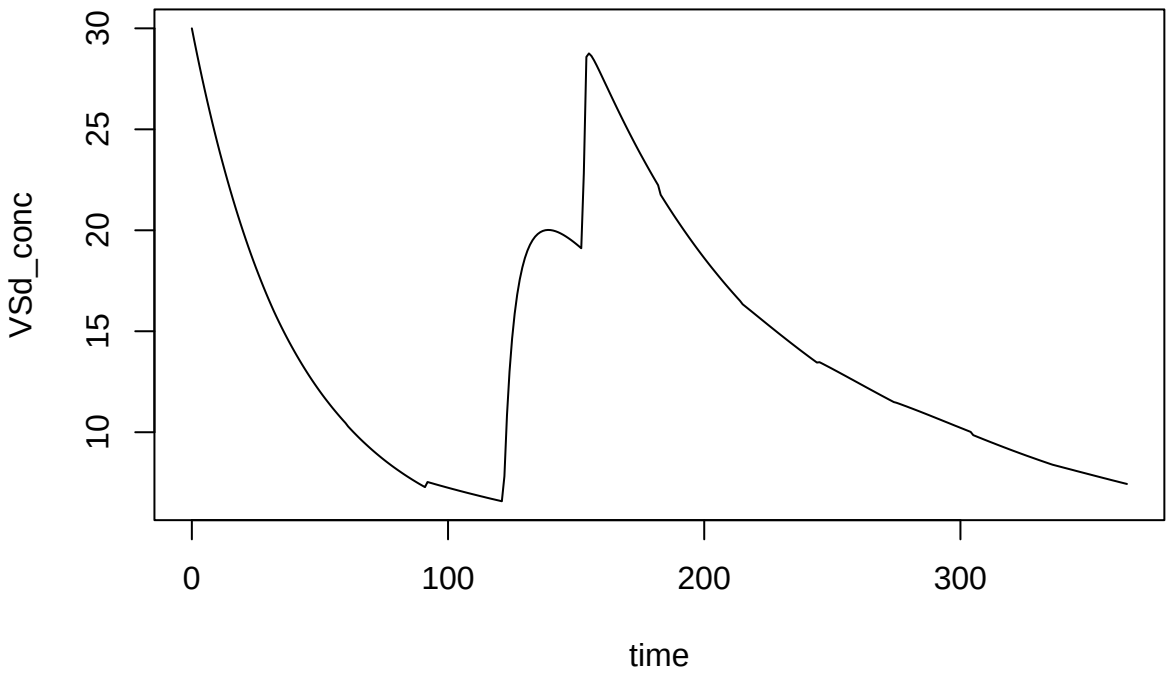
```
plot(slurry_mass ~ time, data = outpodsr, type = 'l')
```



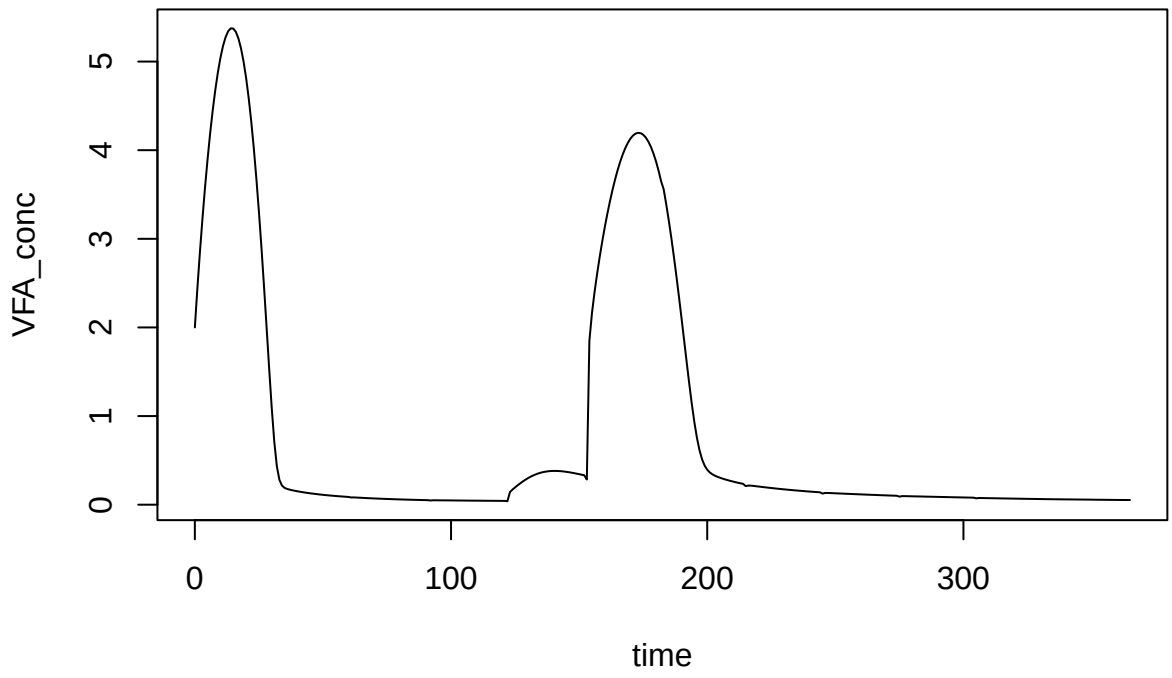

```
matplot(outpodsr$time, outpodsr[, nn <- c('m0','m1','m2', 'sr1')], col = 'black', lty = 1:length(nn), ty
legend('topright', legend = nn, lty = 1:length(nn))
```



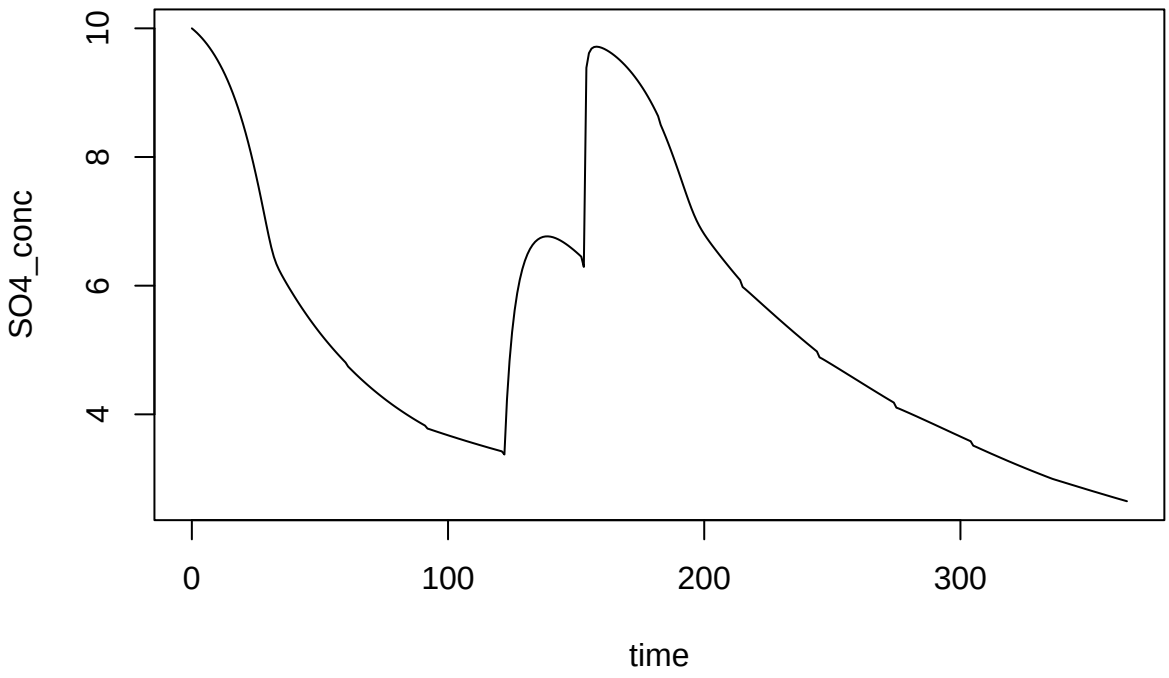

```
plot(VSd_conc ~ time, data = outpodsr, type = 'l')
```




```
plot(VFA_conc ~ time, data = outpodsr, type = 'l')
```




```
plot(SO4_conc ~ time, data = outpodsr, type = 'l')
```



Notice that the sulfate reduction reaction is not complete, because the products are missing. Neither is methanogenesis, for that matter. Additional arbitrary chemical products can be added easily (and would be included in default parameter sets presumably).

Let's add H2S and CO2. The micstoich package can help sort out stoichiometry easily.

```
library(micstoich)
r1 <- micstoich(donor = 'CH3COOH', acceptor = 'SO4-2', product = 'H2S')
r1
```

```
## CH3COOH   SO4-2      H+     H2S     CO2     H2O     HS-
## -0.1250 -0.1250 -0.1875  0.0625  0.2500  0.2500  0.0625
```

But these are molar, and we need COD etc. based coefficients. We need the coefficient on CO2 in g C, and g S on H2S and SO4.

First, combine H2S and HS- (see https://github.com/AU-BCE-EE/micstoich/issues/2).

```
r1['H2S'] <- r1['H2S'] + r1['HS-']
r1 <- r1[names(r1) != 'HS-']
r1
```

```
## CH3COOH   SO4-2      H+     H2S     CO2     H2O
## -0.1250 -0.1250 -0.1875  0.1250  0.2500  0.2500
```

```
r1mc <- r1
for (i in seq_along(r1)) {
  mc <- micstoich:::massconv(names(r1)[i])
  r1mc[i] <- mc * r1[i]
}
```

```
r1mc
```

```
## CH3COOH   SO4-2      H+     H2S     CO2     H2O
## -8.0000 -4.0125 -1.5000  2.0000  3.0025  4.5040
```

Let's check some of them (I don't trust them yet).

```
calcCOD('SO4-2')
```

```
## [1] -0.6659729
```

```
molmass('S')
```

```
## [1] 32.1
```

```
0.1250 * 32.1
```

```
## [1] 4.0125
```

OK.

```
calcCOD('CH3COOH') * 0.125
```

```
## [1] 0.1332179
```

OK.

Now CO2.

```
0.25 * 12
```

```
## [1] 3
```

OK.

But note that SO4-2 and H2S have different bases!

```
calcCOD('H2S')
```

```
## [1] 0.4689882
```

We need to discuss this issue.

Then convert to 1 g VFA COD basis.

```
r1mc / -r1mc[1]
```

```
##    CH3COOH      SO4-2         H+        H2S        CO2        H2O
## -1.0000000 -0.5015625 -0.1875000  0.2500000  0.3753125  0.5630000
```

```
mstoich_src <- matrix(
  c(-1,    -1,    -1,     -1,
     0,     0,     0,     -0.5016,
     0.187, 0.187, 0.187,  0.3753,
     0,     0,     0,      0.25,
     1,     1,     1,      0),
  nrow = 5,
  byrow = TRUE,
  dimnames = list(
    c('VFA', 'SO4', 'CO2', 'H2S', 'CH4'),
    c('m0', 'm1', 'm2', 'sr1')
  )
)

mstoich_src
```

```
##         m0     m1     m2     sr1
## VFA -1.000 -1.000 -1.000 -1.0000
## SO4  0.000  0.000  0.000 -0.5016
## CO2  0.187  0.187  0.187  0.3753
## H2S  0.000  0.000  0.000  0.2500
## CH4  1.000  1.000  1.000  0.0000
```

We also need to add these new components:

```
inf_src <- list(VFA_fresh = 2,
          VFA_init = 2,
          comps = c('SO4', 'H2S', 'CO2'),
                  comp_fresh = c(SO4 = 10, H2S = 0, CO2 = 0),
                  comp_init = c(SO4 = 10, H2S = 0, CO2 = 0),
                  pH = 7,
          dens = 1000)
```

```
chem_src <- list(COD_conv = c(CH4 = 5.32), gases = c('CH4', 'CO2', 'H2S'))
```

Let's try to use the old `add_pars` function instead of creating a new grp_pars list. We'll use that for the gases too.

```
devtools::load_all()
```

```
## i Loading abmneo
```

```
outpodsrc <- abmneo(
  storage = stor_ser_ref,
  365,
  inf_pars = inf_src,
  grp_pars = grp_sr,
  sub_pars = sub_ref,
  chem_pars = chem_src,
  add_pars = list(mstoich = mstoich_src)
)
```

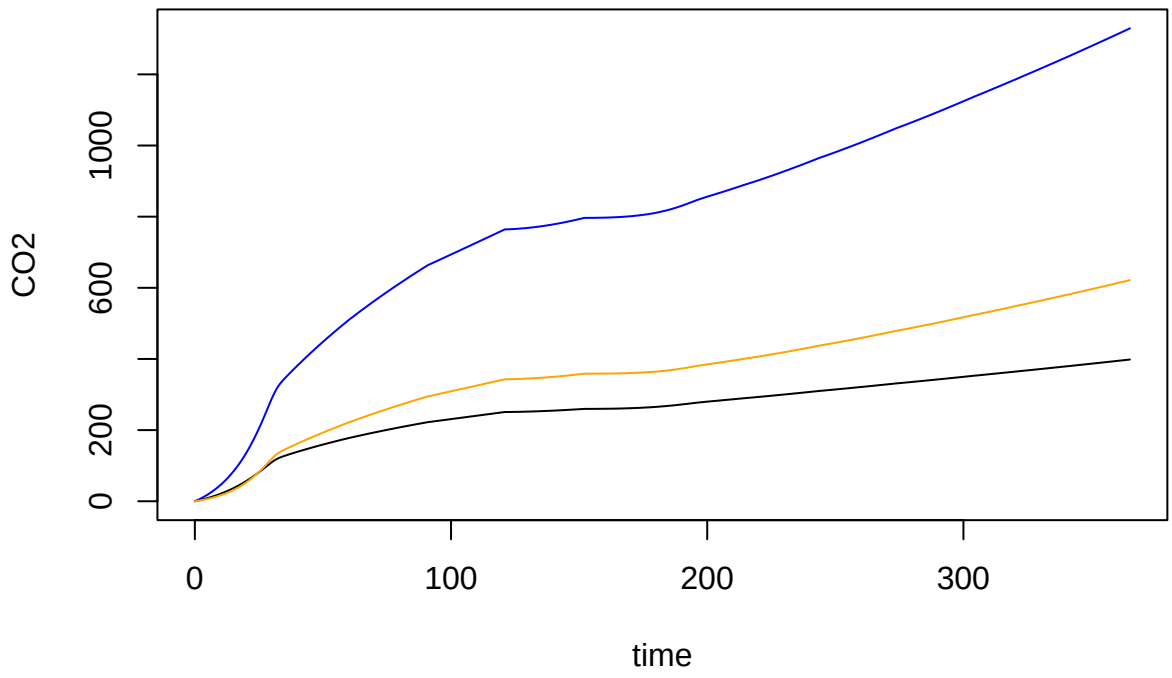
```
## Warning in check_COD(dat = dat, grps = pars$grps, subs = pars$subs, COD_conv =
## pars$COD_conv, : COD balance is off by 51%
```

Notice that the COD balance is off now. SO4-2 consumes it.

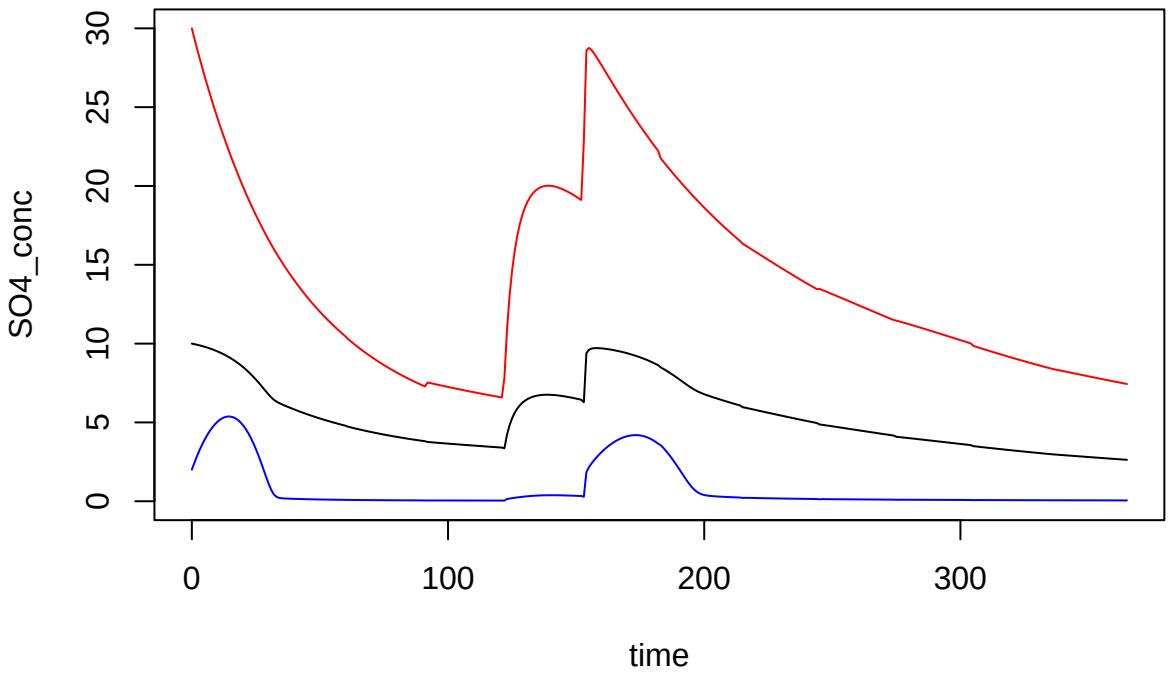
(Also note that adding gases CO2 and H2S in add_pars didn't work–look into this.)

Remember that all this could be added to default parameter sets!

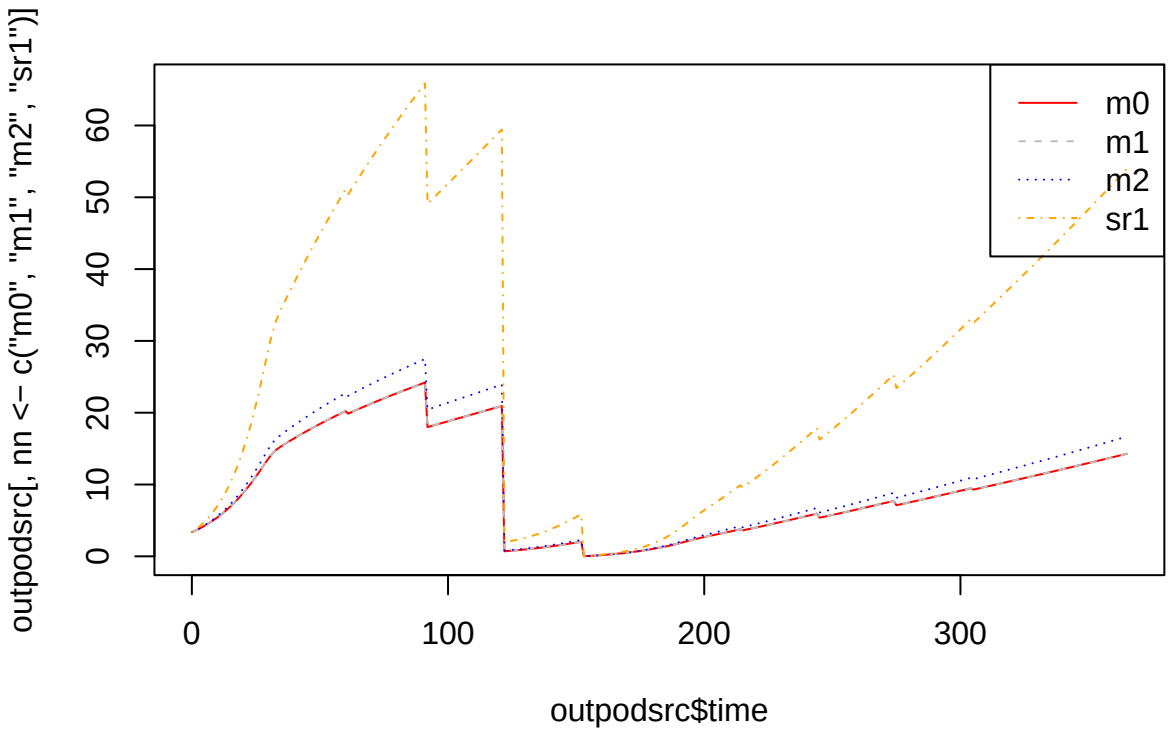
```
plot(CO2 ~ time, data = outpodsrc, type = 'l', col = 'blue')
lines(CH4 ~ time, data = outpodsrc, type = 'l')
lines(H2S ~ time, data = outpodsrc, type = 'l', col = 'orange')
```




```
plot(SO4_conc ~ time, data = outpodsrc, type = 'l', ylim = c(0, 30))
lines(VFA_conc ~ time, data = outpodsrc, col = 'blue')
lines(VSd_conc ~ time, data = outpodsrc, col = 'red')
```




```
matplot(outpodsrc$time, outpodsrc[, nn <- c('m0', 'm1', 'm2', 'sr1')], col = cc <- c('red', 'gray', 'blu
legend('topright', legend = nn, col = cc, lty = 1:length(nn))
```



Repeat without any sulfate for sulfate reducers.

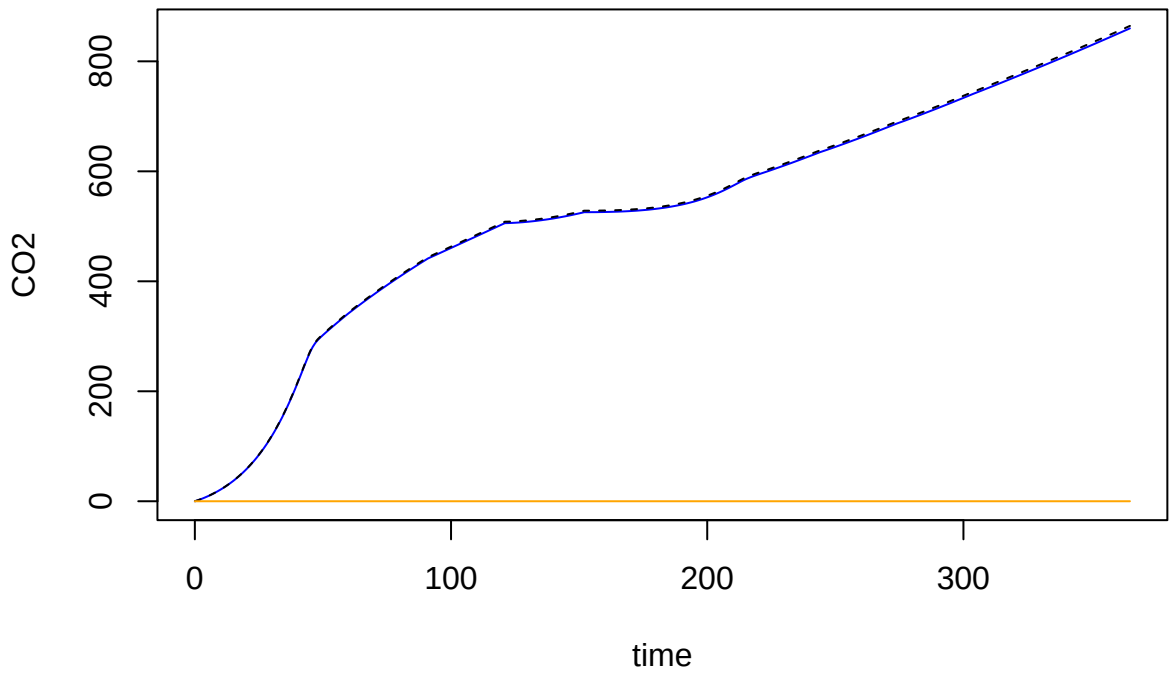
```
devtools::load_all()
```

```
## i Loading abmneo
```

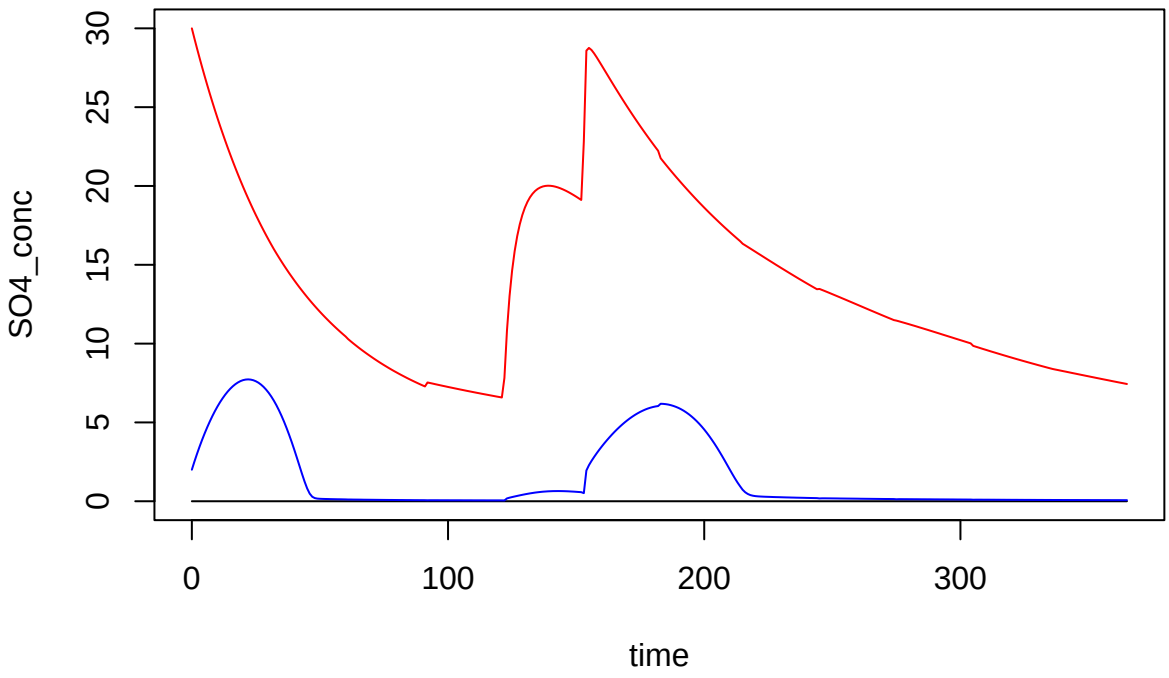
```
outpodsrn <- abmneo(
  storage = stor_ser_ref,
  365,
  inf_pars = inf_src,
  grp_pars = grp_sr,
  sub_pars = sub_ref,
  chem_pars = chem_src,
  add_pars = list(mstoich = mstoich_src, comp_fresh.SO4 = 0, comp_init.SO4 = 0)
)
```

```
## Warning in check_COD(dat = dat, grps = pars$grps, subs = pars$subs, COD_conv =
## pars$COD_conv, : COD balance is off by 2.8%
```

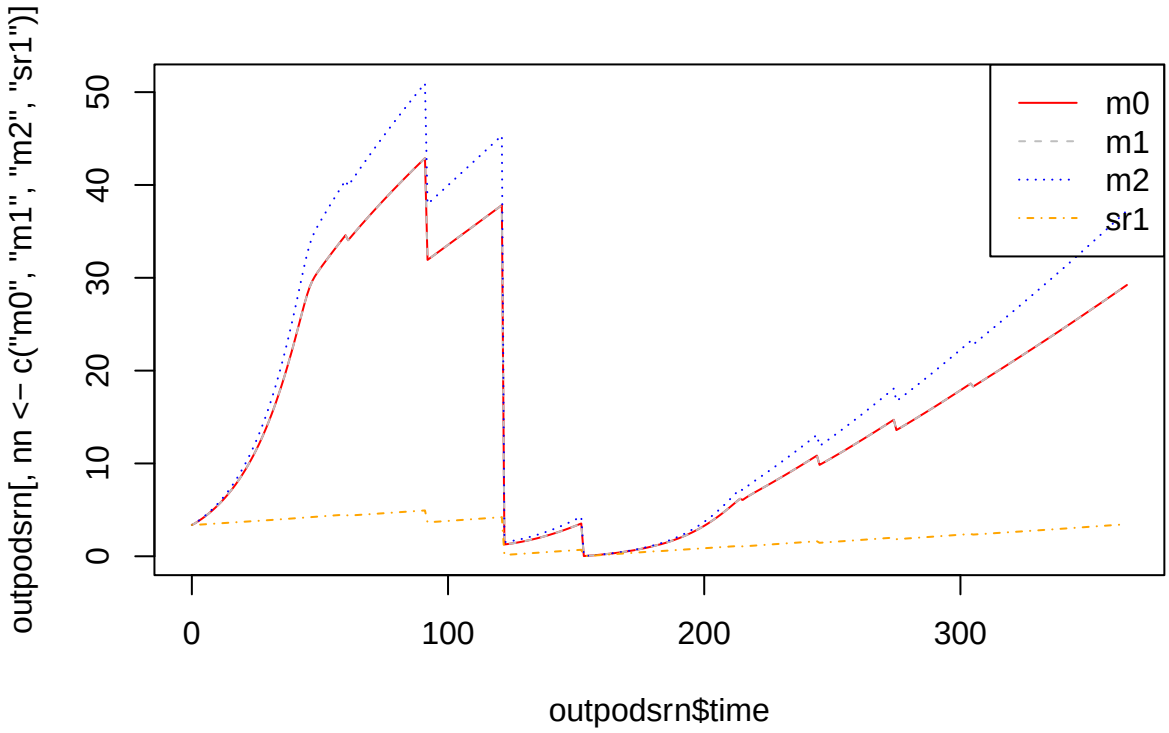
```
plot(CO2 ~ time, data = outpodsrn, type = 'l', col = 'blue')
lines(CH4 ~ time, data = outpodsrn, type = 'l', lty = 2)
lines(H2S ~ time, data = outpodsrn, type = 'l', col = 'orange')
```




```
plot(SO4_conc ~ time, data = outpodsrn, type = 'l', ylim = c(0, 30))
lines(VFA_conc ~ time, data = outpodsrn, col = 'blue')
lines(VSd_conc ~ time, data = outpodsrn, col = 'red')
```




```
matplot(outpodsrn$time, outpodsrn[, nn <- c('m0', 'm1', 'm2', 'sr1')], col = cc <- c('red', 'gray', 'blu
legend('topright', legend = nn, col = cc, lty = 1:length(nn))
```



Remember all these additions are through input changes only, and the `rates()` code is simple.

Time-variable inputs part 2

Almost any model parameter can now be varied over time through var_pars. If it is a single value without named elements for any time, it can be combined in the var data frame. This excludes anything having to do with emptying, loading rate, or slurry level–if any of those need to vary non-regularly over time, the storage argument must be used! So, that leaves probably only pH and temperature. For example, we could simulate acidification with this.

```
tp_dat <- data.frame(
  time = c(0, 30, 100, 200, 250),
  temp_C = c(10, 20, 25, 15, 10),
  pH = c(7, 6.5, 5, 5, 5)
)
```

```
devtools::load_all()
```

```
## i Loading abmneo
```

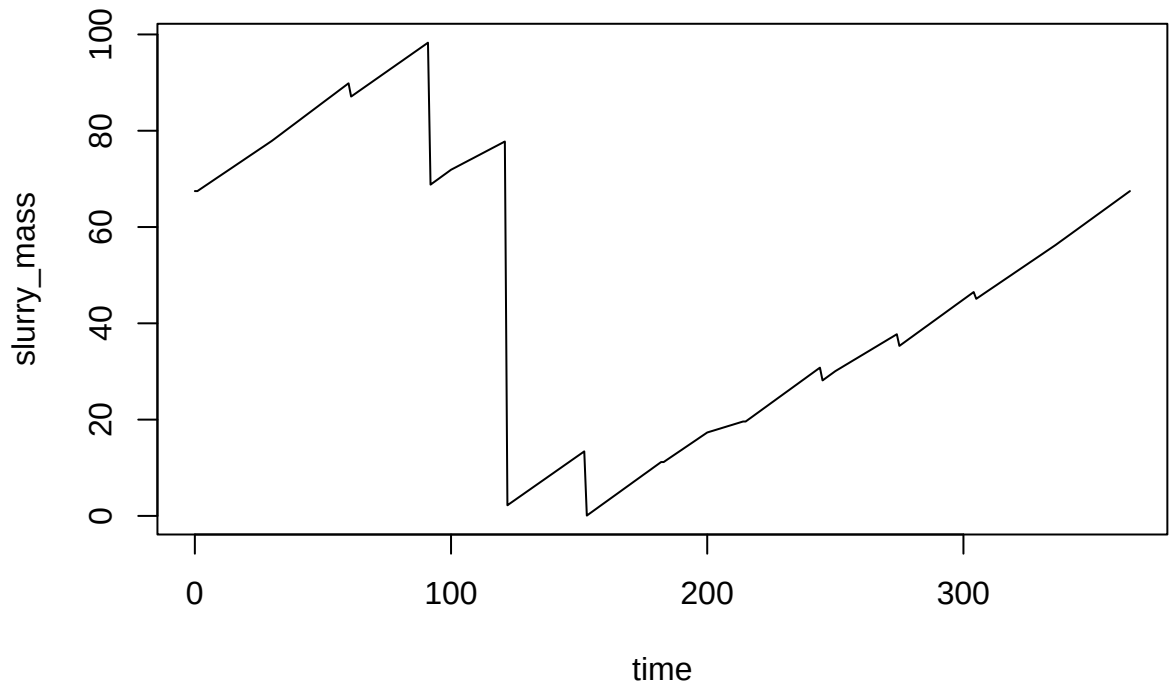
```
outpodvtp <- abmneo(
  storage = stor_ser_ref,
  365,
  inf_pars = inf_ref,
  grp_pars = grp_ref,
  sub_pars = sub_ref,
  chem_pars = chem_ref,
```

```
  var_pars = list(var = tp_dat)
)
```

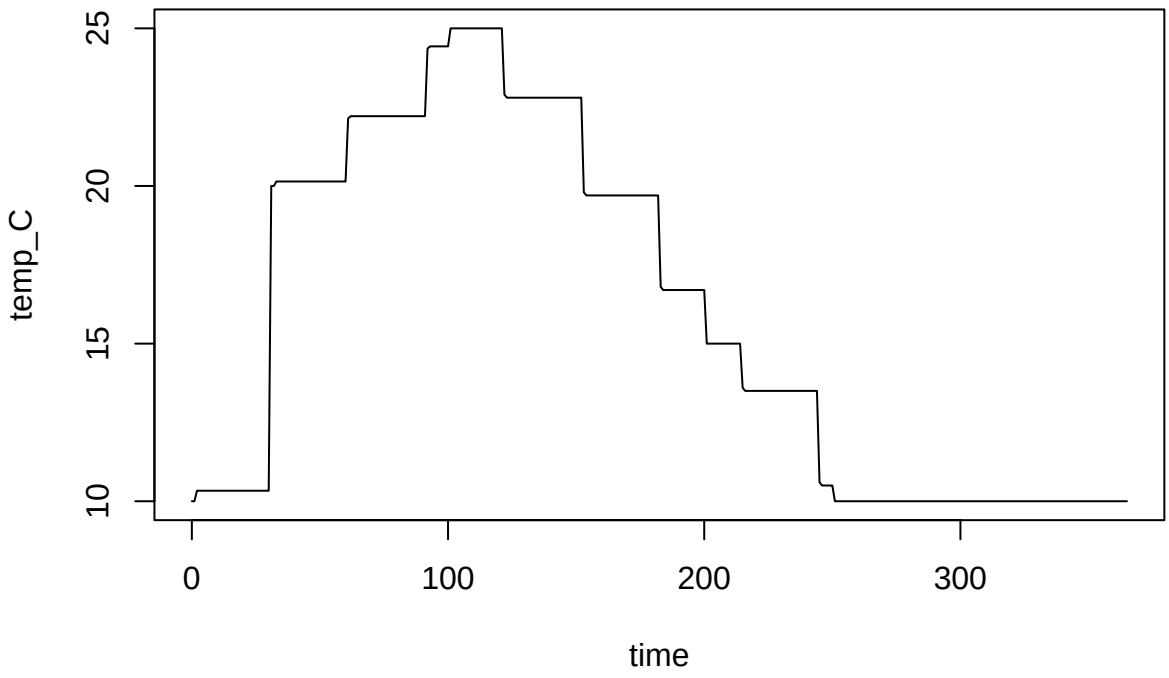
```
## Warning in empty_store(y, resid_mass = pars$resid_mass, resid_enrich =
## pars$resid_enrich, : Emptying skipped.
```

```
## Warning in check_COD(dat = dat, grps = pars$grps, subs = pars$subs, COD_conv =
## pars$COD_conv, : COD balance is off by 2.7%
```

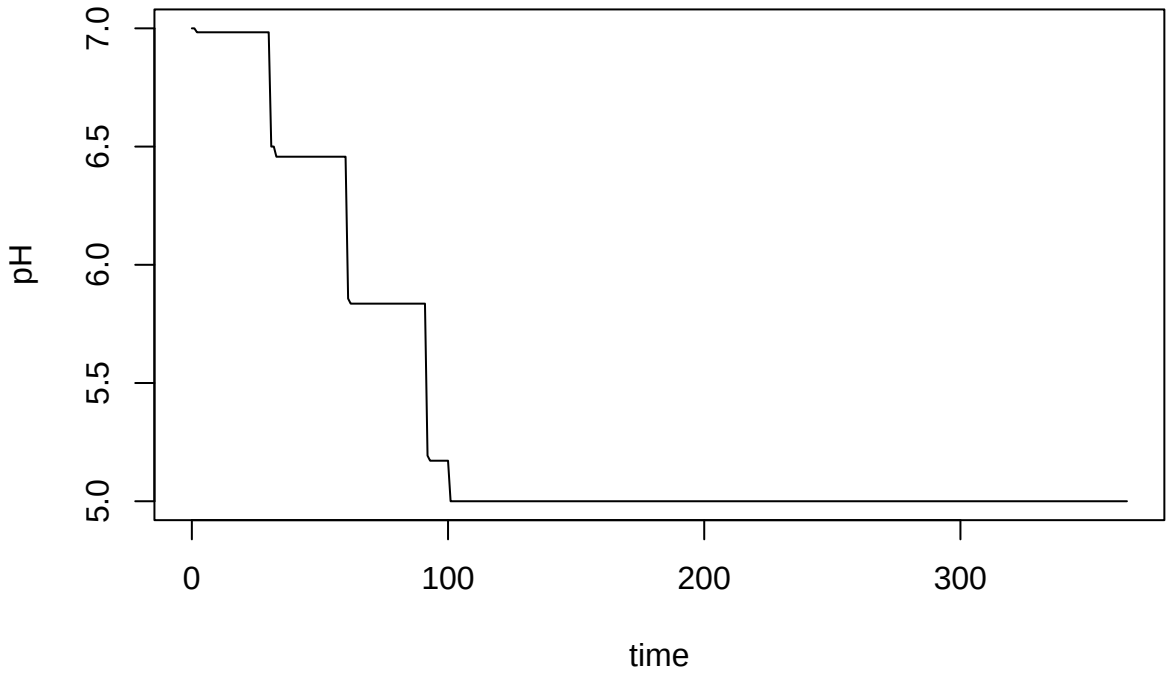
```
plot(slurry_mass ~ time, data = outpodvtp, type = 'l')
```



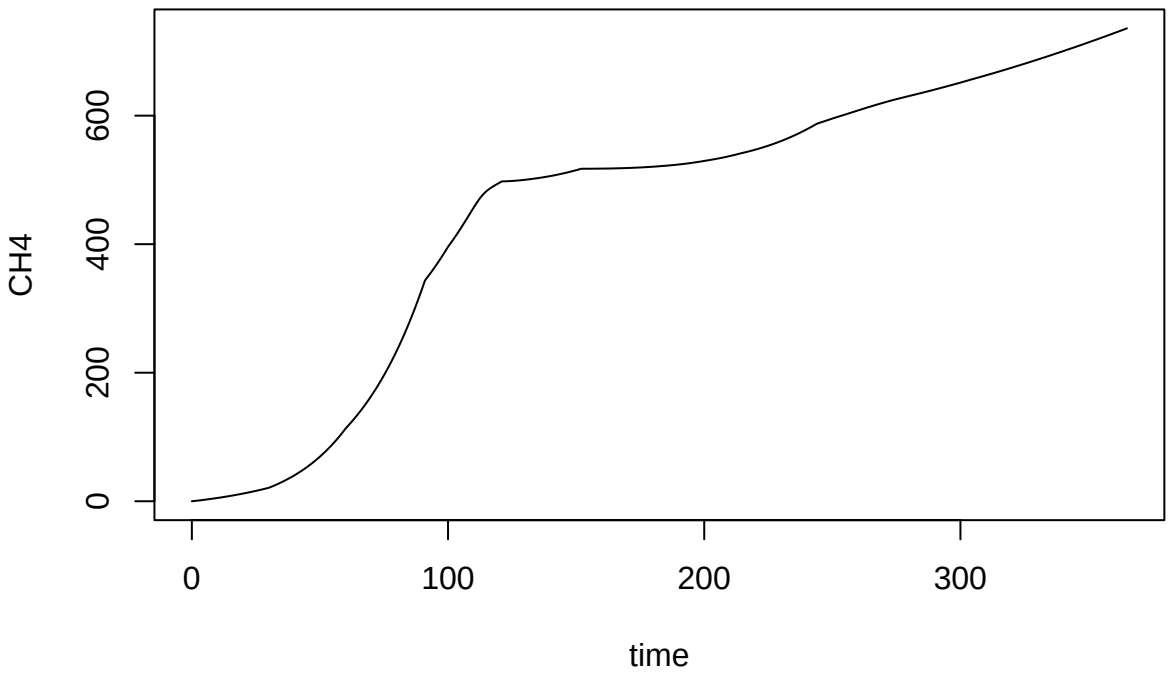

```
plot(temp_C ~ time, data = outpodvtp, type = 'l')
```




```
plot(pH ~ time, data = outpodvtp, type = 'l')
```




```
plot(CH4 ~ time, data = outpodvtp, type = 'l')
```



For other inputs that are set in named elements, make a separate data frame for each other argument with column names that give the element names. This gives us a lot of flexibility. For example, we could simulate inhibition here.

```
qq <- cbind(time = c(0, 30, 100),
                m0 = 0.5, m1 = 1, m2 = 1)
qq[, -1] <- qq[, -1] * c(1, 0.2, 0.1)
qhat_dat <- data.frame(qq)
```

Then combine them all in a list, using the parameter element names for element names.

```
var_inhib <- list(var = tp_dat, qhat_opt = qhat_dat)
```

Note that times do not need to be the same! The data frames are merged and missing values are interpolated or filled forward (we need to align the language for this and mid etc.)

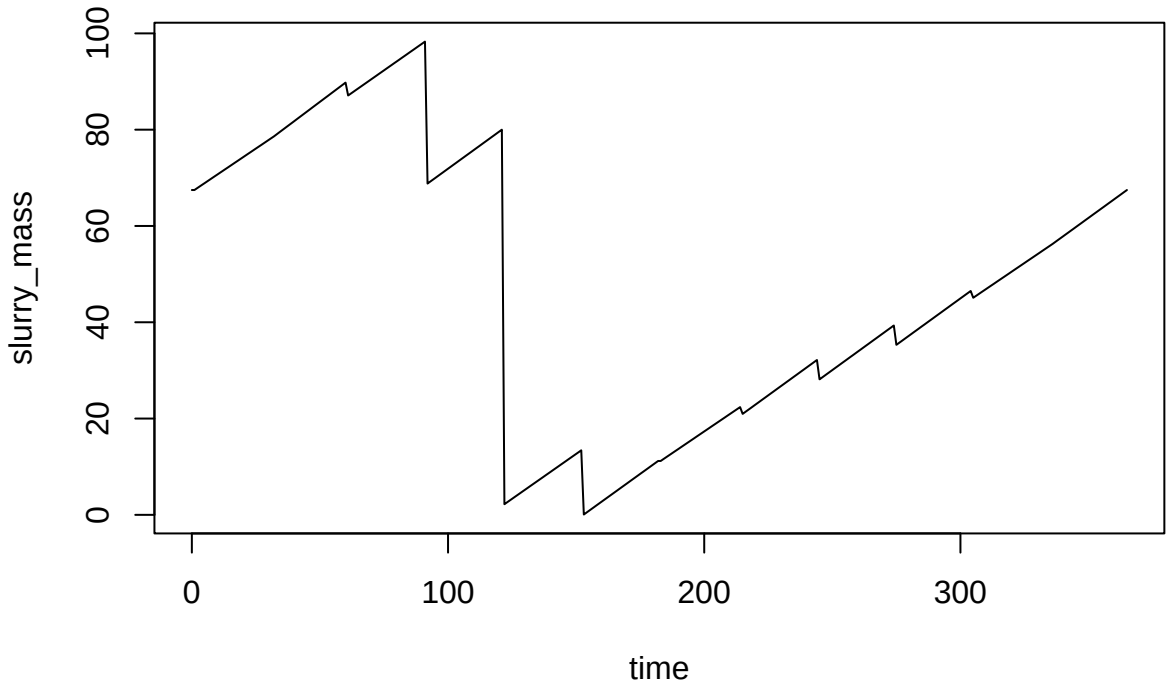
```
devtools::load_all()
```

```
## i Loading abmneo
```

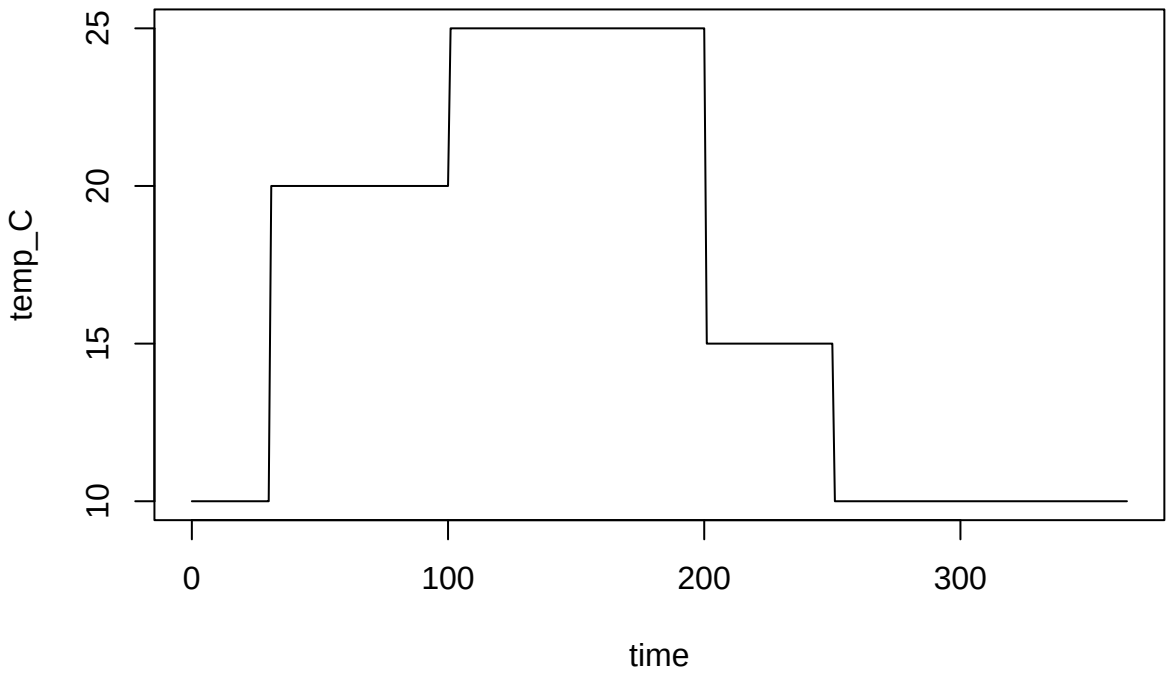
```
outpodvib <- abmneo(
  storage = stor_ser_ref,
  365,
  inf_pars = inf_ref,
  grp_pars = grp_ref,
  sub_pars = sub_ref,
  chem_pars = chem_ref,
  var_pars = var_inhib
)
```

```
## Warning in clean_series(series, pars, days): ctrl_pars element fill_method
## "interp" cannot be used with list elements, so reverting to "forward"
```

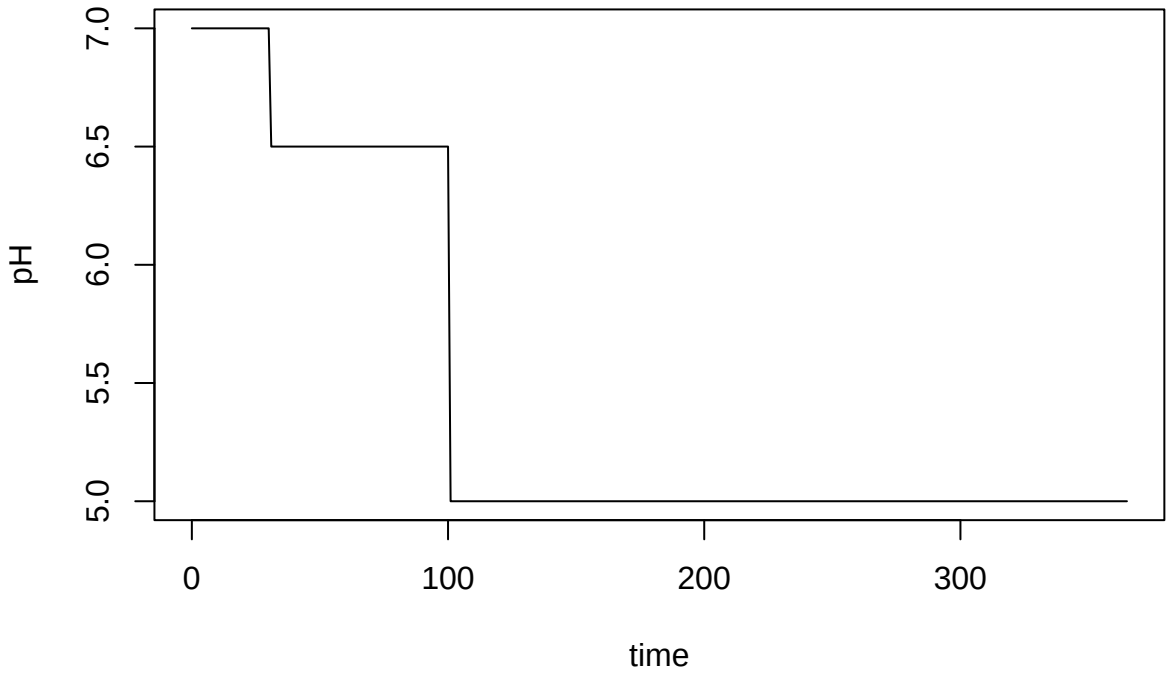
```
plot(slurry_mass ~ time, data = outpodvib, type = 'l')
```



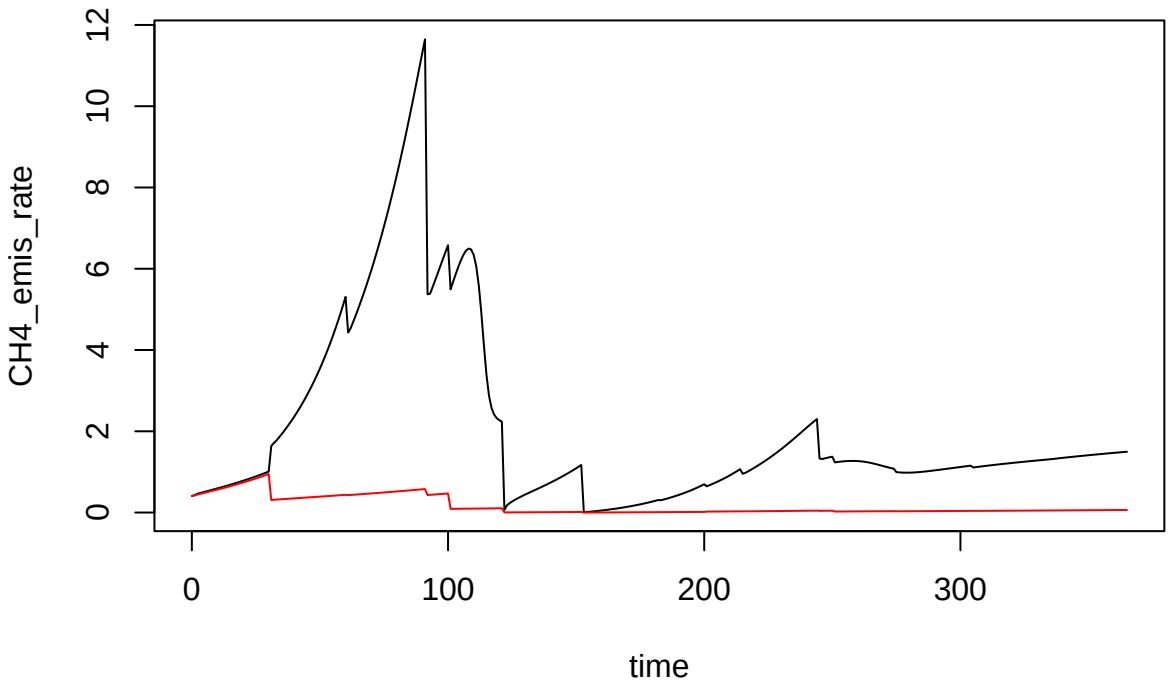

```
plot(temp_C ~ time, data = outpodvib, type = 'l')
```




```
plot(pH ~ time, data = outpodvib, type = 'l')
```




```
plot(CH4_emis_rate ~ time, data = outpodvtp, type = 'l')
lines(CH4_emis_rate ~ time, data = outpodvib, col = 'red')
```



That is a lot of flexibility, I think.

The startup argument

Works as before.

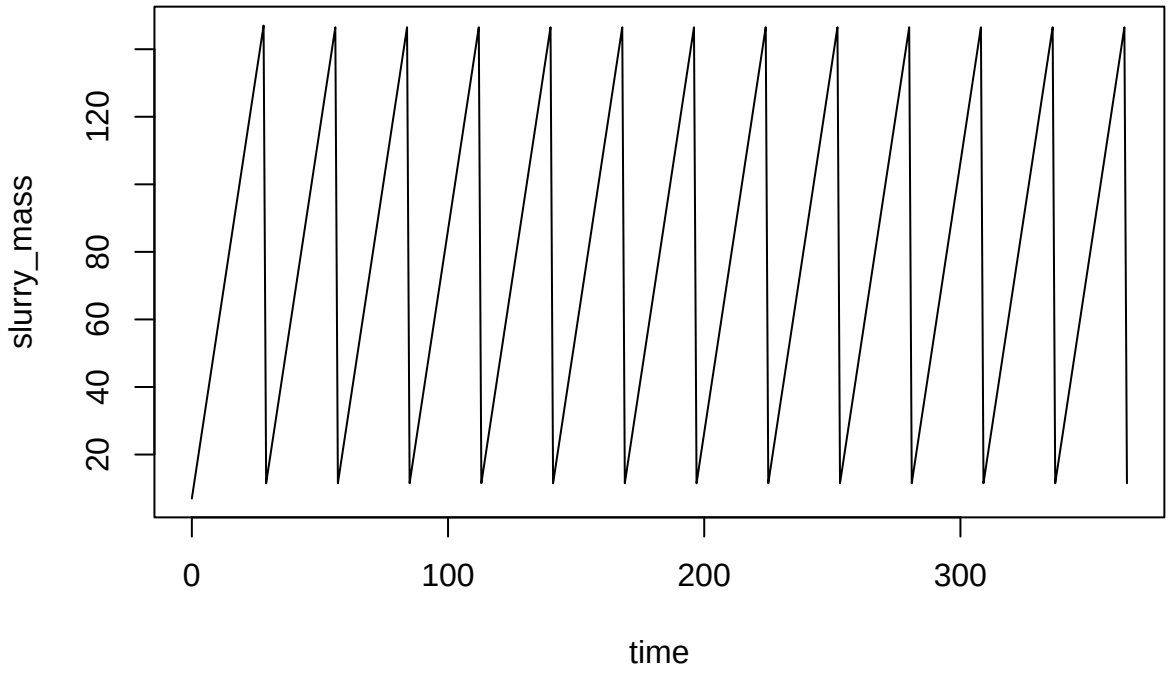
```
devtools::load_all()
```

```
## i Loading abmneo
```

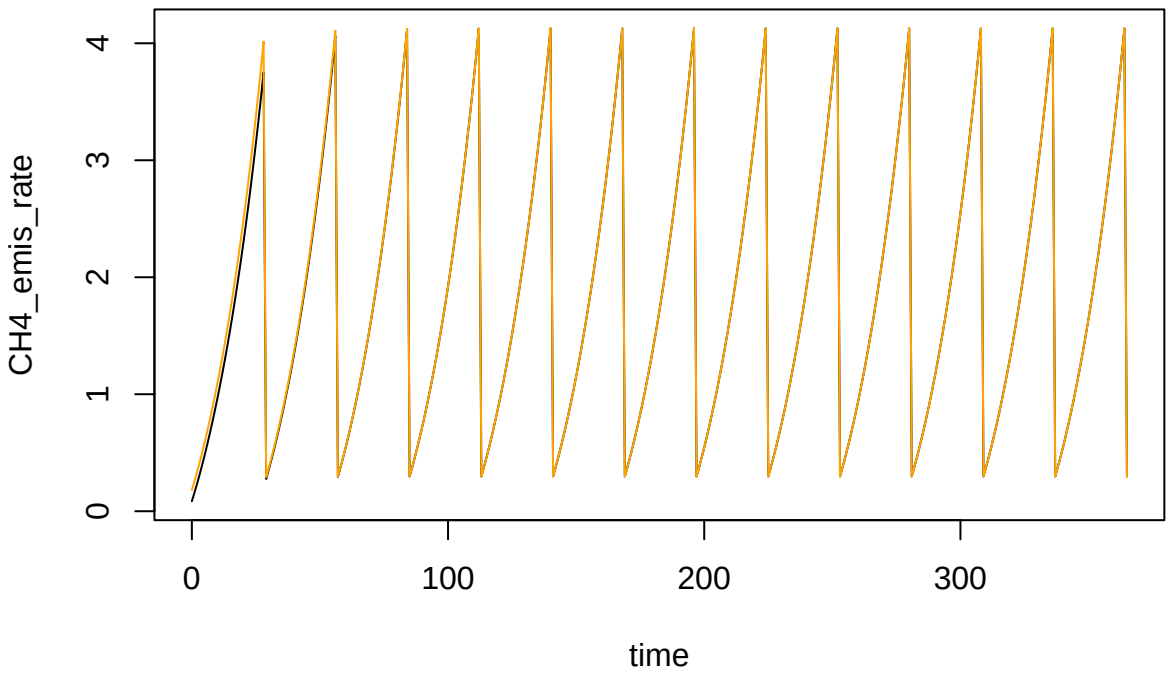
```
outpins <- abmneo(
  storage = stor_reg_ref,
  365,
  inf_pars = inf_ref,
  grp_pars = grp_ref,
  sub_pars = sub_ref,
  chem_pars = chem_ref,
  startup = 3
)
```

```
##
## Startup run 1x -> 2x ->
## Using starting conditions from `starting` argument
## 3x ->
## Using starting conditions from `starting` argument
```

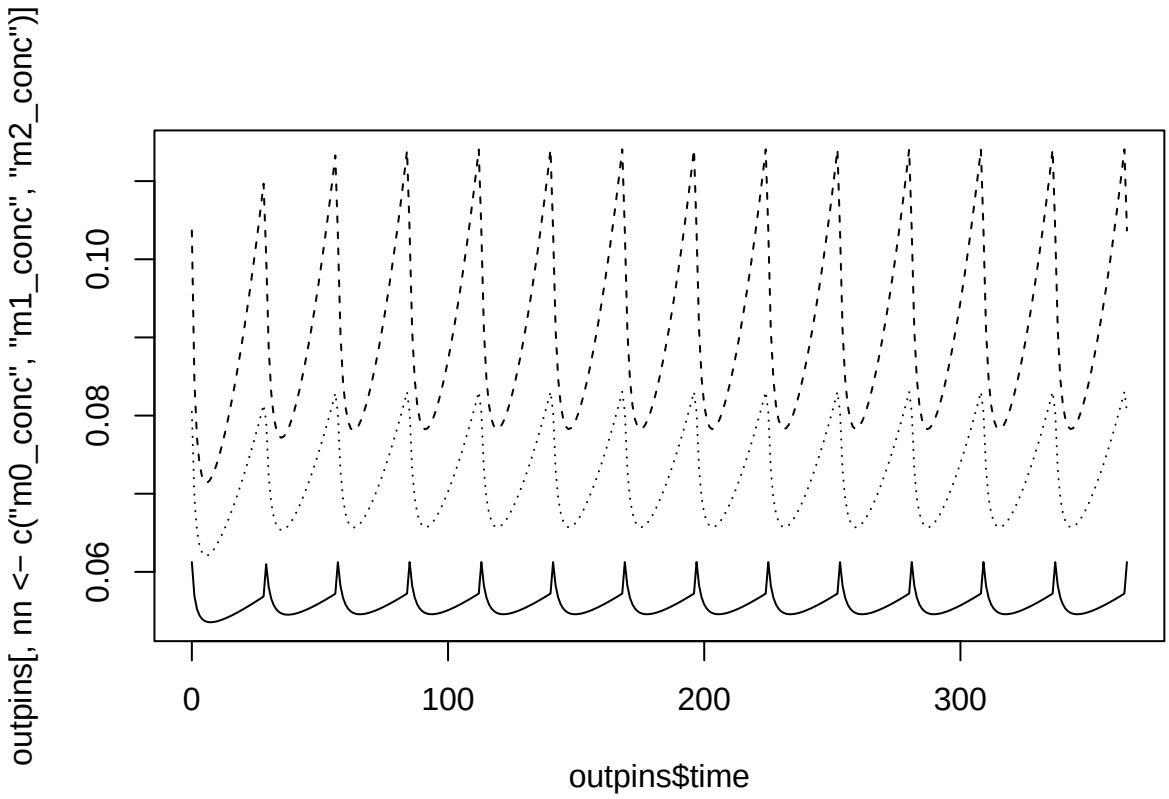
```
## and final run
## Using starting conditions from `starting` argument
plot(slurry_mass ~ time, data = outpinr, type = 'l')
```



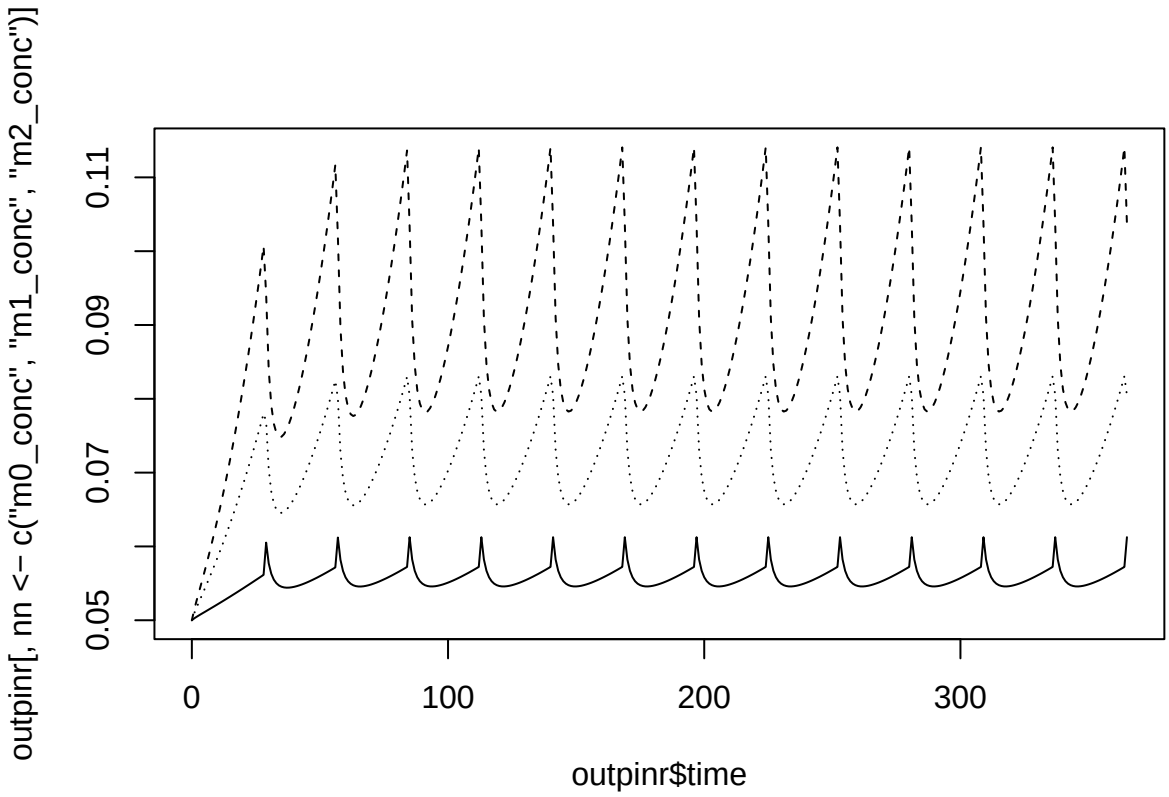

```
plot(CH4_emis_rate ~ time, data = outpinr, type = 'l')
lines(CH4_emis_rate ~ time, data = outpins, col = 'orange')
```




```
matplot(outpins$time, outpins[, nn <- c('m0_conc','m1_conc','m2_conc')], col = 'black', lty = 1:length(
```




```
matplot(outpinr$time, outpinr[, nn <- c('m0_conc','m1_conc','m2_conc')], col = 'black', lty = 1:length(
```



Inhibition

Now we have inhibition. I went for the IC0/IC100 approach. And I thought of a clever way to have inhibition at both low and high inhibitor levels (needed for at least pH) that fit into this approach very well.

```
ic0 <- matrix(
  c(8, 8, 8,
    8, 8, 8,
    6, 6, 6),
  nrow = 3,
  byrow = TRUE,
  dimnames = list(
    c('VFA_conc', 'pH', 'pH'),
    c('m0', 'm1', 'm2')
  )
)

ic100 <- matrix(
  c(15, 15, 15,
    9, 9, 9,
    5, 5, 5),
  nrow = 3,
  byrow = TRUE,
  dimnames = list(
    c('VFA_conc', 'pH', 'pH'),
```

```
    c('m0', 'm1', 'm2')
  )
)


ic0
```

```
##          m0 m1 m2
## VFA_conc  8  8  8
## pH        8  8  8
## pH        6  6  6
```

```
ic100
```

```
##          m0 m1 m2
## VFA_conc 15 15 15
## pH        9  9  9
## pH        5  5  5
```

Notice how pH is entered twice and reversed? These inhibition parameter matrices go into the `grp_pars` list.

```
grp_inhib <- list(
  grps = c('m0', 'm1', 'm2'),
  yield = c(default = 0.05),
  xa_fresh = c(default = 0.05),
  xa_init = c(default = 0.05),
  dd_rate = c(default = 0.02),
  qhat_opt = c(m0 = 0.5, m1 = 1, m2 = 1),
  T_opt = c(m0 = 10, m1 = 18, m2 = 28),
  T_min = c(m0 = 0, m1 = 5, m2 = 10),
  T_max = c(m0 = 25, m1 = 25, m2 = 38),
  ksmat = ksmat_ref,
  mstoich = mstoich_ref,
  ic0 = ic0,
  ic100 = ic100
)
```

Here are some pH data for inhibition

```
pH_dat <- data.frame(
  time = c(0, 30, 100, 200, 250),
  pH = c(7, 6.5, 5, 5, 5)
)
```

Below we'll run with and without inhibition, and time. I worked hard to make the slowdown as small as possible (and Claude Code had some good suggestions).

With inhibition.

```
devtools::load_all()
```

```
## i Loading abmneo
```

```
system.time(
outpinin <- abmneo(
  storage = stor_reg_ref,
  365,
  inf_pars = inf_ref,
  grp_pars = grp_inhib,
```

```
  sub_pars = sub_ref,
  chem_pars = chem_ref,
  var_pars = list(var = pH_dat)
)
)
```

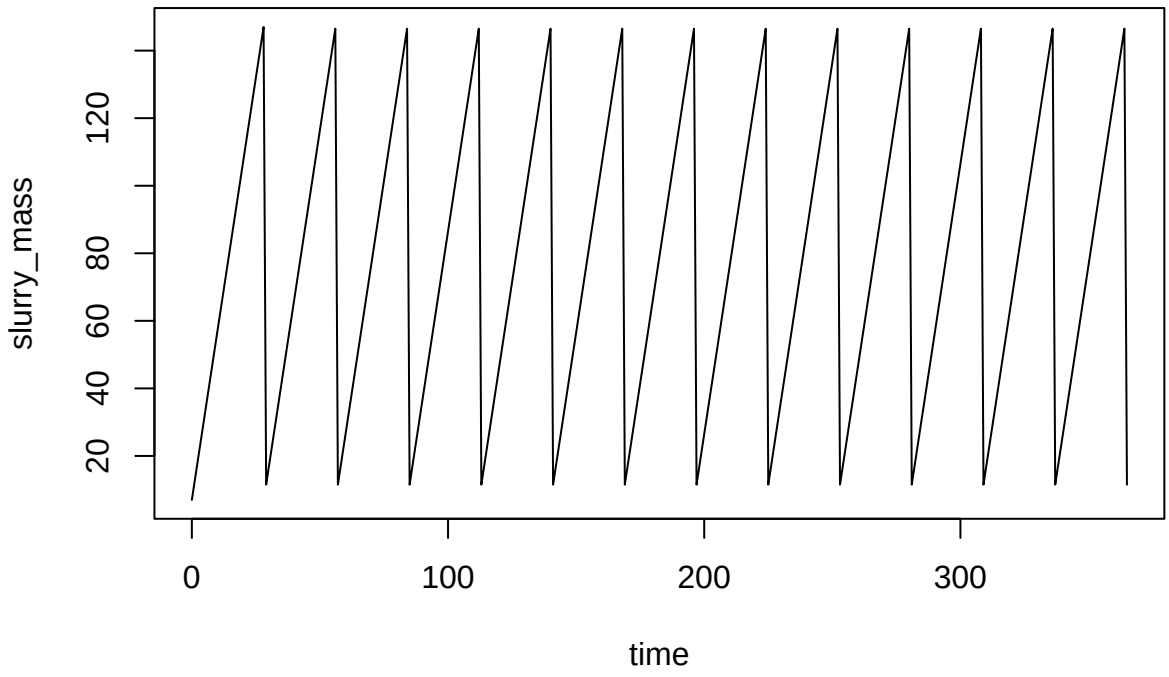
```
##    user  system elapsed
##   0.302   0.004   0.309
```

Without

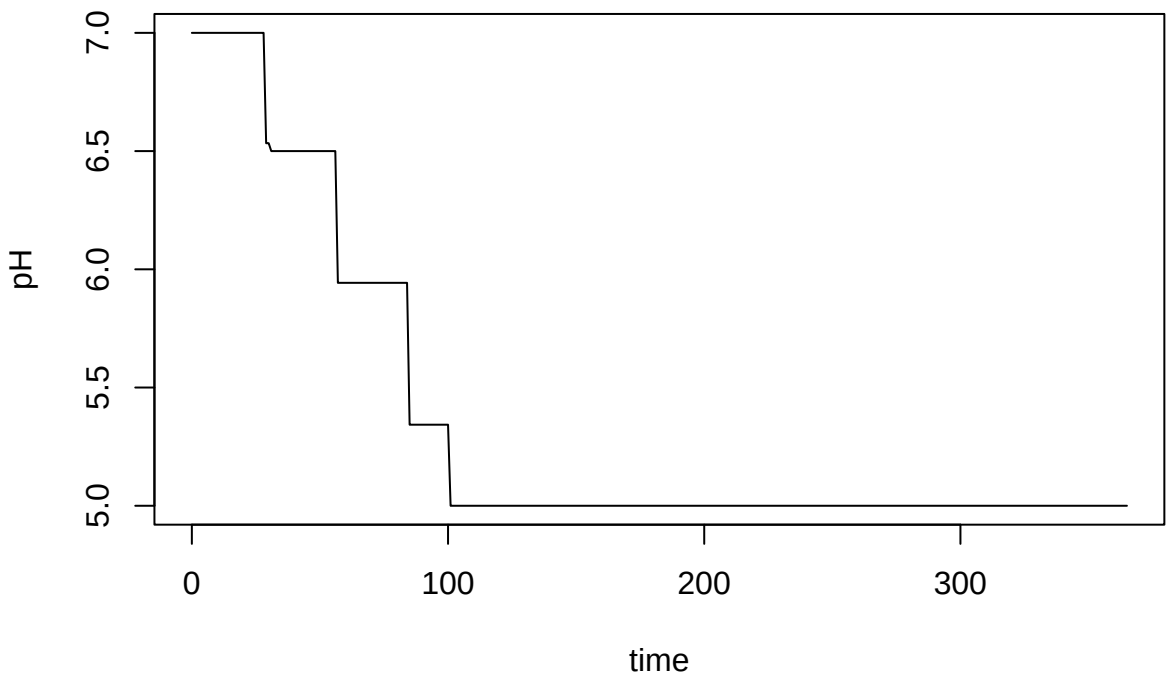
```
system.time(
outpinni <- abmneo(
  storage = stor_reg_ref,
  365,
  inf_pars = inf_ref,
  grp_pars = grp_ref,
  sub_pars = sub_ref,
  chem_pars = chem_ref,
  var_pars = list(var = pH_dat)
)
)
```

```
##    user  system elapsed
##   0.203   0.000   0.206
```

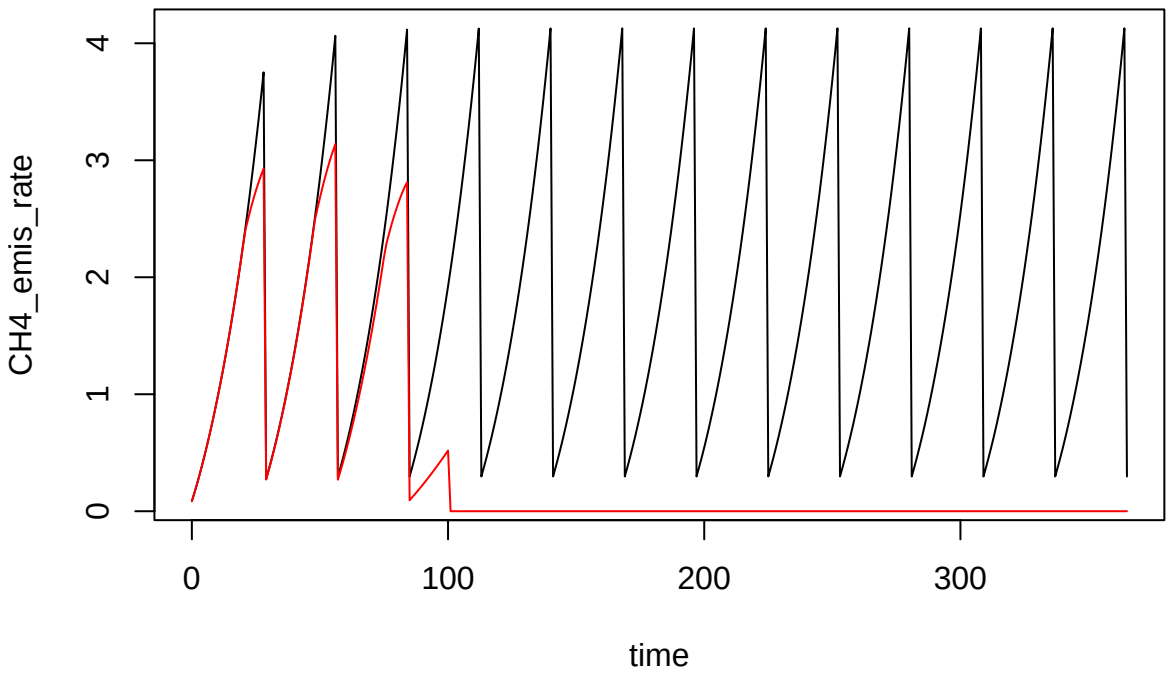
```
plot(slurry_mass ~ time, data = outpinin, type = 'l')
```



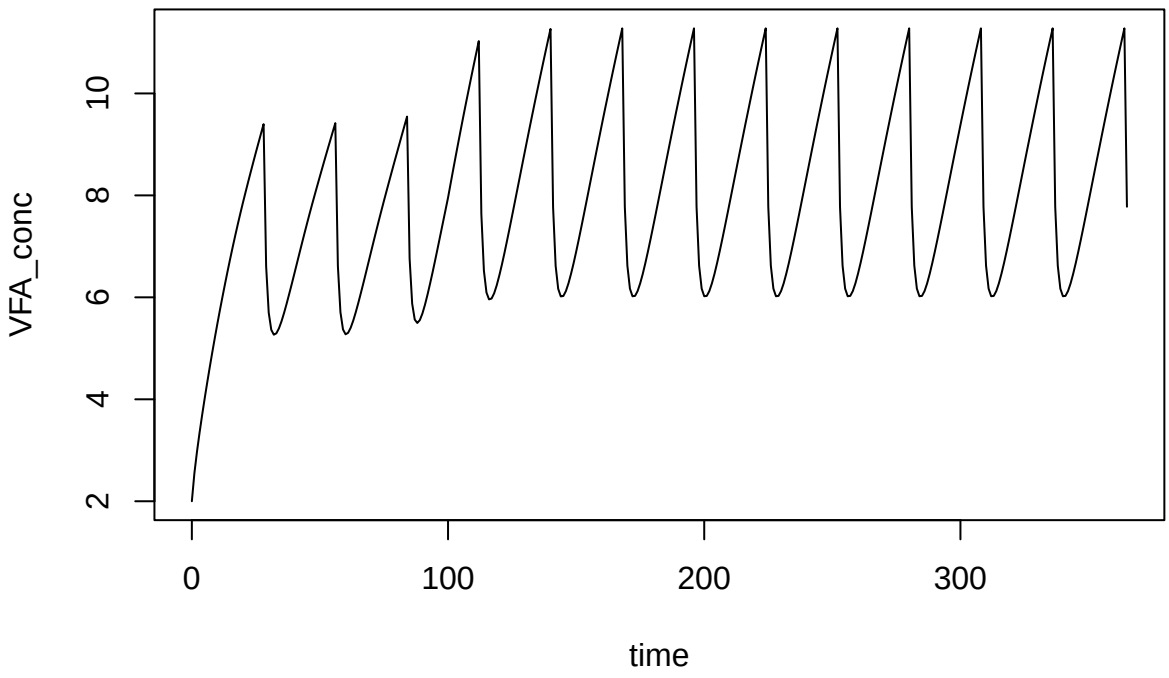
```
plot(pH ~ time, data = outpinin, type = 'l')
```




```
plot(CH4_emis_rate ~ time, data = outpinni, type = 'l')
lines(CH4_emis_rate ~ time, data = outpinin, col = 'red')
```




```
plot(VFA_conc ~ time, data = outpinin, type = 'l')
```



NEW Respiration

```
stor_reg_rsp <- list(
  type = 'regular',
  slurry_prod_rate = 5,
  storage_depth = 2,
  area = 0.65,
  temp_C = 20,
  resid_enrich = 0.2,
  slurry_mass = 7,
  resid_depth = 0.01,
  empty_int = 28,
  wash_water = 0,
  wash_int = 90,
  rest_d = 1,
  O2_flux = 2
)
```

```
devtools::load_all()
```

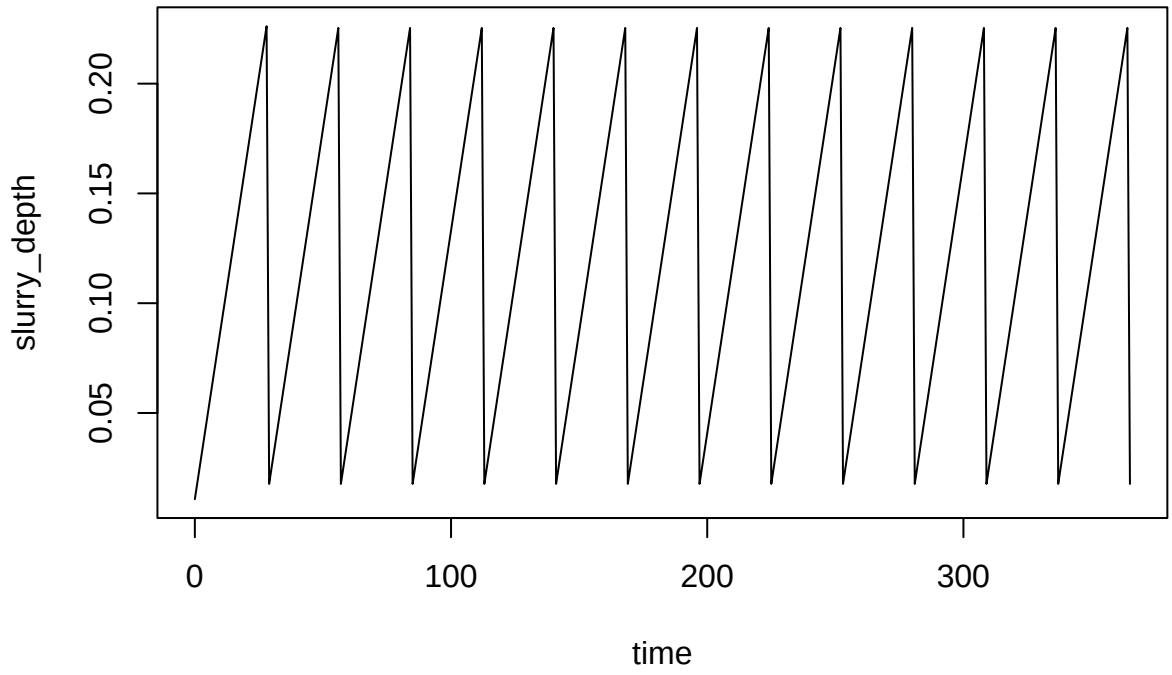
```
## i Loading abmneo
```

```
outpinrsp <- abmneo(
  storage = stor_reg_rsp,
  365,
  inf_pars = inf_ref,
```

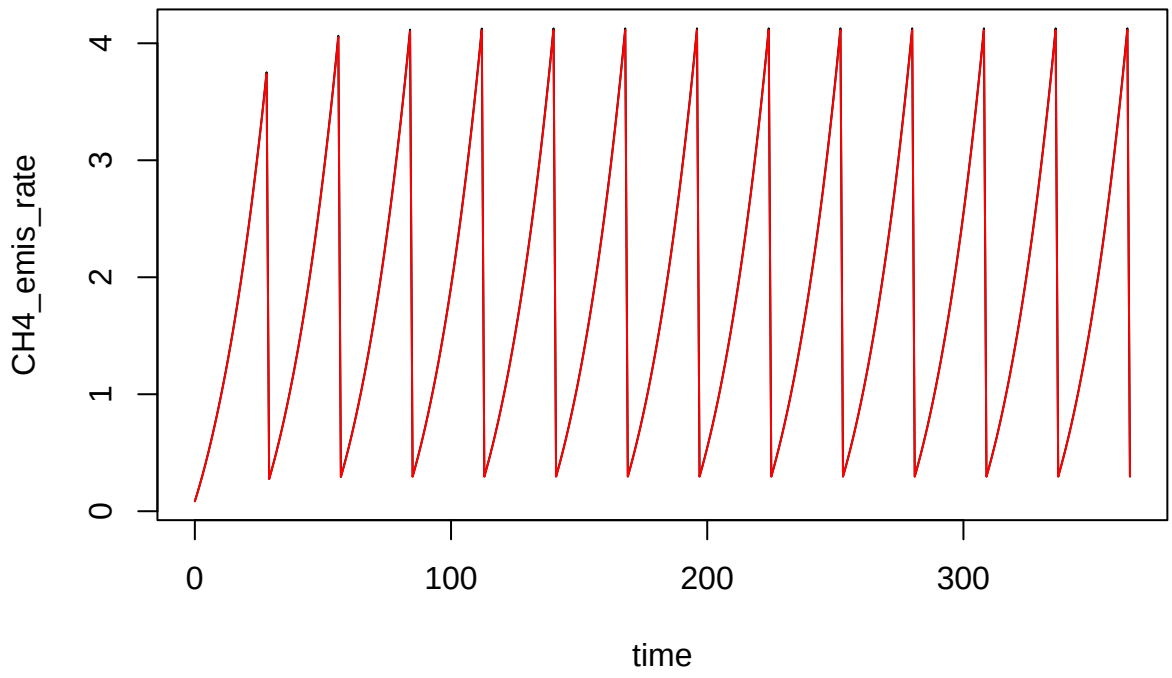
```
  grp_pars = grp_ref,
  sub_pars = sub_ref,
  chem_pars = chem_ref
)
```

```
## Warning in check_COD(dat = dat, grps = pars$grps, subs = pars$subs, COD_conv =
## pars$COD_conv, : COD balance is off by 1.6%
```

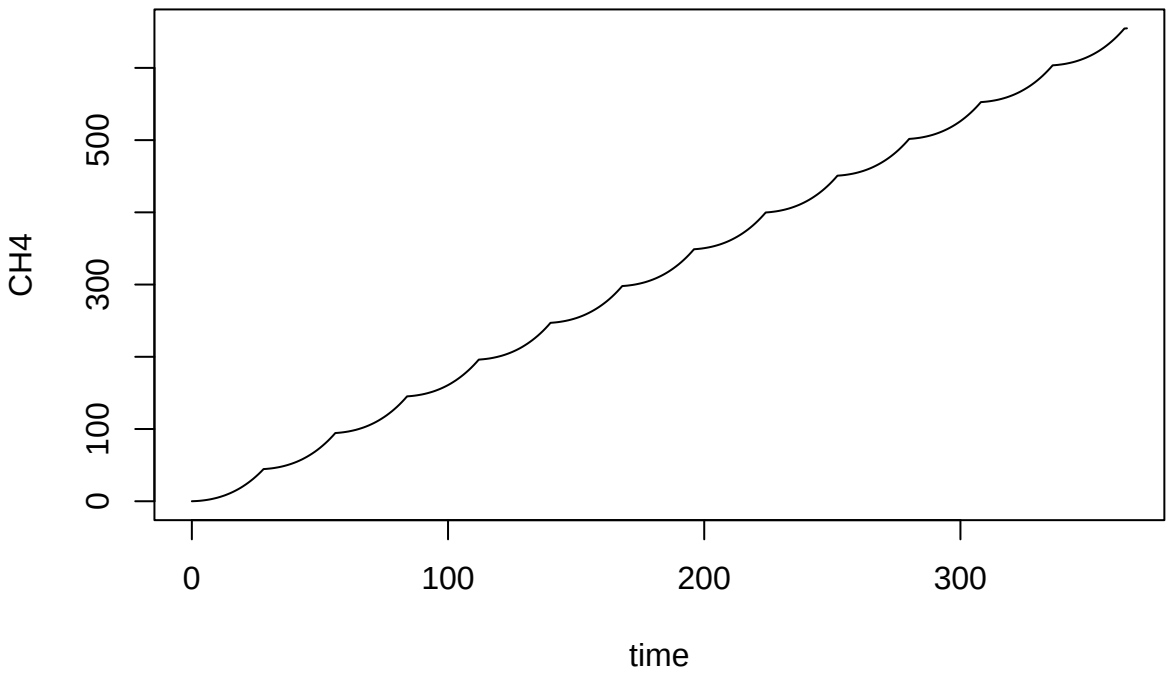
```
plot(slurry_depth ~ time, data = outpinr, type = 'l')
```



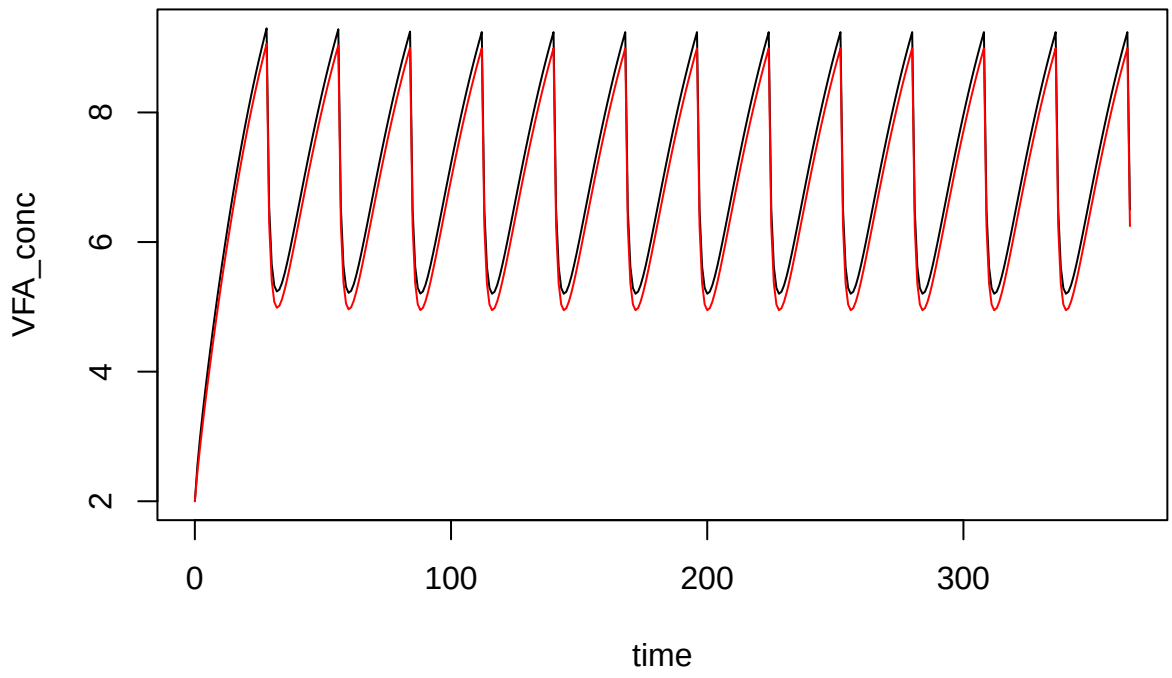

```
plot(CH4_emis_rate ~ time, data = outpinr, type = 'l')
lines(CH4_emis_rate ~ time, data = outpinrsp, col = 'red')
```




```
plot(CH4 ~ time, data = outpinr, type = 'l')
```




```
plot(VFA_conc ~ time, data = outpinr, type = 'l')
lines(VFA_conc ~ time, data = outpinrsp, col = 'red')
```



To add CO2 we need to explicitly define an `rstoich` vector and of course have CO2 as a component.

```
inf_rspc <- list(VFA_fresh = 2,
          VFA_init = 2,
          comps = c('CO2'),
                  comp_fresh = c(CO2 = 0),
                  comp_init = c(CO2 = 0),
                  pH = 7,
          dens = 1000)
```

The `rstoich` vector does not naturally fit well in any particular input! It is stuck with O2 flux in `storage`.

```
stor_reg_rspc <- list(
  type = 'regular',
  slurry_prod_rate = 5,
  storage_depth = 2,
  area = 0.65,
  temp_C = 20,
  resid_enrich = 0.2,
  slurry_mass = 7,
  resid_depth = 0.01,
  empty_int = 28,
  wash_water = 0,
  wash_int = 90,
  rest_d = 1,
  O2_flux = 2,
  rstoich = c(VFA = -1, CO2 = 0.375)
```

```
)
```

And CO2 is a gas.

```
chem_rspc <- list(COD_conv = c(CH4 = 5.32), gases = c('CH4', 'CO2'))
```

And if we are going to track CO2, we'd better include the methanogensis contribution. First sort out stoich coeff.

```
micstoich('CH3COOH', product = 'CH4')
```

```
## CH3COOH     CH4     CO2
##  -0.125   0.125   0.125
```

```
micstoich:::massconv('CO2') / micstoich:::massconv('CH3COOH')
```

```
## [1] 0.1876562
```

```
mstoich_rspc <- matrix(
  c(-1, -1, -1,
    1, 1, 1,
    0.1877, 0.1877, 0.1877),
  nrow = 3,
  byrow = TRUE,
  dimnames = list(
    c('VFA', 'CH4', 'CO2'),
    c('m0', 'm1', 'm2')
  )
)

mstoich_rspc
```

```
##          m0      m1      m2
## VFA -1.0000 -1.0000 -1.0000
## CH4  1.0000  1.0000  1.0000
## CO2  0.1877  0.1877  0.1877
```

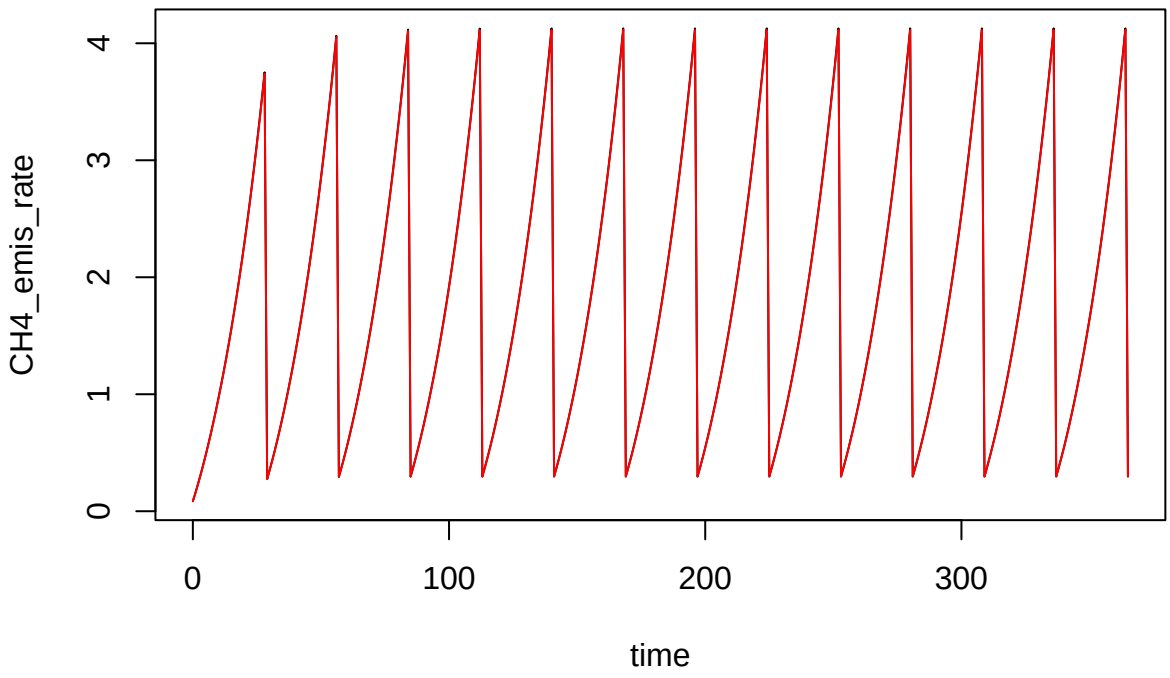
```
devtools::load_all()
```

```
## i Loading abmneo
```

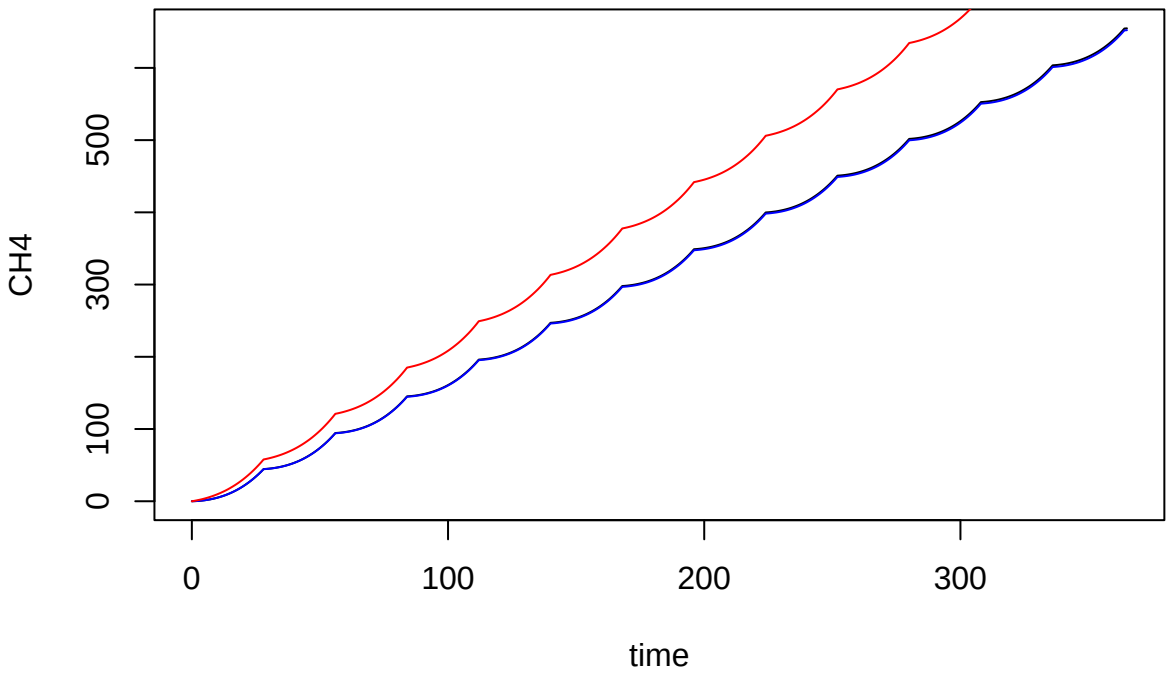
```
outpinrspc <- abmneo(
  storage = stor_reg_rspc,
  365,
  inf_pars = inf_rspc,
  grp_pars = grp_ref,
  sub_pars = sub_ref,
  chem_pars = chem_rspc,
  add_pars = list(mstoich = mstoich_rspc)
)
```

```
## Warning in check_COD(dat = dat, grps = pars$grps, subs = pars$subs, COD_conv =
## pars$COD_conv, : COD balance is off by 1.6%
```

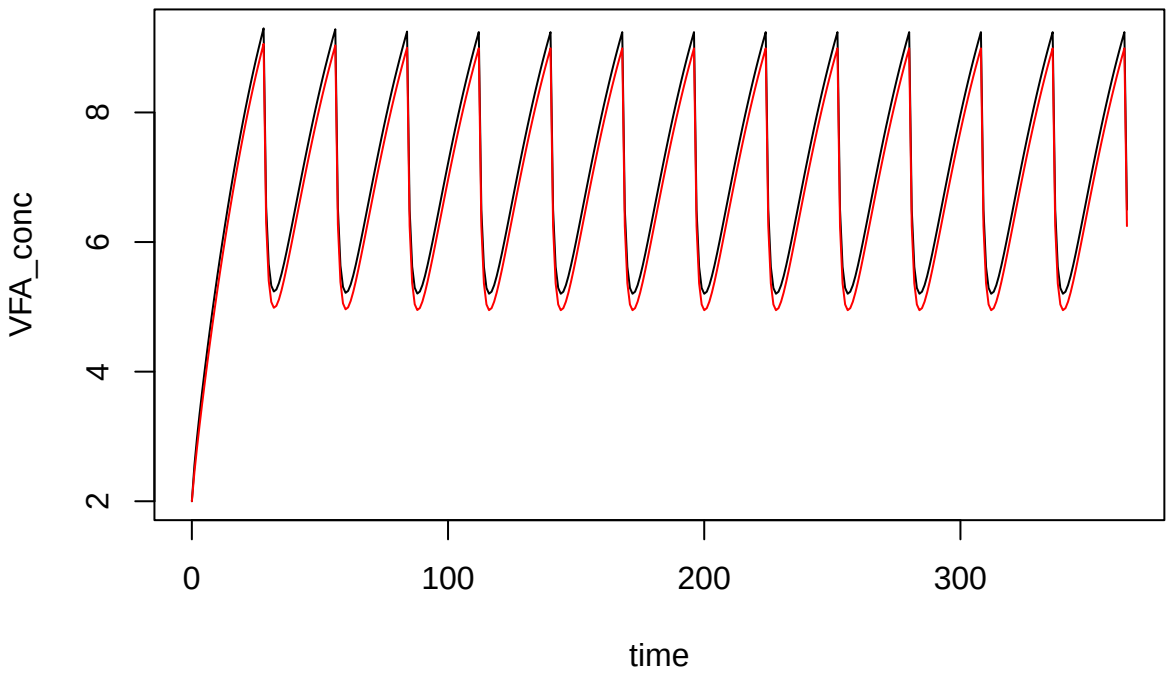
```
plot(CH4_emis_rate ~ time, data = outpinr, type = 'l')
lines(CH4_emis_rate ~ time, data = outpinrspc, col = 'red')
```




```
plot(CH4 ~ time, data = outpinr, type = 'l')
lines(CH4 ~ time, data = outpinrspc, col = 'blue')
lines(CO2 ~ time, data = outpinrspc, col = 'red')
```




```
plot(VFA_conc ~ time, data = outpinr, type = 'l')
lines(VFA_conc ~ time, data = outpinrsp, col = 'red')
```



NEW speed

Here is a demo of speed. 10 year regular simulation:

```
system.time(
abmneo(
  storage = stor_reg_ref,
  10 * 365,
  inf_pars = inf_ref,
  grp_pars = grp_ref,
  sub_pars = sub_ref,
  chem_pars = chem_ref
)
)
```

```
##    user  system elapsed
##   1.873   0.033   1.939
```

10 year series input simulation using startup.

```
system.time(
abmneo(
  storage = stor_ser_ref,
  365,
  inf_pars = inf_ref,
  grp_pars = grp_ref,
  sub_pars = sub_ref,
```

```
  chem_pars = chem_ref,
  startup = 9
)
)
```

```
##
## Startup run 1x ->
## Warning in check_COD(dat = dat, grps = pars$grps, subs = pars$subs, COD_conv =
## pars$COD_conv, : COD balance is off by 2.7%
## 2x ->
## Using starting conditions from `starting` argument
## Warning in check_COD(dat = dat, grps = pars$grps, subs = pars$subs, COD_conv =
## pars$COD_conv, : COD balance is off by 4%
## 3x ->
## Using starting conditions from `starting` argument
## Warning in check_COD(dat = dat, grps = pars$grps, subs = pars$subs, COD_conv =
## pars$COD_conv, : COD balance is off by 4%
## 4x ->
## Using starting conditions from `starting` argument
## Warning in check_COD(dat = dat, grps = pars$grps, subs = pars$subs, COD_conv =
## pars$COD_conv, : COD balance is off by 4%
## 5x ->
## Using starting conditions from `starting` argument
## Warning in check_COD(dat = dat, grps = pars$grps, subs = pars$subs, COD_conv =
## pars$COD_conv, : COD balance is off by 4%
## 6x ->
## Using starting conditions from `starting` argument
## Warning in check_COD(dat = dat, grps = pars$grps, subs = pars$subs, COD_conv =
## pars$COD_conv, : COD balance is off by 4%
## 7x ->
## Using starting conditions from `starting` argument
## Warning in check_COD(dat = dat, grps = pars$grps, subs = pars$subs, COD_conv =
## pars$COD_conv, : COD balance is off by 4%
## 8x ->
## Using starting conditions from `starting` argument
## Warning in check_COD(dat = dat, grps = pars$grps, subs = pars$subs, COD_conv =
## pars$COD_conv, : COD balance is off by 4%
## 9x ->
## Using starting conditions from `starting` argument
## Warning in check_COD(dat = dat, grps = pars$grps, subs = pars$subs, COD_conv =
## pars$COD_conv, : COD balance is off by 4%
```

```
## and final run

## Using starting conditions from `starting` argument

## Warning in check_COD(dat = dat, grps = pars$grps, subs = pars$subs, COD_conv =
## pars$COD_conv, : COD balance is off by 4%

##    user  system elapsed
##   2.954   0.000   2.954
```

And 10 year with variable temperature.

```
system.time(
abmneo(
  storage = stor_ser_ref,
  365,
  inf_pars = inf_ref,
  grp_pars = grp_ref,
  sub_pars = sub_ref,
  chem_pars = chem_ref,
  var_pars = list(var = temp_dat),
  startup = 9
)
)
```

```
##
## Startup run 1x ->

## Warning in empty_store(y, resid_mass = pars$resid_mass, resid_enrich =
## pars$resid_enrich, : Emptying skipped.
## Warning in empty_store(y, resid_mass = pars$resid_mass, resid_enrich =
## pars$resid_enrich, : Emptying skipped.
## Warning in empty_store(y, resid_mass = pars$resid_mass, resid_enrich =
## pars$resid_enrich, : Emptying skipped.
## Warning in empty_store(y, resid_mass = pars$resid_mass, resid_enrich =
## pars$resid_enrich, : Emptying skipped.

## 2x ->

## Using starting conditions from `starting` argument

## Warning in empty_store(y, resid_mass = pars$resid_mass, resid_enrich =
## pars$resid_enrich, : Emptying skipped.
## Warning in empty_store(y, resid_mass = pars$resid_mass, resid_enrich =
## pars$resid_enrich, : Emptying skipped.
## Warning in empty_store(y, resid_mass = pars$resid_mass, resid_enrich =
## pars$resid_enrich, : Emptying skipped.
## Warning in empty_store(y, resid_mass = pars$resid_mass, resid_enrich =
## pars$resid_enrich, : Emptying skipped.

## Warning in check_COD(dat = dat, grps = pars$grps, subs = pars$subs, COD_conv =
## pars$COD_conv, : COD balance is off by 1.6%

## 3x ->

## Using starting conditions from `starting` argument

## Warning in empty_store(y, resid_mass = pars$resid_mass, resid_enrich =
## pars$resid_enrich, : Emptying skipped.

## Warning in empty_store(y, resid_mass = pars$resid_mass, resid_enrich =
```

```
## pars$resid_enrich, : Emptying skipped.
## Warning in empty_store(y, resid_mass = pars$resid_mass, resid_enrich =
## pars$resid_enrich, : Emptying skipped.
## Warning in empty_store(y, resid_mass = pars$resid_mass, resid_enrich =
## pars$resid_enrich, : Emptying skipped.

## Warning in check_COD(dat = dat, grps = pars$grps, subs = pars$subs, COD_conv =
## pars$COD_conv, : COD balance is off by 1.6%

## 4x ->

## Using starting conditions from `starting` argument

## Warning in empty_store(y, resid_mass = pars$resid_mass, resid_enrich =
## pars$resid_enrich, : Emptying skipped.

## Warning in empty_store(y, resid_mass = pars$resid_mass, resid_enrich =
## pars$resid_enrich, : Emptying skipped.
## Warning in empty_store(y, resid_mass = pars$resid_mass, resid_enrich =
## pars$resid_enrich, : Emptying skipped.
## Warning in empty_store(y, resid_mass = pars$resid_mass, resid_enrich =
## pars$resid_enrich, : Emptying skipped.

## Warning in check_COD(dat = dat, grps = pars$grps, subs = pars$subs, COD_conv =
## pars$COD_conv, : COD balance is off by 1.6%

## 5x ->

## Using starting conditions from `starting` argument

## Warning in empty_store(y, resid_mass = pars$resid_mass, resid_enrich =
## pars$resid_enrich, : Emptying skipped.

## Warning in empty_store(y, resid_mass = pars$resid_mass, resid_enrich =
## pars$resid_enrich, : Emptying skipped.
## Warning in empty_store(y, resid_mass = pars$resid_mass, resid_enrich =
## pars$resid_enrich, : Emptying skipped.
## Warning in empty_store(y, resid_mass = pars$resid_mass, resid_enrich =
## pars$resid_enrich, : Emptying skipped.

## Warning in check_COD(dat = dat, grps = pars$grps, subs = pars$subs, COD_conv =
## pars$COD_conv, : COD balance is off by 1.6%

## 6x ->

## Using starting conditions from `starting` argument

## Warning in empty_store(y, resid_mass = pars$resid_mass, resid_enrich =
## pars$resid_enrich, : Emptying skipped.

## Warning in empty_store(y, resid_mass = pars$resid_mass, resid_enrich =
## pars$resid_enrich, : Emptying skipped.
## Warning in empty_store(y, resid_mass = pars$resid_mass, resid_enrich =
## pars$resid_enrich, : Emptying skipped.
## Warning in empty_store(y, resid_mass = pars$resid_mass, resid_enrich =
## pars$resid_enrich, : Emptying skipped.

## Warning in check_COD(dat = dat, grps = pars$grps, subs = pars$subs, COD_conv =
## pars$COD_conv, : COD balance is off by 1.6%

## 7x ->

## Using starting conditions from `starting` argument
```

```
## Warning in empty_store(y, resid_mass = pars$resid_mass, resid_enrich =
## pars$resid_enrich, : Emptying skipped.

## Warning in empty_store(y, resid_mass = pars$resid_mass, resid_enrich =
## pars$resid_enrich, : Emptying skipped.
## Warning in empty_store(y, resid_mass = pars$resid_mass, resid_enrich =
## pars$resid_enrich, : Emptying skipped.
## Warning in empty_store(y, resid_mass = pars$resid_mass, resid_enrich =
## pars$resid_enrich, : Emptying skipped.

## Warning in check_COD(dat = dat, grps = pars$grps, subs = pars$subs, COD_conv =
## pars$COD_conv, : COD balance is off by 1.6%

## 8x ->

## Using starting conditions from `starting` argument

## Warning in empty_store(y, resid_mass = pars$resid_mass, resid_enrich =
## pars$resid_enrich, : Emptying skipped.

## Warning in empty_store(y, resid_mass = pars$resid_mass, resid_enrich =
## pars$resid_enrich, : Emptying skipped.
## Warning in empty_store(y, resid_mass = pars$resid_mass, resid_enrich =
## pars$resid_enrich, : Emptying skipped.
## Warning in empty_store(y, resid_mass = pars$resid_mass, resid_enrich =
## pars$resid_enrich, : Emptying skipped.

## Warning in check_COD(dat = dat, grps = pars$grps, subs = pars$subs, COD_conv =
## pars$COD_conv, : COD balance is off by 1.6%

## 9x ->

## Using starting conditions from `starting` argument

## Warning in empty_store(y, resid_mass = pars$resid_mass, resid_enrich =
## pars$resid_enrich, : Emptying skipped.

## Warning in empty_store(y, resid_mass = pars$resid_mass, resid_enrich =
## pars$resid_enrich, : Emptying skipped.
## Warning in empty_store(y, resid_mass = pars$resid_mass, resid_enrich =
## pars$resid_enrich, : Emptying skipped.
## Warning in empty_store(y, resid_mass = pars$resid_mass, resid_enrich =
## pars$resid_enrich, : Emptying skipped.

## Warning in check_COD(dat = dat, grps = pars$grps, subs = pars$subs, COD_conv =
## pars$COD_conv, : COD balance is off by 1.6%

## and final run

## Using starting conditions from `starting` argument

## Warning in empty_store(y, resid_mass = pars$resid_mass, resid_enrich =
## pars$resid_enrich, : Emptying skipped.

## Warning in empty_store(y, resid_mass = pars$resid_mass, resid_enrich =
## pars$resid_enrich, : Emptying skipped.
## Warning in empty_store(y, resid_mass = pars$resid_mass, resid_enrich =
## pars$resid_enrich, : Emptying skipped.
## Warning in empty_store(y, resid_mass = pars$resid_mass, resid_enrich =
## pars$resid_enrich, : Emptying skipped.

## Warning in check_COD(dat = dat, grps = pars$grps, subs = pars$subs, COD_conv =
```

```
## pars$COD_conv, : COD balance is off by 1.6%
```

```
##    user  system elapsed
##   2.501   0.000   2.500
```

Not sure of ABM package times but this stuff seems pretty fast.

NEW Note on COD balance

It sometimes indicates some problems! Could be real (means programming problem) or in the balance calculation but certainly needs to be sorted out.

Other functionality

Other functionality not shown includes: * CO2 production or consumption from fermentation

Details on package size

Here are file sizes.

First, current ABM master.

```
cat(system('wc -l ../../ABM/R/*.R ../../ABM/src/*.cpp', intern = TRUE), sep = '\n')
```

```
##      5 ../../ABM/R/Arrh_func.R
##     60 ../../ABM/R/CTM.R
##     31 ../../ABM/R/H2SO4_titrat.R
##     23 ../../ABM/R/RcppExports.R
##    570 ../../ABM/R/abm.R
##    148 ../../ABM/R/abm_regular.R
##    262 ../../ABM/R/abm_variable.R
##     13 ../../ABM/R/checkGrpNames.R
##     17 ../../ABM/R/coverfun.R
##     24 ../../ABM/R/doy.R
##     50 ../../ABM/R/emptyStore.R
##     47 ../../ABM/R/et.R
##     15 ../../ABM/R/graze_fun.R
##     99 ../../ABM/R/hard_pars.R
##      3 ../../ABM/R/logistic.R
##     26 ../../ABM/R/makeConcFunc.R
##     20 ../../ABM/R/makeTimeFunc.R
##     28 ../../ABM/R/makeXaFreshFunc.R
##     19 ../../ABM/R/pars_indices.R
##    199 ../../ABM/R/predFerm.R
##     81 ../../ABM/R/readFormula.R
##     86 ../../ABM/R/stoich.R
##    114 ../../ABM/R/stoich_species.R
##     10 ../../ABM/R/tempsC2K.R
##     16 ../../ABM/src/Arrh_func_cpp.cpp
##     42 ../../ABM/src/CTM_cpp.cpp
##     95 ../../ABM/src/RcppExports.cpp
##     12 ../../ABM/src/call_int_cpp.cpp
##     10 ../../ABM/src/extract_xa_cpp.cpp
##    563 ../../ABM/src/rates_cpp.cpp
##   2688 total
```

Then abmneo.

```
cat(system('wc -l ../R/*.R', intern = TRUE), sep = '\n')
```

```
##   194 ../R/core.R
##    96 ../R/misc.R
##   383 ../R/pars.R
##   168 ../R/post.R
##    28 ../R/prep.R
##    91 ../R/rates.R
##    64 ../R/startup.R
##    68 ../R/stoich.R
##   233 ../R/timing.R
##  1325 total
```

To be fair, I am sure ABM has some unused code. But still. . .

Look at rates() in particular.

```
cat(system('wc -l ../../ABM/src/rates_cpp.cpp ../R/rates.R', intern = TRUE), sep = '\n')
```

```
##   563 ../../ABM/src/rates_cpp.cpp
##    91 ../R/rates.R
##   654 total
```

That `rates.R` file includes some other functions. Here is the `rates()` function, around 50 lines of code that used to call no other abmneo functions. Now it does call an inhibition function. . . :

```
print(rates)
```

```
## function(t, y, parms) {
##
##   # Short name for parms to make indexing in code below simpler
##   p <- parms
##
##   # Update any inhibition effects
##   ## NTS: yc <- y[intersect(names(y), gsub('_conc', '', rownames(ic0)))] / y['slurry_mass']
##   qhat <- update_inhib(p, y)
##
##   # Initialize vectors with derivative components, all with same order of y elements
##   inflow <- metab <- hyferm <- resp <- 0 * y
##
##   # Inflow from slurry addition
##   inflow[names(p$conc_fresh)] <- p$conc_fresh * p$slurry_prod_rate
##   inflow[c('slurry_mass', 'slurry_load')] <- 1 * p$slurry_prod_rate
##   inflow['COD_load'] <- sum(inflow[names(p$conc_fresh)])
##
##   # Substrate matrix for utilization rate
##   sm <- matrix(y[rownames(p$mstoich)],
##                nrow = nrow(p$mstoich),
##                ncol = ncol(p$mstoich),
##                byrow = FALSE)
##
##   # Names for debugging
##   #dimnames(sm) <- dimnames(p$mstoich)
##
##   # Monod term
##   monod <- sm / (sm + y['slurry_mass'] * p$ksmat)
```

```
##   # Force to 1 for non-substrates
##   monod[p$mstoich >= 0] <- 1
##   # Product across multiple substrates
##   monodprod <- apply(monod, 2, prod)
##
##   # Utilization rate
##   rut <- qhat * monodprod * y[p$grps] / y['slurry_mass']
##
##   # And consumption, growth, production all in one matrix operation
##   metab[rownames(p$mstoich)] <- p$mstoich %*% rut * y['slurry_mass']
##
##   # Convert CH4 from g COD / d to g C / d
##   metab['CH4'] <- metab['CH4'] / p$COD_conv['CH4']
##
##   # Hydrolysis and fermentation of particulate substrates
##   # Consumption and production all in one line, including arbitrary products based on specified ferm
##   hyferm[rownames(p$fstoich)] <- p$fstoich %*% (p$alpha[p$subs] * y[p$subs])
##
##   # Surface respiration: aerobic oxidation limited by O2 surface flux
##   # Monod term on VFA prevents negative values at low concentrations (ks = 0.05 g COD/kg)
##   if (p$has_resp) {
##     resp_rate <- p$O2_flux * p$area * y['VFA'] / (y['VFA'] + 0.05 * y['slurry_mass'])
##     resp[names(p$rstoich)] <- resp_rate * p$rstoich
##   }
##
##   # Add vectors to get derivatives
##   # All elements in g/d as COD except
##   #   * slurry_mass (kg/d as fresh slurry mass)
##   #   * CH4 (g/d as CH4-C)
##   #   * user-defined solutes other than VFA (as C, N, or S as described in documentation)
##   ders <- inflow + metab + hyferm + resp
##
##   return(list(ders, c(CH4_emis_rate = metab[['CH4']], temp_C = p$temp_C, pH = p$pH)))
##
## }
## <environment: namespace:abmneo>
```